KV-Cache, Attention, and Context Compression

Reducing the Memory and Compute of Long-Context Large-Language-Model Inference

A monograph-style survey with derivations, algorithms, capability tables, risk analysis and a priority assessment

Paper 3 of the LLM Efficiency Series

Literature and open-model configurations surveyed through October 2026

Status of the numbers. Quantitative claims are tagged by provenance: values taken from a named source are cited; values computed in this document from stated formulas are marked "derived"; round figures that are typical but not tied to one source are marked "illustrative". Figures drawn by the document itself from formulas are synthetic and say so in their captions.

29 October 2026

Abstract

The key–value (KV) cache is the dominant consumer of accelerator memory and memory bandwidth in long-context transformer serving. This monograph surveys the techniques that shrink it or avoid reading it. We derive the size of the cache from first principles, show with a roofline argument why autoregressive decoding is memory-bound, and then organise the field into families: architectural reduction (multi-query, grouped-query and multi-head latent attention, cross-layer sharing, sliding-window and local–global patterns, hybrids with state-space and linear-attention layers); token eviction and query-aware selection; trainable sparse attention (native sparse attention, DeepSeek Sparse Attention, block attention and the compressed-sparse designs of DeepSeek-V4); merging, low-rank and learned-projection compression, including conversion of existing models to latent attention; paging, prefix reuse, offloading and disaggregated serving; and prompt and context compression, from gist tokens to context folding for agents. A mathematical chapter collects the results that explain why these methods work and where they must fail: error bounds for eviction, rate–distortion and low-rank optimality, streaming-algorithm and hashing connections, and communication-complexity lower bounds that show query-agnostic compression cannot be lossless. We then review evaluation methodology, tabulate what is achievable today with sourced numbers, catalogue the risks (silent retrieval failure, prefix-sharing breakage, reasoning-trace degradation, safety-text eviction, adversarial interaction, nondeterminism and hardware coupling), provide a practical decision guide, and give a transparent, rubric-based, explicitly subjective priority estimate for each sub-topic. The central conclusion is that the field has moved from post-hoc eviction of a trained dense cache toward architectures that are trained to have a small cache, with serving-system techniques as the stable layer beneath them.

Contents

Abstract ii

Executive Summary viii

1 Introduction and Motivation 1

1.1 Why context length became a systems problem . . . 1
1.2 What is being cached, and why . . . 1
1.3 Scope . . . 2
1.4 Conventions and provenance tags . . . 3
1.5 A compressed history . . . 3
1.6 Contributions and structure . . . 4

2 Mathematical Foundations 5

2.1 Attention and the cache . . . 5
2.2 Cache size . . . 5
2.3 Compute, bandwidth and the roofline . . . 7
2.4 Sparsity of attention weights . . . 9
2.5 Low-rank structure of keys and values . . . 11
2.6 Attention as a kernel smoother . . . 12

3 A Taxonomy of Cache and Context Reduction 14

3.1 Five axes . . . 14
3.2 A tree of methods . . . 15
3.3 Composability . . . 15
3.4 Cost-model view: what each family buys . . . 16

4 Architectural Cache Reduction I: Heads, Latents and Layers 18

4.1 From multi-head to grouped-query attention . . . 18
4.2 Multi-head latent attention . . . 19
4.3 Other channel-compressing designs . . . 21
4.4 Cross-layer sharing . . . 22
4.5 Worked example: assembling a small-cache dense model . . . 23

5 Architectural Cache Reduction II: Windows, Recurrence and Hybrids 24

5.1 Sliding-window and local–global attention 24
5.2 State-space models and linear attention 25
5.3 Hybrids 27
5.4 Surveying the frontier 29

6 Token Eviction and Selection **31**
6.1 Problem formulation 31
6.2 Attention sinks and StreamingLLM 32
6.3 Accumulated-score methods 33
6.4 Prefill-time compression 34
6.5 Adaptive and head-specialised policies 36
6.6 Query-aware selection: keep everything, read little 37
6.7 Decode-time compression for reasoning traces 38
6.8 Learned eviction and merging-by-gating 39
6.9 Comparison 39
6.10 Engineering notes 39

7 Trainable Sparse Attention **41**
7.1 Design requirements 41
7.2 Native sparse attention (NSA) 41
7.3 DeepSeek Sparse Attention and the lightning indexer 43
7.4 Mixture of block attention (MoBA) 44
7.5 DeepSeek-V4: compressed sparse and heavily compressed attention 45
7.6 Training-free dynamic sparse attention for prefill 46
7.7 Comparison of trainable sparse designs 46
7.8 Why training the selector helps: a simple model 46

8 Merging, Low-Rank Compression, Model Conversion and the Quantization Interface **48**
8.1 KV merging and token merging 48
8.2 Low-rank key–value compression 49
8.3 Converting existing models to latent attention 51
8.4 Linearisation as conversion 51
8.5 Interface with quantization 52
8.6 Summary table 54

9 Memory Management and Serving Systems **55**
9.1 Exact attention kernels: the FlashAttention lineage 55

9.2 Paged memory management . . . 56
9.3 Prefix caching and shared computation . . . 57
9.4 Chunked prefill and scheduling . . . 59
9.5 Tiered storage and offloading . . . 59
9.6 Disaggregated prefill and decode . . . 60
9.7 Parallelism and data layout for long contexts . . . 61
9.8 Batching, mixture-of-experts and the cache-bound regime . . . 61
9.9 Speculative decoding and the cache (adjacent method) . . . 61
9.10 Kernel support for compressed and sparse caches . . . 62

10 Prompt and Context Compression, Memory, Agents and Retrieval 64
10.1 A taxonomy of input-side compression . . . 64
10.2 Hard prompt compression: the LLMLingua family . . . 64
10.3 Soft compression: gist tokens, AutoCompressor, ICAE . . . 65
10.4 Recurrent and external memory . . . 66
10.5 Summary-based memory and agent context management . . . 67
10.6 Retrieval versus long context . . . 69
10.7 Positional encoding as it bears on the cache . . . 70

11 Theory: Error Bounds, Sketching, Lower Bounds and Information 72
11.1 Error bounds for eviction and perturbation . . . 72
11.2 Heavy hitters and streaming algorithms . . . 73
11.3 Hashing and sampling: Reformer, HyperAttention, MagicPIG . . . 74
11.4 Lower bounds . . . 76
11.5 Rate–distortion view of KV compression . . . 77
11.6 The information bottleneck and what to keep . . . 78
11.7 Summary of theoretical statements . . . 79

12 Evaluation Methodology 80
12.1 What must be measured . . . 80
12.2 Needle-in-a-haystack and why it is not enough . . . 80
12.3 Beyond NIAH: structured long-context benchmarks . . . 81
12.4 Cache-specific benchmarks and findings . . . 81
12.5 Evaluating long-generation reasoning . . . 82
12.6 Evaluating agentic, multi-turn and shared-prefix use . . . 83
12.7 Mechanistic and cheap diagnostics . . . 83
12.8 A layered protocol . . . 84

12.9 Benchmark overfitting and the Goodhart problem 84

13 Recent Advances Timeline and Current Capabilities 87

13.1 Timeline, 2022 to October 2026 87

13.2 What is achievable today 89

14 Pros, Cons and Risks 94

14.1 Pros and cons by method family 94

14.2 Risk register 95

14.3 Risk by family 99

15 Practical Decision Guide 101

15.1 Decision flowchart 101

15.2 Scenario tables 101

15.3 From bottleneck to lever 102

15.4 Choosing an open model: what the cache says 102

15.5 Go / no-go checklist for a lossy cache method 104

16 Estimated Priority in the LLM Engineering Community 106

16.1 What "priority" means 106

16.2 Rubric 106

16.3 Scores and ranking 107

16.4 Rationale by sub-topic 107

16.5 Final ranking in tiers, with confidence 111

16.6 Adjacent topics not scored 111

17 Open Problems and Research Directions 112

17.1 Evaluation and measurement 112

17.2 Theory 112

17.3 Architecture 113

17.4 Algorithms for eviction, selection and merging 113

17.5 Systems 114

17.6 Joint compression 114

17.7 Long-context capability itself 115

17.8 Predictions (author's expectations, not findings) 115

18 Conclusions 116

A Notation 118

B Additional Derivations and Worked Examples **120**

B.1 A numerical check of single-token removal 120

B.2 The sink logit equals a zero-value key 120

B.3 Decode-time latency: the batch crossover 120

B.4 Chunkwise-parallel form of a gated linear recurrence 121

B.5 Why MLA absorption preserves the output exactly 121

B.6 Why the expected-mass scoring rule is optimal for additive loss 121

B.7 Break-even of load versus recompute for several models 122

B.8 Expected tokens per speculative step 122

C Glossary **123**

Executive Summary

This monograph asks one question: *how can a language model attend over hundreds of thousands or millions of tokens without paying linear memory and linear memory bandwidth per generated token for all of them?* The answer assembled in the following chapters has seven parts. Each is stated here with its supporting evidence and its caveat; later chapters provide the derivations and the citations.

Finding 1: the cache, not the weights, sets the cost of long contexts

For a transformer with L layers, H_{kv} key–value heads of width d_h, and b bytes per element, the cache costs $2LH_{\text{kv}}d_hb$ bytes *per token* per sequence. For a Llama-3-8B-class configuration (32 layers, 8 KV heads, head width 128, bf16) this is 128 KiB per token, so one 128K-token sequence needs 16 GiB, as large as the bf16 weights themselves (derived, Section 2.2). Decode-time attention has an arithmetic intensity of only about g FLOP/byte for a group of g query heads sharing one KV head, against a hardware ridge point of a few hundred, so decoding is bandwidth-bound and its latency is proportional to the bytes of cache read.

Finding 2: architecture now does most of the work

Between 2019 and 2026 the dominant strategy shifted from *compressing a trained dense cache* to *training a model that has a small cache*. Grouped-query attention [3] is the baseline of nearly all open dense models; multi-head latent attention (MLA) [4] caches 576 values per token per layer in DeepSeek-V3 instead of tens of thousands [5]; local–global patterns are used by Gemma 3 [16] and gpt-oss [17]; and hybrids in which most layers are linear attention or state-space layers are used by Qwen3-Next, Kimi Linear, and the Nemotron-H and Nemotron 3 families [31–33, 38]. Using the public configuration files we compute that Qwen3-Next caches about 24 KiB/token (12 full-attention layers) and Kimi Linear about 8 KiB/token (7 MLA layers), against 144 KiB/token for the dense GQA model Qwen3-8B (derived, Table 2.1).

Finding 3: trainable sparse attention is the new frontier of dense-quality long context

Native sparse attention [73], DeepSeek Sparse Attention (DSA) with its lightning indexer [6], and block attention such as MoBA [74] train the model to attend to a selected subset, removing the train–inference mismatch of post-hoc eviction. DeepSeek-V4 combines 4× compressed sparse attention with 128× heavily compressed attention and reports about 10% of the V3.2 KV cache and 27% of its single-token FLOPs at one million tokens for V4-Pro [7]. These are self-reported numbers for one family; independent replication at scale is thin.

Finding 4: post-hoc eviction works on easy benchmarks and fails on hard ones

Training-free eviction (StreamingLLM, H2O, SnapKV, PyramidKV and relatives) retains most of the accuracy of single-query retrieval and summarisation tasks at 10–25% cache

budgets in the original papers, but there is a consistent pattern of weakness on multi-turn reuse, multi-hop reasoning, long chain-of-thought generation and queries unknown at compression time [70, 166–168]. A communication-complexity argument (Section 11.4) shows this is not an implementation accident: no query-agnostic compression of n tokens into $o(n)$ bits can answer all retrieval queries.

Finding 5: the serving layer is the stable, high-return layer

Paged memory management [95], prefix caching and radix trees [97], chunked prefill [102], disaggregated prefill/decode [103–105] and tiered offload [107, 109] are lossless with respect to model outputs (up to floating-point nondeterminism) and are now default features of production engines. Their benefit is utilisation and reuse rather than a smaller per-token footprint, and it compounds with every architectural reduction above.

Finding 6: prompt and context compression is moving from text to agent policy

Hard-prompt compression (LLMLingua family), soft compression (gist tokens, AutoCompressor, ICAE) and recurrent memory give 2–20× shorter inputs on benchmarks in the original papers, but with task-dependent fidelity loss. For agents, the active research direction is to *learn when to fold, summarise or evict* with reinforcement learning (context folding [144]) or to evict by deterministic, typed policies [145].

Finding 7: evaluation is the binding constraint

Needle-in-a-haystack saturates and hides failures; RULER, HELMET, NoLiMa, BABILong, SCBench and diagnostic benchmarks such as KVDiagnosis expose them [157, 162–164, 166, 168]. A MiniMax team report describes a hybrid linear/full attention design that matched full attention on standard benchmarks yet showed deficits on complex multi-hop reasoning at scale, and the team shipped full attention for M2 [36]. Evidence of this kind is anecdotal but important; it argues for evaluating every compression method on long-generation, multi-hop and multi-turn workloads before deployment.

Priority estimate. Chapter 16 scores sixteen sub-topics on a stated five-criterion rubric (adoption, research momentum, hardware tailwind, remaining benefit, risk-adjusted safety). In the author's subjective judgment, the top tier is prefix caching, MLA-class latent architectures, and paging with chunked prefill (the first two with high confidence); a second tier of KV quantization, disaggregation, GQA as the settled baseline, local–global patterns and tiered offload follows; hybrid linear/SSM architectures, agent context management and trainable sparse attention have the highest upside but low-confidence ranks; and query-aware selection, prompt compression, post-hoc low-rank compression, training-free eviction and KV merging are conditional or niche for general production use, despite training-free eviction having the largest research literature. These are judgments, not measurements; the chapter reports how the ranking shifts under alternative weightings.

How to use this document. Practitioners deciding what to deploy should read Chapters 2, 12, 14 and 15. Researchers should read Chapters 11 and 17. Chapters 4 through 10 are the method-family chapters and can be read independently after the taxonomy in Chapter 3.

Chapter 1

Introduction and Motivation

1.1 Why context length became a systems problem

Until about 2022 a typical deployed language model read a few thousand tokens. A prompt of 2K–4K tokens, a model of a few billion parameters and a modest batch fit comfortably on one accelerator, and the engineering effort went into the weights: sharding them, quantizing them, scheduling their matrix multiplications. Three developments changed the picture.

First, *context windows grew by two orders of magnitude.* Open and proprietary models now advertise windows of 128K, 256K, one million and, in a few cases, more tokens [7, 41–43]. Second, the dominant *use cases* consume that context. Retrieval-augmented pipelines paste tens of documents into the prompt [191, 192]. Coding assistants load whole repositories. Agents run for hundreds of tool calls, each appending observations to a transcript. Third, *reasoning models* generate long chains of thought before answering [8, 195]; a single response may be tens of thousands of tokens, all of which must be attended to at every later step. Context is no longer only an input resource; the model itself manufactures it.

The cost of a long context is not primarily the cost of reading it once. Prefill is a dense, compute-bound computation that GPUs perform efficiently, and its quadratic attention component is addressed by the FlashAttention lineage [110–112]. The recurring, dominant cost is in *decoding*: every new token must read the keys and values of every previous token at every layer. If the model has produced or ingested n tokens, one decode step moves $\Theta(n)$ bytes of cache through the memory hierarchy while doing $\Theta(n)$ floating-point operations, an arithmetic intensity so low that the accelerator's matrix units idle. Memory *capacity* compounds the problem: the cache of each in-flight sequence lives in high-bandwidth memory (HBM) for the whole life of the request, so the number of concurrent sequences, and therefore throughput per dollar, is capped by cache size.

The two bills of the KV cache

Capacity bill: bytes of HBM held per sequence for the whole generation; limits concurrent batch size. **Bandwidth bill:** bytes of cache read per generated token; limits tokens per second per sequence and, at large batch, total throughput. Methods differ in which bill they reduce. Eviction reduces both; quantization reduces both by a constant factor; paging reduces waste in the first; offloading trades the first for the second on a slower link; sparse selection reduces the second and, unless the unselected entries are dropped, not the first.

1.2 What is being cached, and why

An autoregressive transformer computes, at each layer, queries, keys and values for each token position from the layer's input hidden states. The output at position t depends on the keys and

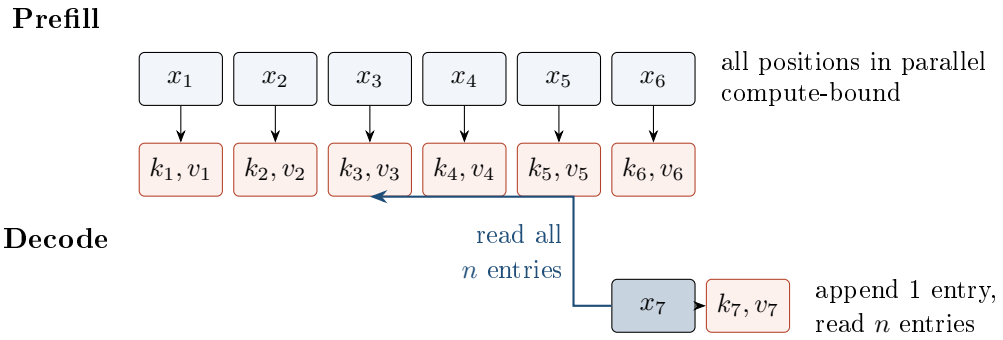


Figure 1.1: Schematic of the cache. Prefill computes and stores keys and values for all prompt positions; each decode step computes one new pair, appends it, and reads the whole stored history.

values at positions $\leq t$ of the same layer, but those depend only on the hidden states at those positions, which themselves depend only on the earlier tokens. Therefore, once computed, the keys and values of a prefix never change when new tokens are appended, and recomputing them at each step would cost $\mathcal{O}(n)$ forward passes' worth of linear-layer work per token. Storing them is the standard time–memory trade: $\mathcal{O}(n)$ memory buys a decode step whose linear-layer cost is independent of n.

The word "cache" is slightly misleading. It is not an optional accelerator that can be dropped with a performance penalty; for a transformer it is the *only representation of the past*. Discarding an entry changes the function the model computes. This is the root of the central tension of the field: every method in this monograph either (a) changes the model architecture so that a smaller past representation is sufficient, (b) changes which parts of the past the model reads, accepting an approximation, or (c) changes where and how the past is stored without changing the computation.

1.3 Scope

This paper covers memory and compute reduction for long-context *inference* with transformer-family models. In scope are the topics enumerated in the table of contents: cache sizing and roofline analysis; architectural reduction; token eviction and selection; trainable sparse attention; merging and low-rank compression; memory management and serving systems; prompt and context compression; positional encoding as it bears on the cache; retrieval versus long context; speculative decoding as an adjacent technique; theory; kernels; evaluation; risks and priorities.

Out of scope or treated only by cross-reference: (i) the numerics of quantization, which has its own paper in this series; we discuss KV quantization only where it interacts with eviction and low-rank structure; (ii) weight compression and pruning; (iii) training-time long-context methods beyond what is needed to explain inference-time behaviour; (iv) multimodal token reduction, except where the same algorithm applies.

1.4 Conventions and provenance tags

Because the field publishes aggressive efficiency claims and the author's own knowledge is bounded in time, every number in this document carries one of four provenance labels:

- **Cited**: reported by the named source, usually in its abstract or main results. These are the source's claims under the source's setup; we do not re-measure them.
- **Derived**: computed here from a formula and public configuration parameters (layer counts, head counts). These are exact given the stated assumptions but ignore allocator overheads, alignment and implementation-specific storage such as scale factors.
- **Illustrative**: round numbers used to build intuition, such as an accelerator with 3 TB/s of bandwidth. They are not measurements.
- **Estimate**: a subjective judgment of the author, used only in the priority and risk chapters.

Plots generated by the document itself from formulas are synthetic, and each caption says so.

Notation. Vectors are bold lowercase and matrices bold uppercase. n is sequence (context) length, B batch size, L number of layers, H_q and H_{kv} numbers of query and key–value heads, d_h head dimension, d model width, b bytes per stored element. Appendix A lists all symbols.

1.5 A compressed history

The efficiency of attention has been a theme since the transformer itself [1]. Surveys of the pre-LLM era catalogue dozens of "efficient transformer" variants [173]: sparse patterns (Sparse Transformers, Longformer, BigBird [11–13]), low-rank projections (Linformer [180]), hashing (Reformer [174]), kernel feature maps (Performer, linear transformers [21, 179]) and recurrence over segments (Transformer-XL, compressive transformers [138, 139]). Most of these did not displace dense softmax attention in frontier models, because quality at scale mattered more than asymptotics at the 2K-token regime then relevant, and because FlashAttention made dense attention fast enough in practice [110].

The modern cache-reduction story starts with multi-query attention in 2019 [2], which was largely ignored until memory-bound decoding of large models made it attractive, and was generalised to grouped-query attention in 2023 [3]. In 2023 a wave of training-free eviction methods appeared (Scissorhands, H2O, StreamingLLM, FastGen [46–48, 52]); simultaneously, PagedAttention [95] reframed the problem as memory management. In 2024 DeepSeek-V2 introduced MLA [4], hybrids with state-space layers reached production scale [29], and query-aware selection (Quest [61]) and head-specialised policies (DuoAttention [68]) appeared. In 2025 trainable sparse attention matured (NSA, MoBA, DSA [6, 73, 74]), hybrid linear attention entered open flagship models [33, 38], and the attention of the community turned to long-generation reasoning workloads [70]. In 2026, DeepSeek-V4 pushed compressed-plus-sparse attention to a one-million-token window as a default [7]. Chapter 13 gives a dated timeline.

1.6 Contributions and structure

The contributions of this monograph are fourfold. (1) A unified derivation of cache size and decode cost, with worked numbers for named models using public configurations. (2) A taxonomy that separates *where* compression acts (architecture, token, channel, precision, placement, input text) from *when* it is decided (training, prefill, decode) and *what it assumes* (query-agnostic or query-aware). (3) A mathematical chapter that collects, with proofs, the statements that explain the empirical landscape and bound what is possible. (4) A practical layer: capability tables, a risk register, a decision flowchart and a transparent priority assessment.

Reader caution: efficiency claims are conditional

Almost every number quoted in this paper is conditional on a model, a benchmark, a hardware platform and a baseline implementation. A "× speedup" relative to an unoptimised baseline is not comparable to one relative to a tuned FlashAttention or FlashInfer kernel. A "no accuracy loss" on a retrieval benchmark does not imply none on reasoning. We flag the conditioning variables wherever we know them and recommend treating unconditioned claims as hypotheses.

The next chapter begins with the mathematics that underlie everything else.

Chapter 2

Mathematical Foundations

This chapter fixes notation and derives the quantities that the rest of the paper manipulates: the size of the cache, the cost of a decode step, the sparsity of attention weights and the low-rank structure of keys and values. Proofs that are standard are given in full when short and sketched otherwise.

2.1 Attention and the cache

Consider one decoder layer with model width d, H_q query heads and H_{kv} key–value heads, each of width d_h. We assume H_q is a multiple of H_{kv} and write $g = H_q/H_{\mathrm{kv}}$ for the group size (the number of query heads sharing a key–value head). For hidden states $\mathbf{X} \in \mathbb{R}^{n\times d}$ the layer forms

$$\mathbf{Q}_i = \mathbf{X}\mathbf{W}_i^Q, \quad \mathbf{K}_j = \mathbf{X}\mathbf{W}_j^K, \quad \mathbf{V}_j = \mathbf{X}\mathbf{W}_j^V, \qquad i \in [H_q],\ j \in [H_{\mathrm{kv}}], \tag{2.1}$$

and head i attends using key–value head $j = \lceil i/g \rceil$:

$$\mathbf{O}_i = \mathrm{softmax}\Big(\tfrac{1}{\sqrt{d_h}}\mathbf{Q}_i\mathbf{K}_j^\top + \mathbf{M}\Big)\mathbf{V}_j, \tag{2.2}$$

where $\mathbf{M}$ is the causal mask. At decode time, with the prefix keys $\mathbf{K}_j \in \mathbb{R}^{n\times d_h}$ and values $\mathbf{V}_j$ stored, the new token contributes a query $\mathbf{q}_i \in \mathbb{R}^{d_h}$ and a new key–value pair that is appended, and

$$\mathbf{o}_i = \sum_{s=1}^{n} p_{i,s}\,\mathbf{v}_{j,s}, \qquad p_{i,s} = \frac{\exp(\mathbf{q}_i^\top \mathbf{k}_{j,s}/\sqrt{d_h})}{\sum_{u=1}^{n}\exp(\mathbf{q}_i^\top \mathbf{k}_{j,u}/\sqrt{d_h})}. \tag{2.3}$$

Multi-head attention (MHA) is $g = 1$, multi-query attention (MQA) is $H_{\mathrm{kv}} = 1$, and grouped-query attention (GQA) interpolates.

2.2 Cache size

Proposition 2.1 (Cache size)**.** *A transformer whose layer ℓ stores, per token, c_ℓ scalar elements of b bytes each, and that holds n tokens for each of B sequences, has a cache of*

$$S_{\mathrm{KV}} = B\,n\,b\sum_{\ell=1}^{L} c_\ell \quad \textit{bytes.} \tag{2.4}$$

For standard (MHA/MQA/GQA) attention $c_\ell = 2H_{\mathrm{kv}}d_h$, so $S_{\mathrm{KV}} = 2\,B\,n\,L\,H_{\mathrm{kv}}\,d_h\,b$.

Proof. Each layer stores one key vector and one value vector of width d_h per KV head per token, giving $2H_{\mathrm{kv}}d_h$ elements. Summing over layers and multiplying by tokens, sequences and bytes per element gives the formula. The general form simply replaces the per-layer constant. □

Table 2.1: Per-token cache size and long-context footprint for public model configurations, *derived* from the formulas above assuming 2 bytes per element. Configurations for Qwen3-8B, Qwen3-Next, gpt-oss-120b and Kimi Linear are from the public model repositories [17, 37–39]; the Llama and DeepSeek rows use published model-card/paper values [4, 5, 42]. Kimi Linear assumes 64 decoupled-RoPE dimensions on top of the configured latent rank of 512. Recurrent states of linear layers and the constant-size windows of local layers are listed separately and excluded from the per-token column. Allocator overheads and quantization scales are ignored.

Model (attention type)	Cached layers	Per layer per token	KiB/token	GiB at 128K	Reduction vs. MHA-32L
Llama-2-7B (MHA)	32	2·32·128	512	64.0†	1×
Llama-3-8B (GQA 4:1)	32	2·8·128	128	16.0	4×
Qwen3-8B (GQA 4:1)	36	2·8·128	144	18.0	3.6×
Llama-3-70B (GQA 8:1)	80	2·8·128	320	40.0	1.6×
Llama-3.1-405B (GQA 16:1)	126	2·8·128	504	63.0	1.0×
DeepSeek-V3 (MLA)	61	$512+64$	68.6	8.6	7.5×
gpt-oss-120b (GQA, 1:1 window)	18 global	2·8·64	36.0	4.5	14.2×
Qwen3-Next-80B-A3B (3:1 DeltaNet)	12 global	2·2·256	24.0	3.0	21.3×
Kimi-Linear-48B-A3B (3:1 KDA)	7 MLA	$512+64$	7.9	1.0	65×

†Illustrative only: Llama-2-7B was released with a 4K window; the 128K figure shows what MHA would cost at that length. The final column divides 512 KiB by the model's KiB/token; it compares cache per token, not model quality.

The per-token, per-sequence cost $2LH_{\text{kv}}d_h b$ is the quantity most often needed. It is independent of n and B and can be read off a model's configuration file.

Example 2.1 (Llama-3-8B-class configuration)**.** With $L = 32$, $H_{\text{kv}} = 8$, $d_h = 128$, $b = 2$ (bf16): $2 \cdot 32 \cdot 8 \cdot 128 \cdot 2 = 131{,}072$ bytes = 128 KiB per token. A single sequence of $n = 131{,}072$ tokens (128K) needs 128 KiB × 131,072 = 16 GiB. The bf16 weights of an 8.0B-parameter model occupy about 16 GB, so *one* long-context sequence needs as much memory as the whole model. For comparison, the MHA configuration of Llama-2-7B ($L = 32$, $H_{\text{kv}} = 32$, $d_h = 128$) costs 512 KiB per token, four times more (derived).

2.2.1 Heterogeneous layers

Equation (2.4) accommodates all the architectural variants of Chapters 4 and 5 by choosing c_ℓ and, for windowed layers, replacing n by an effective length:

$$S_{\text{KV}} = B\,b\Big[\sum_{\ell\in\mathcal{G}} c_\ell\, n + \sum_{\ell\in\mathcal{W}} c_\ell \min(n, w_\ell)\Big] + B\,S_{\text{state}}, \tag{2.5}$$

where $\mathcal{G}$ is the set of global-attention layers, $\mathcal{W}$ the set of sliding-window layers with window w_ℓ, and S_{state} the constant-size recurrent state of linear-attention or state-space layers, which does not grow with n. MLA layers have $c_\ell = d_c + d_h^R$, the latent dimension plus the decoupled positional-key dimension (Section 4.2).

Table 2.1 makes the architectural trend concrete: across seven years of design the per-token footprint of a frontier open model fell by roughly two orders of magnitude, with the biggest drops coming from replacing dense layers (hybrid) and from latent compression (MLA).

Example 2.2 (How many sequences fit?)**.** Take an 80 GB accelerator, reserve 10% for activations and fragmentation, and serve Llama-3-8B in bf16 (16 GB of weights). About $80-16-8 =$ 56 GB remain for cache. At 8K tokens per sequence (1.07 GB each) roughly 52 sequences fit;

at 128K tokens (17.2 GB each, using decimal gigabytes) only 3 fit (derived). The *throughput-optimal batch size at 128K is therefore three*, which is far below the batch size needed to amortise the weight reads (see Section 2.3). This is the capacity bill made concrete.

2.2.2 Tensor parallelism and replication

When tensor parallelism of degree T shards attention heads, a model with $H_{\text{kv}} < T$ KV heads must *replicate* KV heads across GPUs: each device holds $\max(1, H_{\text{kv}}/T)$ heads. Llama-3.1-405B has 8 KV heads and is commonly served with $T = 8$, so no replication occurs at $T = 8$, whereas at $T = 16$ every KV head is duplicated and the per-device saving from parallelism disappears for the cache. MLA's latent cache has no head dimension to shard and is typically replicated across tensor-parallel ranks, which is one reason latent-attention serving systems favour data-parallel attention with expert parallelism for the feed-forward layers. We return to this in Chapter 9.

2.3 Compute, bandwidth and the roofline

The roofline model [121] bounds the throughput of a kernel by the lesser of the machine's peak compute rate P (FLOP/s) and the product of its memory bandwidth β (bytes/s) and the kernel's arithmetic intensity I (FLOP per byte moved):

$$\text{attainable throughput} = \min\big(P,\ \beta I\big). \tag{2.6}$$

The *ridge point* $I^\star = P/\beta$ separates memory-bound ($I < I^\star$) from compute-bound kernels. For illustration, a current datacenter accelerator with $P \approx 10^{15}$ FLOP/s of dense bf16 and $\beta \approx 3.35 \times 10^{12}$ B/s has $I^\star \approx 300$ FLOP/B (round vendor-peak figures [123]; achieved rates are lower). Roofline analyses of LLM inference are surveyed in [120, 122].

2.3.1 Arithmetic intensity of decode attention

Proposition 2.2 (Decode-attention intensity)**.** *Consider one KV head with n cached tokens serving g query heads, one decode token, elements of b bytes. The attention computation performs $4gnd_h$ FLOPs and reads $2nd_hb$ bytes, so*

$$I_{\text{attn}} = \frac{2g}{b}\ \textit{FLOP/byte}, \tag{2.7}$$

independent of n and d_h. For bf16 ($b = 2$), $I_{\text{attn}} = g$.

Proof. Scoring requires g dot products of length d_h with each of n keys: $2gnd_h$ FLOPs. The weighted sum over values requires another $2gnd_h$. Total $4gnd_h$. The bytes read are the nd_h key and nd_h value elements, $2nd_hb$ (queries and outputs are $\mathcal{O}(gd_h)$ and negligible). Dividing gives $2g/b$. □

With $g = 1$ (MHA) the intensity is one FLOP per byte; with $g = 8$ it is eight. The hardware ridge point is of order 300. Hence decode attention uses between 0.3% and 3% of peak compute no matter how long the context is: it is thoroughly memory-bound, and its time is $2nd_hb\,H_{\text{kv}}L/\beta$ regardless of how many arithmetic units are available. This is why (i) MQA/GQA speed up

decoding almost linearly in g despite performing the same FLOPs, (ii) quantizing the cache speeds decoding almost linearly in the bit-width reduction, and (iii) *sparse selection that reads fewer entries* reduces time proportionally to the fraction read, whereas selection that reads all entries to compute a score (for instance exact top-k over all keys) does not.

Proposition 2.3 (MLA with weight absorption)**.** *In MLA decode with absorbed up-projection (Section 4.2), all H_q query heads attend to the same cached latent vector of width $d_c + d_h^R$ per token. The computation performs approximately $2H_q n(d_c + d_h^R) + 2H_q n d_c$ FLOPs and reads $n(d_c + d_h^R)b$ bytes, so*

$$I_{\text{MLA}} \approx \frac{2H_q(2d_c + d_h^R)}{(d_c + d_h^R)\, b};$$

with DeepSeek-V3 numbers ($H_q = 128$, $d_c = 512$, $d_h^R = 64$, $b = 2$) this is about 240 FLOP/B.

Proof. Absorption moves the key up-projection into the query and the value up-projection after the output sum, so scores are H_q dot products of length $d_c + d_h^R = 576$ per cached token ($2 \cdot 128 \cdot 576$ FLOPs) and the weighted sum uses the 512-wide latent ($2 \cdot 128 \cdot 512$ FLOPs); total $\approx 2.8 \times 10^5$ FLOPs per token. The bytes read are $576 \cdot 2 = 1152$ per token. The quotient is ≈ 242. □

Proposition 2.3 explains why MLA is more than a capacity trick: by sharing one latent among 128 heads it moves decode attention from the bandwidth-bound to nearly the compute-bound regime on current hardware, so its benefit is not only a smaller cache but also better hardware utilisation; DeepSeek's public kernel work for MLA decoding is built around this observation (the intensity figure itself is derived here, not quoted from a source).

2.3.2 Whole-step decode latency model

Let W be the bytes of weights read per decode step (for a dense model, all of them; for a mixture-of-experts model, the bytes of experts that the batch activates), $S_{\text{KV}}^{(1)} = n\, b \sum_\ell c_\ell$ the cache of one sequence, and β the bandwidth. Ignoring compute and communication, a decode step for a batch of B sequences takes

$$t_{\text{step}}(B) \approx \frac{W + B\, S_{\text{KV}}^{(1)}}{\beta}, \qquad \frac{t_{\text{step}}}{B} \approx \frac{S_{\text{KV}}^{(1)}}{\beta} + \frac{W}{B\beta}. \tag{2.8}$$

The first term of the per-token latency is the cache term and does not amortise with batching; the second is the weight term and does. The batch size at which the two are equal is $B^\star = W/S_{\text{KV}}^{(1)}$. For Llama-3-8B ($W \approx 16$ GB), $B^\star \approx 30$ at $n = 4$K, ≈ 3.7 at 32K and ≈ 0.9 at 128K (derived). Above $B^\star$ the system is cache-bound: throughput is asymptotically $\beta / S_{\text{KV}}^{(1)}$ sequences per second regardless of batch size, and *every* reduction of the cache converts one-for-one into throughput. Figure 2.1 plots the model.

2.3.3 Prefill and the quadratic term

Prefill of n tokens is dense matrix multiplication and is compute-bound. Per layer and per token the linear (projection and feed-forward) layers cost about $2 \times 12d^2 = 24d^2$ FLOPs for a standard block with a $4d$ hidden feed-forward width (mixture-of-experts models substitute

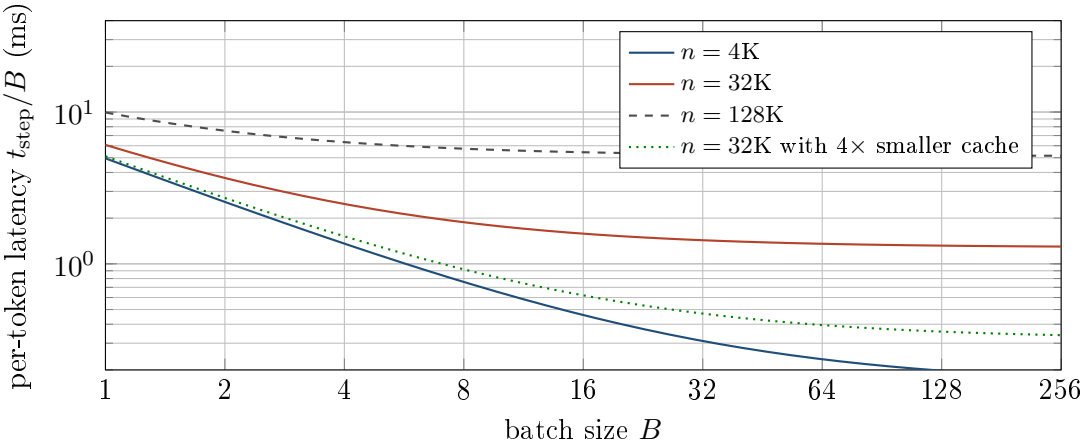


Figure 2.1: **Synthetic, from Equation (2.8).** Per-token decode latency for a Llama-3-8B-class model ($W = 16.06$ GB, 128 KiB/token) on an illustrative accelerator with $\beta = 3.35$ TB/s. The curves flatten at the cache term $S_{\text{KV}}^{(1)}/\beta$; the green curve shows that a 4× cache reduction lowers the asymptote 4×. Compute time and communication are ignored, so real latencies are higher.

active parameters), while causal attention costs on average $2nd_hH_q = 2nd$ FLOPs for the scores and values combined (average over positions $t \le n$ of $4td$). The two are equal at $n \approx 12d$; for $d = 4096$ the attention term overtakes the linear terms near 50K tokens. Beyond this length prefill cost grows quadratically in n and *time-to-first-token* becomes the dominant user-visible latency. Sparse prefill methods (Chapter 7) target this term; cache-reduction methods mostly target the decode bills.

2.4 Sparsity of attention weights

Almost every eviction and selection method rests on an empirical observation: for most heads and queries, a small subset of keys carries most of the softmax mass. We state two simple models that quantify when this is plausible and what it implies for the budget.

Proposition 2.4 (Mass outside a logit gap)**.** *Let the logits of a query against n keys be $s_1 \ge s_2 \ge \cdots \ge s_n$, and suppose the top k exceed the remaining $n-k$ by at least Δ, i.e., $s_k - s_{k+1} \ge \Delta$. Then the attention mass outside the top k satisfies*

$$\epsilon_k \le \frac{(n-k)e^{-\Delta}}{k + (n-k)e^{-\Delta}} \le \frac{n-k}{k} e^{-\Delta}. \tag{2.9}$$

Proof. Let $Z_{\text{top}} = \sum_{i \le k} e^{s_i} \ge k e^{s_k}$ and $Z_{\text{rest}} = \sum_{i > k} e^{s_i} \le (n-k)e^{s_{k+1}} \le (n-k)e^{s_k - \Delta}$. Then $\epsilon_k = Z_{\text{rest}}/(Z_{\text{top}} + Z_{\text{rest}})$ is increasing in Z_{rest} and decreasing in Z_{top}; substituting the extreme values and cancelling e^{s_k} gives the first inequality, and dropping $(n-k)e^{-\Delta}$ in the denominator gives the second. □

A gap of $\Delta = 10$ nats lets one discard $n = 10^5$ keys outside $k = 100$ at a mass cost below $10^3 e^{-10} \approx 0.05$ (derived), while a gap of $\Delta = 15$ gives $< 5 \times 10^{-4}$. The proposition also shows the cost: the required gap grows like $\ln(n/k)$, so sparsity at long context needs correspondingly sharp attention.

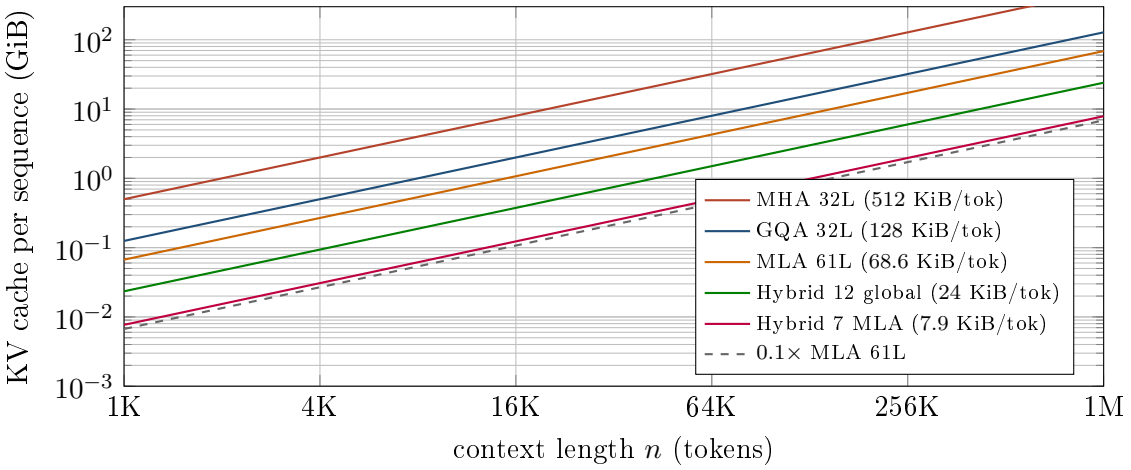


Figure 2.2: **Synthetic, from Equation (2.4).** Cache size per sequence versus context length for the per-token footprints of Table 2.1 (bf16, batch 1; the Hybrid curves ignore their small constant recurrent states). All curves are straight lines on log–log axes with unit slope; the architectural reductions shift them down. The dashed line, 10% of the 61-layer MLA footprint, corresponds to the V4-Pro ratio versus V3.2 reported in [7] applied to a derived V3-class baseline; it is illustrative, not V4's measured footprint.

Proposition 2.5 (Heavy-tailed weights have length-independent support)**.** *Suppose the sorted attention weights follow a power law $p_{(r)} = r^{-\alpha}/Z_n$ with $\alpha > 1$ and $Z_n = \sum_{r \le n} r^{-\alpha} \to \zeta(\alpha)$. Then the number of keys needed to capture mass $1 - \epsilon$ is $k(\epsilon) \approx \big((\alpha-1)\zeta(\alpha)\,\epsilon\big)^{-1/(\alpha-1)}$, independent of n. For $\alpha \le 1$, $k(\epsilon)$ grows with n (and is $\Theta(n)$ for $\alpha < 1$ at fixed ϵ).*

Proof. The tail mass beyond rank k is $\sum_{r>k} r^{-\alpha}/Z_n \approx \frac{k^{1-\alpha}}{(\alpha-1)\zeta(\alpha)}$ by the integral comparison. Setting this equal to ϵ and solving gives $k(\epsilon)$. For $\alpha \le 1$, Z_n diverges with n and the tail beyond any fixed k retains a mass fraction approaching one. □

If real attention behaved like a power law with $\alpha > 1$ for all heads, fixed-size caches would suffice at any context length. The empirical situation is mixed: some heads are sharply peaked (attention sinks, local heads), whereas retrieval heads [188] and induction heads [187] are peaked only *conditionally*, on the one token that happens to match the current query, which may be anywhere. The same head can be sparse on one query and diffuse on another. This is why *query-agnostic* caches lose information that *query-aware* selectors recover (Chapter 6).

Proposition 2.6 (Random-energy phase transition)**.** *If the logits are i.i.d. $\mathcal{N}(0, \sigma^2)$ across n keys, then as $n \to \infty$: if $\sigma < \sqrt{2 \ln n}$ the softmax mass is spread over $n^{1-\sigma^2/(2\ln n)}$ keys, a number that grows polynomially with n; if $\sigma > \sqrt{2 \ln n}$ it is concentrated on $\mathcal{O}(1)$ keys.*

Proof sketch. The number of keys with logit near σx is about $n e^{-x^2/2}$, and each carries weight proportional to $e^{\sigma^2 x}$; the total contribution of level x is $\exp(\ln n - x^2/2 + \sigma^2 x)$, maximised at $x = \sigma$ if that level is populated, i.e., $\ln n - \sigma^2/2 > 0$. In that case the dominant level contains $n e^{-\sigma^2/2}$ keys and the mass is spread. If $\sigma^2/2 > \ln n$, the maximum is at the edge of the populated region where the count is $\mathcal{O}(1)$ and the mass is concentrated on the extremes. This is Derrida's random energy model with $\beta = 1$ and logit scale σ. □

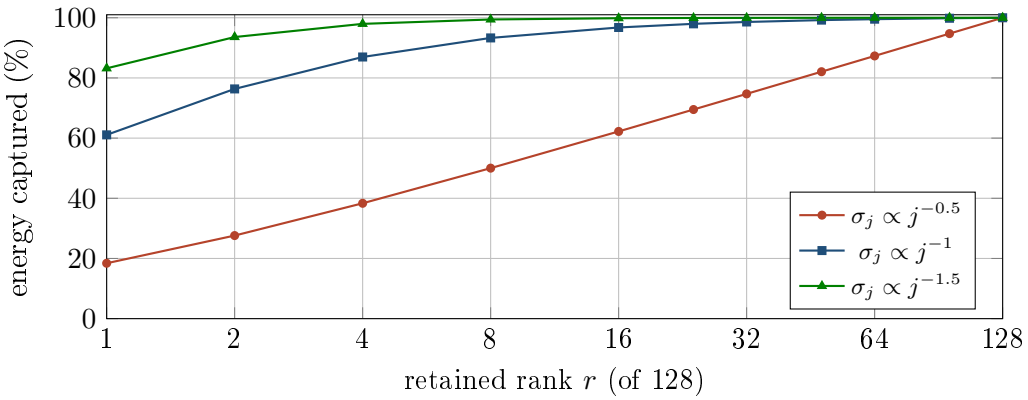


Figure 2.3: **Synthetic.** Fraction of squared Frobenius energy captured by the top r singular directions of a 128-dimensional head under three assumed power-law spectra $\sigma_j \propto j^{-\gamma}$. The point is the shape: if keys decay like $\gamma \geq 1$, a rank of 16–32 (12–25% of d_h) captures almost all energy; if spectra are flat, low-rank compression is lossy. Real spectra vary by layer, head, and model and must be measured [80, 81].

Proposition 2.6 is a null model, since real keys and queries are strongly structured, but it conveys a useful scaling: for $n = 10^5$ the critical logit scale is $\sigma = \sqrt{2 \ln 10^5} \approx 4.8$. Attention logits $\mathbf{q}^\top \mathbf{k}/\sqrt{d_h}$ with standard deviations of this order appear plausible in trained models' retrieval heads, while heads with small logit variance behave like averaging operators and cannot be sparsified. The practical lesson is that sparsity is a property of *heads and queries*, not of "attention" in general.

Takeaway: sparsity budgets must be per-head and per-query

Uniform budgets across heads and layers (as in early eviction methods) waste capacity on sharp heads and starve diffuse ones. Later methods allocate budgets per layer (PyramidKV [50]), per head (Ada-KV, HeadKV, DuoAttention [53, 54, 68]) or per decoding phase [71], which is exactly what the theory suggests.

2.5 Low-rank structure of keys and values

A different source of compressibility is that the vectors of the cache span a low-dimensional subspace. Let $\mathbf{K} \in \mathbb{R}^{n \times d_h}$ stack the keys of one head, with SVD $\mathbf{K} = \sum_j \sigma_j \mathbf{u}_j \mathbf{w}_j^\top$.

Theorem 2.1 (Eckart–Young–Mirsky [200, 201])**.** *The best rank-r approximation of $\mathbf{K}$ in Frobenius or spectral norm is $\mathbf{K}_r = \sum_{j \leq r} \sigma_j \mathbf{u}_j \mathbf{w}_j^\top$, with errors $\|\mathbf{K} - \mathbf{K}_r\|_F^2 = \sum_{j>r} \sigma_j^2$ and $\|\mathbf{K} - \mathbf{K}_r\|_2 = \sigma_{r+1}$.*

Storing $\mathbf{K}_r$ as the factors $\mathbf{U}_r \mathbf{\Sigma}_r \in \mathbb{R}^{n \times r}$ and $\mathbf{W}_r \in \mathbb{R}^{d_h \times r}$ costs $nr + d_h r$ numbers instead of nd_h, a ratio of r/d_h for $n \gg d_h$. The effect on attention logits for queries $\mathbf{Q}$ is bounded by $\left\|\mathbf{Q}(\mathbf{K} - \mathbf{K}_r)^\top\right\|_F \leq \|\mathbf{Q}\|_2 \, \|\mathbf{K} - \mathbf{K}_r\|_F$, but this bound ignores that queries are not isotropic; the optimal compression should be *query-aware*, a point developed in Section 8.2.

Empirically, key matrices of trained models tend to have faster-decaying spectra than value matrices, and the concatenation of all heads of a layer has stronger low-rank structure than any single head. Palu and the MLA-conversion line of work (Chapter 8) exploit exactly this [80–82].

MLA goes further by *training* the model so that the joint key–value space has low rank from the start [4].

2.6 Attention as a kernel smoother

Write $\tau = 1/\sqrt{d_h}$. Then

$$\exp(\tau\, \mathbf{q}^\top \mathbf{k}) = \underbrace{e^{\tau\|\mathbf{q}\|^2/2}}_{\text{row scaling}}\ \underbrace{e^{\tau\|\mathbf{k}\|^2/2}}_{\text{key weight}}\ \exp\Big(-\tfrac{\tau}{2}\,\|\mathbf{q}-\mathbf{k}\|^2\Big). \tag{2.10}$$

The first factor cancels in the softmax normalisation; the last is a Gaussian (RBF) kernel; the middle factor is a key-dependent weight that favours large-norm keys. Hence softmax attention is a Nadaraya–Watson kernel smoother with a Gaussian kernel and a key prior $\propto e^{\tau\|\mathbf{k}\|^2/2}$. Three consequences matter for this paper.

1. **Linear attention as feature maps.** If $k(\mathbf{q},\mathbf{k}) \approx \phi(\mathbf{q})^\top \phi(\mathbf{k})$ with $\phi : \mathbb{R}^{d_h} \to \mathbb{R}^m$, then

$$\mathbf{o} = \frac{\phi(\mathbf{q})^\top \sum_s \phi(\mathbf{k}_s)\mathbf{v}_s^\top}{\phi(\mathbf{q})^\top \sum_s \phi(\mathbf{k}_s)}$$

   and the sums are *constant-size running state* $\mathbf{S} = \sum_s \phi(\mathbf{k}_s)\mathbf{v}_s^\top \in \mathbb{R}^{m\times d_h}$ and $\mathbf{z} = \sum_s \phi(\mathbf{k}_s) \in \mathbb{R}^m$ [21]. Random-feature approximations such as Performer [179] approximate the softmax kernel; learned or fixed simple feature maps give linear transformers. The cache becomes $\mathcal{O}(md_h)$ per head, independent of n.
2. **Key norm as importance.** The factor $e^{\tau\|\mathbf{k}\|^2/2}$ explains why key norms correlate with attention mass and why simple norm-based eviction scores can be competitive in some models [58]; it also explains the attention-sink phenomenon from the other side (tokens with large-norm or special keys attract mass; see Chapter 6).
3. **Kernel locality.** For small $\tau\,\|\mathbf{q}-\mathbf{k}\|^2$ differences the Gaussian kernel is nearly flat, which is the diffuse-head regime; for large differences it is sharply peaked, the sparse regime. Per-head τ scaling and logit temperature therefore control compressibility, a design lever in some architectures (for example, scaled attention in long-context training).

Proposition 2.7 (Associative-memory capacity of a linear-attention state)**.** *Let a linear-attention layer store pairs* $(\mathbf{k}_s, \mathbf{v}_s)$ *in* $\mathbf{S} = \sum_s \mathbf{v}_s\mathbf{k}_s^\top \in \mathbb{R}^{d_h\times d_h}$ *(identity feature map, no normalisation) and read with* $\hat{\mathbf{v}} = \mathbf{S}\mathbf{q}$*. If the keys are orthonormal, retrieval of any stored value with* $\mathbf{q} = \mathbf{k}_s$ *is exact, and at most* d_h *pairs can be stored exactly. If* $T > d_h$ *unit-norm keys are stored, then for a stored key* $\mathbf{k}_s$*,* $\hat{\mathbf{v}} - \mathbf{v}_s = \sum_{u\neq s} \mathbf{v}_u\langle \mathbf{k}_u, \mathbf{k}_s\rangle$ *and, for random keys with* $\mathbb{E}\langle \mathbf{k}_u, \mathbf{k}_s\rangle^2 = 1/d_h$ *and unit-norm values,* $\mathbb{E}\,\|\hat{\mathbf{v}} - \mathbf{v}_s\|^2 \approx (T-1)/d_h$*.*

Proof. Linearity gives $\mathbf{S}\mathbf{k}_s = \mathbf{v}_s + \sum_{u\neq s} \mathbf{v}_u\langle \mathbf{k}_u, \mathbf{k}_s\rangle$. For orthonormal keys the sum vanishes. At most d_h vectors in $\mathbb{R}^{d_h}$ can be orthonormal. For independent random keys the cross terms are zero-mean and uncorrelated with variance $1/d_h$ each; with unit-norm values the expected squared norm of the sum is $(T-1)/d_h$. □

This simple calculation is the heart of why fixed-state recurrent layers lose recall at long range: the signal-to-interference ratio falls as d_h/T. The delta rule and gating (Chapter 5) are different ways of managing the interference, but none can beat the counting limit of Section 11.4.

Chapter summary

- Cache per token is $\sum_\ell c_\ell\, b$; for standard attention $2LH_{\text{kv}}d_h b$. Reduction acts on layer count, head count, head width, element width, or number of tokens.
- Decode attention has intensity $2g/b$ FLOP/B and is bandwidth-bound; latency per token approaches $S^{(1)}_{\text{KV}}/\beta$ at large batch, so cache reductions translate directly into throughput.
- Prefill attention becomes quadratic-dominant beyond $n \approx 12d$.
- Sparsity is conditional, per-head and per-query; low-rank structure is real but spectrum-dependent; linear attention trades growth for interference.

Chapter 3

A Taxonomy of Cache and Context Reduction

The literature on long-context efficiency is organised by this paper along five axes. The axes are not independent, but each answers a question a practitioner must answer before choosing a method.

3.1 Five axes

Axis 1: where does the reduction act? The cache is a tensor of shape (layers × KV heads × tokens × channels × bits). Each dimension has a family of methods:

Dimension	Reduction	Chapters
Layers	cross-layer sharing (CLA, YOCO), layer-wise mixing of attention with recurrence, depth merging	4, 5, 8
Heads	MQA, GQA, head-selective policies (retrieval vs. streaming heads)	4, 6
Tokens	eviction, selection, windowing, merging, trainable sparsity, prompt compression	6–10
Channels	low-rank projection, latent attention, SVD, partial-RoPE	4, 8
Bits	quantization (cross-referenced)	8
Placement	paging, prefix sharing, offload, disaggregation, network compression	9

Axis 2: when is the reduction decided? (a) *At architecture design and pre-training*: the model is trained to need a small cache (GQA, MLA, hybrids, trainable sparse attention). (b) *Post-training conversion*: an existing dense model is uptrained or distilled into a smaller-cache form (GQA from MHA, MHA2MLA, linearization). (c) *Training-free at prefill*: a policy compresses the cache after the prompt is processed (SnapKV, PyramidKV, KVzip). (d) *Training-free during decode*: a policy runs continually while tokens are generated (StreamingLLM, H2O, R-KV). Costs rise from (d) to (a); fidelity usually improves in the opposite direction.

Axis 3: query-aware or query-agnostic? A *query-agnostic* method compresses before it knows what will be asked; a *query-aware* method uses the current query (or the user's question) to decide. Query-agnostic compression is reusable across many future queries and across turns; query-aware compression can be dramatically better on the query it saw but must be redone for the next one, and breaks prefix caching. Section 11.4 proves a fundamental gap between the two.

Table 3.1: Method families, what they reduce, and how they are decided. Qualitative classification by the author; "Training" indicates whether pre-training or fine-tuning is needed. "Cap." capacity, "BW" decode bandwidth, "Pre." prefill compute.

Family	Cap.	BW	Pre.	Training	Function-preserving?
MQA / GQA	✓	✓	–	pretrain or uptrain	no (trained)
MLA	✓	✓	partial	pretrain / convert	no (trained)
Cross-layer sharing	✓	✓	–	pretrain	no (trained)
Sliding window / local–global	✓	✓	✓	pretrain	no (trained)
Hybrid SSM / linear	✓✓	✓✓	✓	pretrain / distill	no (trained)
Training-free eviction	✓	✓	–	none	no (approximate)
Query-aware selection	–	✓	–	none	no (approximate)
Trainable sparse attention	mixed	✓	✓	pretrain / continue	no (trained)
KV merging	✓	✓	–	none / light	no (approximate)
Low-rank / SVD	✓	✓	–	none / calibrate	no (approximate)
Quantization	✓	✓	–	none / calibrate	no (approximate)
Paging / prefix sharing	utilisation	–	reuse	none	yes
Chunked prefill	–	–	latency	none	yes
Offload / disaggregation	placement	transfer	–	none	yes
Prompt compression	✓	✓	✓	none / train compressor	no (input changed)

Axis 4: is it function-preserving? Lossless methods (paging, prefix sharing, chunked prefill, offload, disaggregation, exact attention kernels) do not change the model's function, up to floating-point reordering. Lossy methods change it; the question is whether the change is measured at all (eviction typically is not; MLA-from-scratch is trained, so the new function is by construction what was trained).

Axis 5: which bill does it reduce? The capacity bill, the decode-bandwidth bill, the prefill-compute bill, or the network bill (for disaggregated serving). Table 3.1 summarises. Note that a method that only *selects* which entries to read (Quest, for example) lowers bandwidth but not capacity, whereas a method that *drops* entries lowers both but is irreversible.

3.2 A tree of methods

3.3 Composability

In production, several reductions are stacked. Table 3.2 records, in the author's judgment informed by the systems cited, whether pairs of families compose cleanly. A "C" means they are routinely combined; "S" means they combine with care (interactions are known); "X" means they conflict or are redundant; "?" means evidence is thin.

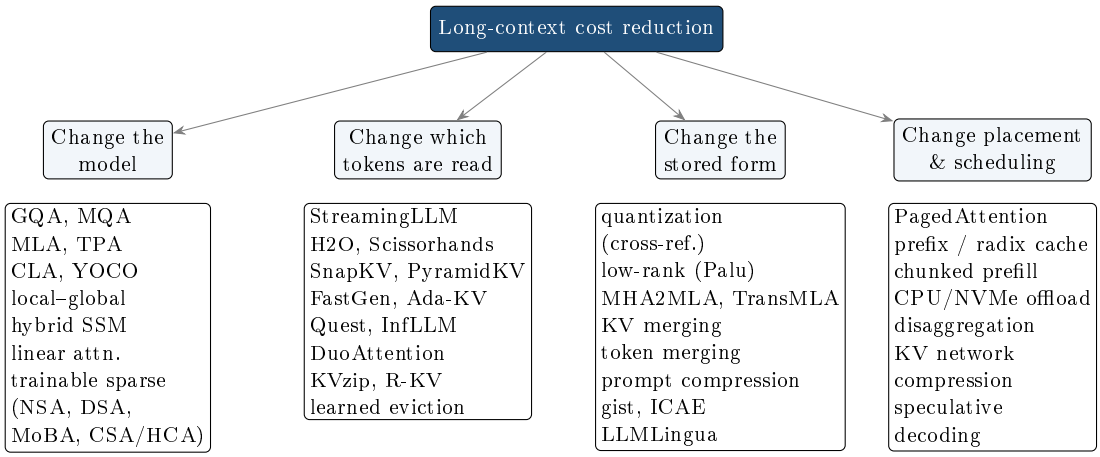


Figure 3.1: A coarse tree of the methods surveyed. Methods in different branches typically compose; methods in the same branch often compete or interfere (Section 3.3).

Table 3.2: Composability of method families (author's qualitative judgment from cited systems; not measured). Rows and columns: GQA/MLA (A), hybrid/linear (H), eviction (E), query-aware selection (Q), trainable sparse (T), low-rank (R), quantization (Z), paging/prefix (P), offload/disaggregation (D), prompt compression (C).

	A	H	E	Q	T	R	Z	P	D	C
A GQA/MLA	–	C	C	C	C	S	C	C	C	C
H Hybrid/linear	C	–	S	S	S	?	S	S	S	C
E Eviction	C	S	–	X	X	S	C	X	S	S
Q Selection	C	S	X	–	S	S	S	C	C	C
T Trainable sparse	C	S	X	S	–	?	C	C	C	C
R Low-rank	S	?	S	S	?	–	C	C	C	C
Z Quantization	C	S	C	S	C	C	–	C	C	C
P Paging/prefix	C	S	X	C	C	C	C	–	C	S
D Offload/disagg.	C	S	S	C	C	C	C	C	–	C
C Prompt comp.	C	C	S	C	C	C	C	S	C	–

The "X" entries between eviction and prefix sharing deserve explanation. Eviction that depends on the question (query-aware) produces a different compressed cache for each question, so the shared-prefix cache cannot be reused; eviction that mutates a cache in place breaks content-addressed block sharing. Both are discussed in Chapter 14.

3.4 Cost-model view: what each family buys

To compare families on a common footing, write the per-token decode cost as

$$t_{\text{tok}} \approx \frac{W/B + \rho\, S_{\text{KV}}^{(1)}\,(1-\eta)}{\beta}, \tag{3.1}$$

where $\rho \in (0, 1]$ is the fraction of the retained cache that is *read* each step (selection and sparsity set $\rho < 1$), and $\eta \in [0, 1)$ is the fraction of the baseline cache that has been removed or never created (architecture, eviction, low-rank and quantization increase η). Methods that raise η

also lower the capacity bill; methods that lower ρ do not. For a retrieval-style selector that reads $\rho = 1/16$ of the cache but retains it in full, t_{tok} falls almost sixteen-fold at large batch while capacity is unchanged, so the concurrent batch is unchanged and the throughput gain is limited by what else is bandwidth-bound (weights, selection scoring). For eviction by a factor of four with $\rho = 1$, both capacity and bandwidth improve by four, and the concurrent batch grows four-fold. Only when the batch is large enough that weights are not dominant do the two translate into equal throughput gains.

Reading the taxonomy

Pick one method *from each of several branches* rather than comparing methods within a branch: architecture (set at model selection), a stored-form reduction (quantization, perhaps low-rank), and placement/scheduling (paging, prefix sharing, disaggregation). Token-level lossy methods are the branch to add last, only where measured workloads justify the fidelity risk.

Chapter 4

Architectural Cache Reduction I: Heads, Latents and Layers

The most reliable way to shrink the cache is to design the attention layer so that less needs to be stored. This chapter treats reductions that keep the softmax-attention form: sharing keys and values across heads (MQA, GQA), compressing them to a shared latent (MLA and relatives), and sharing them across layers (CLA, YOCO). Chapter 5 treats windows and recurrent layers.

4.1 From multi-head to grouped-query attention

4.1.1 Definitions and cache ratio

Multi-head attention gives every query head its own key and value head ($g = 1$). Multi-query attention shares a single key–value head among all query heads [2]; grouped-query attention uses H_{kv} heads with $g = H_q/H_{\text{kv}}$ query heads per group [3]. By Proposition 2.1 the cache falls by the factor g and by Proposition 2.2 the decode intensity rises by g. The parameter count of the key and value projections also falls by g, a minor saving, and the model must typically widen elsewhere to compensate.

4.1.2 Uptraining

The GQA paper showed that an existing MHA checkpoint can be converted rather than re-trained [3]. The recipe is (1) *mean-pool* the key (and value) projection matrices of the heads in each group into a single projection; (2) continue pre-training for a small fraction of the original compute (the authors report about 5% of the original pre-training compute for T5-scale models). Mean pooling is justified by a simple argument: if the heads of a group are nearly collinear, the pooled head is close to each of them, and the fine-tuning phase repairs the rest. This conversion pattern, "pool, then briefly retrain," recurs in the MLA-conversion literature (Chapter 8).

Proposition 4.1 (Uptraining initialisation error)**.** *Let a group contain g heads with key projections $\mathbf{W}_1^K, \dots, \mathbf{W}_g^K$ and let $\bar{\mathbf{W}}^K = \frac{1}{g}\sum_i \mathbf{W}_i^K$. For hidden state $\mathbf{h}$ and query $\mathbf{q}_i$, the logit error of replacing head i's key by the pooled key is $\mathbf{q}_i^\top(\mathbf{h}^\top(\mathbf{W}_i^K - \bar{\mathbf{W}}^K))^\top$, bounded by $\|\mathbf{q}_i\|\,\|\mathbf{h}\|\,\|\mathbf{W}_i^K - \bar{\mathbf{W}}^K\|_2$. The mean-pooled matrix minimises $\sum_i \|\mathbf{W}_i^K - \mathbf{W}\|_F^2$ over all single matrices $\mathbf{W}$.*

Proof. The bound is Cauchy–Schwarz. The minimiser follows from setting the gradient $2\sum_i(\mathbf{W} - \mathbf{W}_i^K)$ to zero, giving the mean. □

The proposition is a sufficiency argument rather than a guarantee: it minimises the Frobenius

distance of the *weights*, not the change in attention output, so it is a heuristic starting point for continued training and not a function-preserving transformation.

4.1.3 Position in current models

GQA is the baseline of nearly all open dense models. Llama 3 uses 8 KV heads at all scales [42]; Qwen3-8B uses 8 KV heads with 32 query heads [37]; gpt-oss-120b uses 8 KV heads with 64 query heads [17]; Mistral 7B uses 8 KV heads [14] (Table 2.1). The ratio g ranges from 4 to 16 across these models. Quality loss from MQA-level sharing ($H_{\text{kv}} = 1$) is noticeable at large scale, so most designs stop at moderate g and look elsewhere (latent compression, windows, hybrids) for further reductions.

4.2 Multi-head latent attention

MLA, introduced in DeepSeek-V2 [4] and retained in V3, V3.2, Kimi K2, GLM-4.5-class models and Kimi Linear's full-attention layers [5, 6, 33, 44], compresses keys and values *jointly* into one low-dimensional latent vector per token and reconstructs per-head keys and values from it by learned up-projections.

4.2.1 Low-rank joint compression

Let $\mathbf{h}_t \in \mathbb{R}^d$ be the layer input at position t. MLA computes

$$\mathbf{c}_t^{KV} = \mathbf{W}^{DKV}\mathbf{h}_t \in \mathbb{R}^{d_c}, \tag{4.1}$$

$$\mathbf{k}_{t,i}^{C} = \mathbf{W}_i^{UK}\mathbf{c}_t^{KV}, \qquad \mathbf{v}_{t,i}^{C} = \mathbf{W}_i^{UV}\mathbf{c}_t^{KV}, \qquad i = 1, \ldots, H_q, \tag{4.2}$$

with $d_c \ll H_q d_h$. In DeepSeek-V2/V3, $H_q = 128$ and $d_h = 128$ while $d_c = 512$, four times the head width but 32 times smaller than $H_q d_h$ [4]. Only $\mathbf{c}_t^{KV}$ is cached. Because Equation (4.2) says the stacked keys and values of a token lie in the d_c-dimensional column space of a fixed matrix, MLA is exactly a rank-d_c factorisation of the concatenated per-token key–value vector, in the sense of Theorem 2.1 but with the factor matrices *learned end-to-end* rather than obtained by post-hoc SVD.

4.2.2 Decoupled rotary embeddings

Rotary position embedding (RoPE) [148] rotates the query at position t and key at position s so that $\mathbf{q}_t^\top \mathbf{k}_s$ becomes a function of $\mathbf{q}_t$, $\mathbf{k}_s$ and $t - s$: $\mathbf{q}_t^\top \mathbf{R}_{s-t}\mathbf{k}_s$ with a block-diagonal rotation $\mathbf{R}$. If RoPE were applied to $\mathbf{k}_{s,i}^{C} = \mathbf{W}_i^{UK}\mathbf{c}_s$ the rotation would sit between the query side and $\mathbf{W}_i^{UK}$, and, because $\mathbf{R}_{s-t}$ depends on both positions and does not commute with $\mathbf{W}_i^{UK}$, the up-projection could not be absorbed into the query (next subsection). MLA therefore *decouples* position: it adds a small extra key $\mathbf{k}_t^R = \text{RoPE}(\mathbf{W}^{KR}\mathbf{h}_t) \in \mathbb{R}^{d_h^R}$ that is shared by all heads and cached alongside the latent, plus a per-head rotary query $\mathbf{q}_{t,i}^R$, and defines the score as the sum

$$a_{t,s,i} = \frac{\mathbf{q}_{t,i}^{C\top}\mathbf{k}_{s,i}^{C} + \mathbf{q}_{t,i}^{R\top}\mathbf{k}_s^R}{\sqrt{d_h + d_h^R}}. \tag{4.3}$$

Algorithm 1 MLA decode step with weight absorption (one layer, one sequence).

Require: hidden state $\mathbf{h}_t$; cache $\mathbf{C} \in \mathbb{R}^{n \times d_c}$, $\mathbf{K}^R \in \mathbb{R}^{n \times d_h^R}$; position t

$\mathbf{c}_t \leftarrow \mathbf{W}^{DKV}\mathbf{h}_t$; $\quad \mathbf{k}_t^R \leftarrow \mathrm{RoPE}_t(\mathbf{W}^{KR}\mathbf{h}_t)$

append $\mathbf{c}_t$ to $\mathbf{C}$, $\mathbf{k}_t^R$ to $\mathbf{K}^R$

$\mathbf{c}^Q \leftarrow \mathbf{W}^{DQ}\mathbf{h}_t$

for each head $i = 1, \ldots, H_q$ **in parallel do**

  $\tilde{\mathbf{q}}_i \leftarrow \mathbf{W}_i^{UK\top}\big(\mathbf{W}_i^{UQ}\mathbf{c}^Q\big)$ ▷ absorbed query, d_c-dim; $\mathbf{W}_i^{UK\top}\mathbf{W}_i^{UQ}$ can be precomputed

  $\mathbf{q}_i^R \leftarrow \mathrm{RoPE}_t(\mathbf{W}_i^{QR}\mathbf{c}^Q)$

  $\mathbf{a}_i \leftarrow \big(\mathbf{C}\tilde{\mathbf{q}}_i + \mathbf{K}^R\mathbf{q}_i^R\big)/\sqrt{d_h + d_h^R}$ ▷ n scores from *shared* cache

  $\mathbf{p}_i \leftarrow \mathrm{softmax}(\mathbf{a}_i)$

  $\mathbf{u}_i \leftarrow \mathbf{C}^\top\mathbf{p}_i$ ▷ latent-space output, d_c-dim

end for

return $\sum_i \mathbf{W}_i^O\mathbf{W}_i^{UV}\mathbf{u}_i$ ▷ $\mathbf{W}_i^O\mathbf{W}_i^{UV}$ precomputed

With $d_h^R = 64$ the cached state per token and layer is $d_c + d_h^R = 576$ elements, i.e., $4.5d_h$. The authors state this equals the cache of a GQA model with 2.25 groups, while reporting quality *above* MHA in their ablations [4]. Queries are also compressed through a low-rank bottleneck $\mathbf{c}_t^Q = \mathbf{W}^{DQ}\mathbf{h}_t$, which does not affect the cache but reduces training activation memory.

4.2.3 Weight absorption

During decoding the cached latent can be used directly, without reconstructing the H_q per-head keys and values, by *absorbing* the up-projections into adjacent matrices:

$$\mathbf{q}_{t,i}^{C\top}\mathbf{W}_i^{UK}\mathbf{c}_s = \big(\mathbf{W}_i^{UK\top}\mathbf{q}_{t,i}^C\big)^\top\mathbf{c}_s =: \tilde{\mathbf{q}}_{t,i}^\top\mathbf{c}_s, \qquad \sum_s p_s\mathbf{W}_i^{UV}\mathbf{c}_s = \mathbf{W}_i^{UV}\Big(\sum_s p_s\mathbf{c}_s\Big). \tag{4.4}$$

The first identity moves $\mathbf{W}_i^{UK}$ onto the query; the second pulls $\mathbf{W}_i^{UV}$ out of the sum and lets it be folded into the output projection $\mathbf{W}^O$. Both are exact. After absorption, every query head attends over the *same* cached tensor $[\mathbf{c}_s; \mathbf{k}_s^R]$, which makes MLA decoding, structurally, multi-query attention with one KV "head" of width 576 (keys) and 512 (values) and 128 query heads. This is why Proposition 2.3 finds an intensity two orders of magnitude above that of GQA decoding, and why recent sparse-attention designs use MLA in its MQA form for decode efficiency [6].

4.2.4 Prefill versus decode forms

Absorption reduces memory traffic during decode but increases FLOPs per score (dot products of length 576 rather than $d_h + d_h^R = 192$ per head). During prefill, which is compute-bound, it is cheaper to reconstruct per-head keys and values once per layer and run standard MHA-style attention. Production implementations therefore use *two modes*: expanded (MHA-form) for prefill and absorbed (MQA-form) for decode. This is an example of a general principle: the same mathematical function can be realised with different dataflow in the two regimes of the roofline.

4.2.5 Expressivity relative to GQA

Proposition 4.2 (GQA is a special case of MLA with equal cache)**.** *Any GQA layer with H_{kv} KV heads of width d_h can be written as an MLA layer with latent width $d_c = 2H_{\text{kv}}d_h$ (without decoupled RoPE), with identical outputs. The converse fails: for general $\mathbf{W}_i^{UK}$ an MLA layer with $d_c = 2H_{\text{kv}}d_h$ is not equivalent to any GQA layer with H_{kv} heads.*

Proof. Let the GQA key and value projections of group j be $\mathbf{W}_j^K, \mathbf{W}_j^V \in \mathbb{R}^{d_h \times d}$. Set $\mathbf{W}^{DKV} = [\mathbf{W}_1^K; \ldots; \mathbf{W}_{H_{\text{kv}}}^K; \mathbf{W}_1^V; \ldots; \mathbf{W}_{H_{\text{kv}}}^V] \in \mathbb{R}^{2H_{\text{kv}}d_h \times d}$, so $\mathbf{c}_t$ stacks all group keys and values. Choose $\mathbf{W}_i^{UK}$ to be the selector that extracts the key block of the group of head i (a $d_h \times d_c$ matrix with an identity block, zero elsewhere), and likewise $\mathbf{W}_i^{UV}$ for the value block. Then $\mathbf{k}_{t,i}^C = \mathbf{W}_{j(i)}^K \mathbf{h}_t$ and $\mathbf{v}_{t,i}^C = \mathbf{W}_{j(i)}^V \mathbf{h}_t$, which is GQA. For the converse, take $H_{\text{kv}} = 1$, $d_h = 1$ so $d_c = 2$ and $H_q = 2$ heads with $\mathbf{W}_1^{UK} = [1, 0]$, $\mathbf{W}_2^{UK} = [0, 1]$; the two heads then use *different* linear functionals of the latent as keys, which an MQA layer (one shared key) cannot represent for general $\mathbf{W}^{DKV}$. □

This is the observation behind TransMLA [82]: since GQA embeds into MLA at equal cache, it makes sense to convert GQA models into MLA models and use the extra expressivity during fine-tuning. We return to this in Chapter 8.

4.2.6 What MLA costs

MLA is not free. (i) The decoupled-RoPE key must be cached and handled by kernels. (ii) The up-projection and absorbed matrices add parameters and a more complex kernel; decode kernels for MLA are specialised (the Flash-MLA family) and not all serving stacks supported them at first. (iii) With tensor parallelism, the single shared latent cannot be sharded along heads, so it is replicated, which erodes the capacity benefit unless attention data parallelism is used (Chapter 9). (iv) The compression is lossy at training time by construction; there is no free post-hoc conversion unless one accepts an uptraining stage.

4.3 Other channel-compressing designs

Tensor Product Attention (TPA) [83] factorises queries, keys and values into contextual low-rank tensor products of small factors computed from the hidden state, caching the factors rather than the full vectors; the authors describe a substantial reduction of the cache at inference time with competitive quality in their experiments (the abstract gives no single factor). **Grouped-Tied Attention and Grouped Latent Attention** [84] are decode-oriented designs that tie keys and values within a group or shard the latent across devices to preserve arithmetic intensity under tensor parallelism; they are early-stage relative to MLA and GQA, and independent large-scale adoption remains to be shown. The common theme is consistent with Proposition 2.2: the way to speed up decoding is to raise the number of query heads that reuse each cached byte.

Table 4.1 shows why MLA is attractive: it reaches a cache size within a factor of ≈ 2.25 of MQA while retaining per-head diversity through the learned up-projections, and it keeps the 128-fold reuse of MQA.

Table 4.1: Per-token, per-layer cache elements of attention variants at the DeepSeek-V3-like scale $H_q = 128$, $d_h = 128$ (*derived*); GQA group counts are chosen for illustration. MLA numbers follow [4, 5]. "Reuse" is the number of query heads that read each cached element, i.e., the decode arithmetic intensity in FLOP/byte at 2 bytes per element up to a constant.

Variant	Elements/token/layer	vs. MHA	Reuse (queries/element)	Note
MHA ($H_{\text{kv}} = 128$)	32,768	1×	1	baseline
GQA ($H_{\text{kv}} = 16$, $g = 8$)	4,096	8×	8	typical moderate setting
GQA ($H_{\text{kv}} = 8$, $g = 16$)	2,048	16×	16	Llama-3-405B-style ratio
MLA (d_c=512, d_h^R=64)	576	56.9×	128 (absorbed)	shared latent
MQA ($H_{\text{kv}} = 1$)	256	128×	128	single head, quality risk

4.4 Cross-layer sharing

4.4.1 Cross-layer attention

Cross-layer attention (CLA) [9] lets several consecutive layers share the same key and value tensors: layers in a sharing group compute queries independently but attend to the K/V computed by the group's first layer, so only one layer per group stores a cache and only one pays the K/V projection. With sharing factor 2 ("CLA2") the cache halves; the authors report experiments at the 1B and 3B scale in which CLA combined with MQA reaches a better accuracy-versus-memory trade-off than MQA alone, enabling a further 2× reduction at modest accuracy cost. The result is an architecture decision at pretraining; it cannot be applied after the fact without retraining, though post-hoc *merging* of adjacent layers' caches is explored in Chapter 8 [86].

4.4.2 YOCO: you only cache once

YOCO [10] splits a decoder-only model into a *self-decoder* followed by a *cross-decoder*. The self-decoder uses efficient attention (sliding window or a gated recurrent form) and so has constant cache. Its output is projected *once* into a global key–value cache which every cross-decoder layer reads through cross-attention. If the model has L layers split evenly, the global cache holds one set of keys and values instead of $L/2$. The authors' accounting gives memory of order $\mathcal{O}((n + L)\, d)$ rather than $\mathcal{O}(nLd)$ and, because the prefill of the cross-decoder is needed only for the last token (the earlier tokens' cross-decoder states are never consumed), they also report reduced prefill latency. The cost is restricted expressivity: all cross-decoder layers see the same keys and values, which differ from what separate layers could compute.

Proposition 4.3 (Cache of cross-layer sharing)**.** *Let a model with L layers be partitioned into m groups, with layers in each group sharing a single KV cache of c elements per token. The cache is $mcbn$ bytes per sequence, a reduction by L/m. If, in addition, a fraction ϕ of layers is local (window $w \ll n$), their contribution is constant in n and for $n \gg w$ the per-token cache is $m_{\text{glob}}\, c\, b$ where m_{glob} counts only the global groups.*

Proof. Direct from Equation (2.5) with $c_\ell = c$ for one layer per group and $c_\ell = 0$ for the rest. □

4.5 Worked example: assembling a small-cache dense model

Suppose we want a 64-layer, 128-query-head model to have a cache per token of at most 10 KiB in bf16. Equation (2.4) requires $\sum_\ell c_\ell \le 5120$ elements. With MHA, $c_\ell = 32768$, total 2.1×10^6 elements, so we need a 410× reduction. Options: GQA with $H_{\text{kv}} = 8$ gives $2048 \cdot 64 = 131{,}072$ (25.6× over the budget); MLA with 576 elements gives 36,864 (7.2× over); MLA plus 8-bit quantization gives 18,432 bytes-equivalent (about 3.6× over); MLA with CLA2 and 8-bit gives ≈9,216 (still above); only a hybrid with ≤8 global MLA layers of 576 elements (4608 elements) meets the budget in bf16. This calculation is why the frontier of cache reduction for long-context models is hybrid rather than purely dense (Chapter 5). It is also a template: the reader can substitute their own target and see which combinations are feasible (derived).

Chapter summary

- GQA reduces the cache by g and raises decode intensity by g; it is the default.
- MLA caches a rank-d_c latent plus a small decoupled-RoPE key; with absorption it is MQA in structure; DeepSeek reports 576 elements per token per layer, which we compute as ≈57× below MHA at the same head count.
- GQA ⊂ MLA at equal cache (Proposition 4.2), which motivates conversion.
- Cross-layer sharing (CLA, YOCO) reduces the layer dimension of the cache at the price of fixed architectural structure.
- No dense-only design reaches the ∼10 KiB/token regime without further reduction; hybrids do (Table 2.1).

Chapter 5

Architectural Cache Reduction II: Windows, Recurrence and Hybrids

Chapter 4 reduced the size of each token's contribution. This chapter reduces the *number of tokens* each layer holds, either by restricting attention to a window or by replacing the growing cache with a fixed-size recurrent state. These designs make the biggest dent in the cache of current frontier open models (Table 2.1), and they change the system's properties more profoundly than GQA or MLA do.

5.1 Sliding-window and local–global attention

5.1.1 Windows

In sliding-window attention (SWA) token t attends only to positions $s \in (t-w, t]$ [11, 12, 14]. The cache of such a layer is a rolling buffer of w tokens, constant in n. A model of L layers, all windowed, can still propagate information across more than w tokens, since layer ℓ's output at position t depends on layer $\ell-1$ outputs within the window, which themselves depend on earlier positions:

Proposition 5.1 (Receptive field of stacked windows)**.** *In a stack of L causal sliding-window layers with window w, the output at position t depends on inputs at positions in $[t-L(w-1), t]$ and on no earlier position.*

Proof. By induction: the first layer's output at t depends on $[t-(w-1), t]$. If layer $\ell-1$'s output at s depends on $[s-(\ell-1)(w-1), s]$ then layer ℓ's output at t depends on the union over $s \in [t-(w-1), t]$, i.e., on $[t-(w-1)-(\ell-1)(w-1), t] = [t-\ell(w-1), t]$. □

The theoretical receptive field $L(w-1)$ is far larger than w (for Mistral 7B with $w = 4096$ and 32 layers, 131K [14]), but the *effective* information transfer over such distances is attenuated and unreliable, since each hop passes through a bounded-width residual stream. Retrieval of a specific token at distance beyond w requires a relay that the model must learn to perform; experience suggests it does so poorly (see the MiniMax report below).

5.1.2 Local–global interleaving

A middle path alternates windowed and global layers. Gemma 2 alternated local (window 4096) and global layers 1:1 [15]; Gemma 3 moved to five local layers (window 1024) for each global layer and a 128K context window, and its report presents this as a large reduction in the share of memory spent on the cache at long context compared with all-global layers [16] (we did not

re-verify the exact percentages quoted in secondary summaries, so we omit them here). gpt-oss alternates banded attention with a window of 128 tokens and dense attention in equal numbers, with GQA of 8 KV heads [17]; the model configuration we read lists 18 sliding and 18 full layers out of 36 for the 120B model.

For a model with L_g global and L_l local layers (equal per-layer cache c), the per-sequence cache is

$$S_{\mathrm{KV}} = bc\big[L_g\, n + L_l \min(n, w)\big], \tag{5.1}$$

which for $n \gg w$ approaches $bcL_g n$, a reduction by L/L_g relative to all-global layers. For a 5:1 pattern that is 6×, for 1:1 it is 2× (derived). Local layers also cut prefill compute: their attention costs $\mathcal{O}(nw)$ rather than $\mathcal{O}(n^2)$.

Example 5.1 (Local–global at 128K)**.** Take an *illustrative* 48-layer model with 8 KV heads, $d_h = 128$, bf16, and a 5:1 local:global pattern with $w = 1024$. Global layers: 8; local: 40. Per-token global cache: $8 \times 2 \cdot 8 \cdot 128 \cdot 2 = 32$ KiB. At $n = 131{,}072$ the 8 global layers hold $8 \times 4096\ \mathrm{B} \times 131{,}072 = 4$ GiB. Each local layer holds only $1024 \times 4096\ \mathrm{B} = 4$ MiB, so the 40 local layers hold 160 MiB. The total is ≈ 4.16 GiB against 24 GiB for an all-global model of the same size (derived), a 5.8× reduction, and the local layers' share is under 4% of the total.

5.1.3 Sinks and windows

Windows interact badly with the *attention sink* phenomenon (Chapter 6): trained dense models place a large fraction of attention mass on the first token(s), and when a window evicts them, perplexity collapses. Models *trained* with windows can learn to cope, and gpt-oss includes a learned per-head sink logit that is added to the softmax denominator [17]. Formally, with learned scalar z_h per head:

$$p_i = \frac{e^{s_i}}{e^{z_h} + \sum_j e^{s_j}}, \qquad \sum_i p_i = 1 - \frac{e^{z_h}}{e^{z_h} + \sum_j e^{s_j}} < 1. \tag{5.2}$$

This is equivalent to adding a key whose value is the zero vector: the head can output nearly nothing by attending to the sink, without needing a real token to absorb mass. Gated attention [40] achieves a similar end with an output gate and reports eliminating massive activations and sink behaviour in the model, which makes windowed or evicted caches less brittle. We discuss the underlying phenomenon, massive activations [189, 190], in Section 6.2.

5.2 State-space models and linear attention

5.2.1 A common recurrent form

A broad class of sequence layers can be written as a *matrix-valued state* updated by a gated outer-product rule:

$$\mathbf{S}_t = \mathbf{G}_t \odot \mathbf{S}_{t-1} + \mathbf{v}_t \mathbf{k}_t^\top, \qquad \mathbf{o}_t = \mathbf{S}_t \mathbf{q}_t, \qquad \mathbf{S}_t \in \mathbb{R}^{d_v \times d_k}, \tag{5.3}$$

where $\mathbf{G}_t$ is a gate whose structure distinguishes the family. The state has *fixed size* $d_v d_k$ per head regardless of n. Table 5.1 lists the main members.

Table 5.1: Members of the linear-recurrence family, written in the form of Equation (5.3) (simplified; each paper adds normalisation, convolutions, gating of the output and other details). $\alpha_t \in (0,1)$ scalar decay, $\boldsymbol{\alpha}_t$ vector decay, β_t write strength.

Layer	State update (simplified)	Reference
Linear attention	$\mathbf{S}_t = \mathbf{S}_{t-1} + \mathbf{v}_t \phi(\mathbf{k}_t)^\top$	[21]
RetNet	$\mathbf{S}_t = \gamma \mathbf{S}_{t-1} + \mathbf{v}_t \mathbf{k}_t^\top$, fixed γ per head	[25]
Mamba / Mamba-2 (SSD)	$\mathbf{S}_t = \alpha_t \mathbf{S}_{t-1} + \mathbf{v}_t \mathbf{k}_t^\top$, data-dependent scalar α_t (Mamba-2)	[18, 19]
GLA	$\mathbf{S}_t = \mathbf{S}_{t-1} \operatorname{diag}(\boldsymbol{\alpha}_t) + \mathbf{v}_t \mathbf{k}_t^\top$	[26]
RWKV-5/6 (Eagle/Finch)	matrix-valued state, per-channel (dynamic) decay	[23, 24]
DeltaNet	$\mathbf{S}_t = \mathbf{S}_{t-1}(\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top) + \beta_t \mathbf{v}_t \mathbf{k}_t^\top$	[22, 27]
Gated DeltaNet	$\mathbf{S}_t = \alpha_t \mathbf{S}_{t-1}(\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top) + \beta_t \mathbf{v}_t \mathbf{k}_t^\top$	[28]
Kimi Delta Attention	Gated DeltaNet with fine-grained (channel-wise) gate	[33]
Test-time-training layers / Titans	state is the weights of a small model updated by gradient steps	[142, 143]

Mamba-2's state-space duality shows that scalar-gated SSMs equal masked linear attention with a decay mask, and enables matrix-multiplication-friendly chunkwise training [19]. The selective SSM of Mamba [18] makes the recurrence parameters input-dependent, which is what separates it from earlier, time-invariant SSMs such as S4 [20] and long-convolution layers such as Hyena [186].

5.2.2 The delta rule as online regression

Proposition 5.2 (Delta rule). *The DeltaNet update is one step of stochastic gradient descent with step size β_t on the loss $\ell_t(\mathbf{S}) = \frac{1}{2}\|\mathbf{S}\mathbf{k}_t - \mathbf{v}_t\|^2$. If $\|\mathbf{k}_t\| = 1$ and $\beta_t = 1$ the update* overwrites *the association for key $\mathbf{k}_t$: $\mathbf{S}_t \mathbf{k}_t = \mathbf{v}_t$ exactly.*

Proof. $\nabla_{\mathbf{S}} \ell_t = (\mathbf{S}\mathbf{k}_t - \mathbf{v}_t)\mathbf{k}_t^\top$, so the SGD step is $\mathbf{S}_{t-1} - \beta_t(\mathbf{S}_{t-1}\mathbf{k}_t - \mathbf{v}_t)\mathbf{k}_t^\top = \mathbf{S}_{t-1}(\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top) + \beta_t \mathbf{v}_t \mathbf{k}_t^\top$. With $\|\mathbf{k}_t\| = 1$, $(\mathbf{I} - \mathbf{k}_t \mathbf{k}_t^\top)\mathbf{k}_t = \mathbf{0}$, so $\mathbf{S}_t \mathbf{k}_t = \mathbf{v}_t \mathbf{k}_t^\top \mathbf{k}_t = \mathbf{v}_t$. □

Plain linear attention only accumulates; it cannot remove or correct an earlier association with the same key, so once more than d_k pairs are stored the interference of Proposition 2.7 grows. The delta rule *subtracts the current prediction* before writing, which handles repeated or updated keys, and the scalar gate α_t in Gated DeltaNet adds fast forgetting of everything at once [28]. Channel-wise gates (KDA) forget at different rates per feature [33]. None of these relaxes the counting limit: a state of $d_v d_k$ numbers cannot hold more than $\mathcal{O}(d_v d_k)$ numbers of information (Section 11.4).

5.2.3 Training and conversion

The recurrences are trained in a chunkwise-parallel form that uses matrix multiplications within chunks and a short recurrence across chunks; the delta-rule variants use a compact (WY) representation of products of Householder-like matrices to parallelise over the sequence [27, 28]. Existing transformers can be *converted* to linear or hybrid form by distillation: attention weights

Algorithm 2 Gated delta-rule decode step for one head (recurrent form).

Require: state $\mathbf{S}_{t-1} \in \mathbb{R}^{d_v \times d_k}$; input $\mathbf{h}_t$

$\mathbf{q}_t, \mathbf{k}_t \leftarrow$ normalised projections (ℓ_2-normalise); $\mathbf{v}_t \leftarrow \mathbf{W}^V \mathbf{h}_t$

$\alpha_t \leftarrow \sigma(\cdot)$ (decay gate); $\beta_t \leftarrow \sigma(\cdot)$ (write strength)

$\hat{\mathbf{v}}_t \leftarrow \mathbf{S}_{t-1}\mathbf{k}_t$ ▷ what the state currently predicts for this key

$\mathbf{S}_t \leftarrow \alpha_t \mathbf{S}_{t-1} + \beta_t \left(\mathbf{v}_t - \alpha_t \hat{\mathbf{v}}_t\right) \mathbf{k}_t^\top$ ▷ equals $\alpha_t \mathbf{S}_{t-1}(\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top) + \beta_t \mathbf{v}_t \mathbf{k}_t^\top$

$\mathbf{o}_t \leftarrow \mathbf{S}_t \mathbf{q}_t$

return gated, normalised $\mathbf{o}_t$ ▷ training uses a chunkwise-parallel form

initialise the linear layers' projections and the converted model is trained to match the original's outputs [88–90, 211]. Conversion retains most benchmark scores in the original papers, but recall-heavy long-context tasks are where conversions are weakest; this motivates keeping a subset of attention layers (next section).

5.3 Hybrids

5.3.1 Why hybrids

Pure recurrent models are limited at in-context retrieval and copying. Transformers can copy exponentially long strings with a small model, whereas fixed-state recurrent models cannot (Proposition 11.7; empirical and theoretical evidence in [181–185]). Pure attention models pay the cache bill. A hybrid inserts a few global-attention layers among many recurrent layers: the recurrent layers carry cheap running context, and the attention layers handle exact retrieval.

5.3.2 Ratios in practice

Reported designs differ widely in how much attention they keep. Jamba interleaves one attention layer per eight layers with Mamba layers and mixture-of-experts feed-forward blocks [29]. A large empirical comparison of Mamba-based language models found that a hybrid with only a small fraction (≈7–8%) of attention layers outperformed a pure transformer on the evaluated tasks at 8B scale [30]; the Nemotron-H family adopts hybrid layouts in which the large majority of layers are Mamba-2 [31], and Nemotron 3 Nano combines Mamba-2, attention and MoE layers with a context window of up to one million tokens [32]. MiniMax-01 mixes lightning attention (a linear attention variant [35]) and softmax attention at roughly seven linear layers per softmax layer [34]. Qwen3-Next uses a 3:1 pattern of Gated DeltaNet to gated full attention, and Qwen3.5 reuses it [38, 214]. Kimi Linear interleaves three KDA layers per MLA layer, reporting up to 75% lower KV cache and up to 6× higher decoding throughput at one million tokens in its own comparison against a full-MLA baseline trained with the same recipe [33].

The configuration files let us compute what these ratios imply. In the Qwen3-Next-80B-A3B configuration, 48 layers with full attention every fourth layer give 12 attention layers, each with 2 KV heads of width 256 [38]. In the Kimi-Linear-48B-A3B configuration, layers 4, 8, 12, 16, 20, 24 and 27 (seven of 27) are full MLA attention and the other 20 are KDA [39].

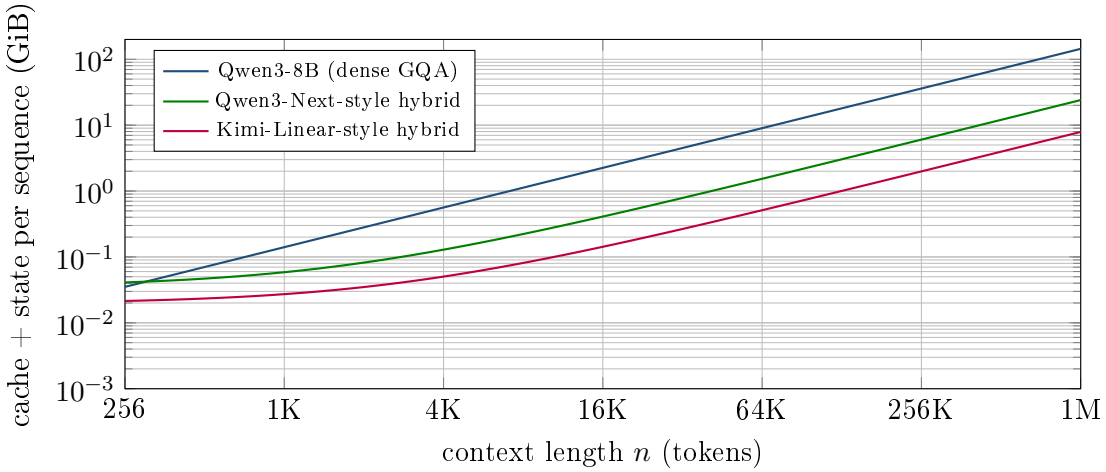


Figure 5.1: **Synthetic, from the derived per-token and state figures of this chapter** (bf16; recurrent state of 1 MiB per linear layer, ignoring convolution buffers and any fp32 state). At short context the hybrids' fixed state makes them *larger* than the dense model; above a few thousand tokens the lower slope wins.

5.3.3 State size and the break-even length

Recurrent layers are not free in memory: each holds a matrix state per head and per sequence. For Qwen3-Next the configuration lists 32 value heads with value width 128 and key width 128 per linear layer [38]; its state is $32 \times 128 \times 128 = 524{,}288$ elements per layer, about 1 MiB in bf16 (some implementations keep the state in higher precision, which doubles it), and 36 linear layers give about 36 MiB per sequence (derived, ignoring short-convolution buffers). The attention cache grows by 24 KiB per token, so the hybrid's attention cache equals its recurrent state at 36 MiB/24 KiB = 1536 tokens. For Kimi Linear, with 20 KDA layers of $32 \times 128 \times 128$ elements, the state is about 20 MiB, equal to its 7.9 KiB/token attention cache at about 2600 tokens (derived). Beyond a few thousand tokens the constant state is negligible and the per-token slope dominates.

5.3.4 System consequences of recurrent state

A recurrent state, unlike a KV cache, is not *truncatable*: the state after token t cannot be obtained from the state after token $t+k$. This has four consequences that make hybrids harder to serve than their cache sizes suggest.

1. **Prefix caching** must store *state checkpoints* at chosen positions rather than slicing a stored sequence; each checkpoint costs the full state size, so checkpoint spacing trades memory for recomputation. A KV cache supports sharing at every token boundary at no cost.
2. **Speculative decoding** needs rollback when drafts are rejected; with a recurrent state this requires saving the pre-draft state or recomputing it.
3. **Eviction** of individual tokens is undefined for recurrent layers; compression of the recurrent part is by design (decay), not by policy.
4. **Low-precision state** storage has numerical sensitivity because errors compound through the recurrence, which the MiniMax team cites among reasons for reluctance [36].

5.3.5 When hybrids lose: the MiniMax-M2 report

MiniMax's M1 model, built on lightning attention, was followed by M2, which returned to full attention. Their public account lists reasons: efficient-attention infrastructure (low-precision state storage, prefix caching, speculative decoding) was immature; a hybrid sliding-window variant degraded as context length grew in agentic settings; and, notably, a hybrid of lightning and full attention matched full attention on standard benchmarks (MMLU, BBH, MATH, LongBench) yet showed deficits on complex multi-hop reasoning at scale [36]. They also note that proxy metrics designed to detect such weaknesses may not generalise, and that benchmarks are quickly saturated once targeted. We read this as an *existence proof of silent failure*: a hybrid can pass the conventional evaluation suite and still fail on the workload that matters. It is a single report from one team and does not show that hybrids generally fail; Qwen, Kimi and NVIDIA all ship hybrids. Labonne's survey of recent models concludes, tellingly, that nobody agrees on attention anymore: Qwen3.5 uses a 3:1 Gated-DeltaNet pattern, Kimi K2.5 and GLM-5 use MLA (GLM-5 with DeepSeek Sparse Attention), and MiniMax-M2.5 uses full attention [214].

5.4 Surveying the frontier

Table 5.2 summarises the attention designs of widely used open-weight models as documented in the cited sources, through October 2026. We include only facts we could verify from papers, model configurations or the cited secondary summaries (see also the survey in [215]); the list is not exhaustive and some entries (for example, exact layer counts of newer models) are omitted where we could not verify them.

Chapter summary

- Windows give constant-size caches but need sinks (learned or retained) and a global channel for retrieval; local–global interleaving at 5:1 cuts the cache $\approx 6\times$ at long context.
- Linear recurrences are a family (5.3); the delta rule corrects stored associations (Proposition 5.2) and gating forgets, but state size is fixed.
- Hybrids with roughly one global layer in four to eight achieve the lowest per-token caches in current open models (7.9–24 KiB/token derived from public configurations); their fixed state breaks even with the cache at 1.5K–3K tokens.
- Recurrent state is not truncatable: prefix caching, speculative decoding and low-precision storage are harder.
- A credible report shows hybrid designs can pass standard benchmarks yet fail on multi-hop reasoning; evaluation, not asymptotics, decides.

Table 5.2: Attention design of selected open-weight and open-report models. "Source" is the primary reference; entries taken from secondary summaries are marked with *. Cache figures are in Table 2.1 where derivable.

Model	Context (claimed)	Attention design	Source
Llama 3 (8B/70B/405B)	128K	GQA, 8 KV heads at all sizes	[42]
Qwen3 (dense)	32K native (config 40K for 8B)	GQA	[37]
Gemma 3	128K	5 local (window 1024) : 1 global, GQA	[16]
gpt-oss-20b/120b	128K	alternating banded (128) and dense, GQA, learned sink	[17]
DeepSeek-V3	128K	MLA (576 elements/token/layer)	[5]
DeepSeek-V3.2	128K	MLA + DeepSeek Sparse Attention (lightning indexer)	[6]
DeepSeek-V4 (Pro/Flash)	1M	CSA (4× compression + sparse top-k + window) alternating with HCA (128×)	[7, 213]
Kimi K2	128K	MLA	[44]
GLM-5	—	MLA + DSA*	[214]
Qwen3-Next-80B-A3B	262K config	3:1 Gated DeltaNet : gated attention (2 KV heads, width 256)	[38]
Qwen3.5	—	same 3:1 pattern as Qwen3-Next*	[214]
Kimi-Linear-48B-A3B	1M	3:1 KDA : MLA	[33, 39]
Nemotron 3 Nano	1M	hybrid Mamba-2 / attention / MoE	[32, 212]
Nemotron-H	—	hybrid, mostly Mamba-2	[31]
Jamba	256K	1 attention : 7 Mamba, MoE	[29]
MiniMax-01 / M1	up to 1M+	lightning (linear) : softmax ≈7:1	[34]
MiniMax-M2(.5)	—	full attention (reverted)	[36, 214]

Chapter 6

Token Eviction and Selection

Given a trained transformer with a dense cache, can one discard or skip most of the cached tokens at inference time and retain the model's behaviour? This chapter surveys the methods that answer "yes, often" without any retraining. We first formalise the problem and derive the exact effect of removing one token; we then treat attention sinks and windows, accumulated-score methods, prefill-time compression, head-specialised policies, query-aware selection, and decode-time methods for reasoning traces.

6.1 Problem formulation

Fix one head with cached keys $\mathbf{k}_1, \ldots, \mathbf{k}_n$ and values $\mathbf{v}_1, \ldots, \mathbf{v}_n$. A *retention policy* chooses a subset $\mathcal{S} \subseteq [n]$ with $|\mathcal{S}| \leq k$ (the budget). Attention then runs only over $\mathcal{S}$:

$$\mathbf{o}_{\mathcal{S}}(\mathbf{q}) = \sum_{s \in \mathcal{S}} \frac{e^{\mathbf{q}^\top \mathbf{k}_s / \sqrt{d_h}}}{\sum_{u \in \mathcal{S}} e^{\mathbf{q}^\top \mathbf{k}_u / \sqrt{d_h}}} \mathbf{v}_s. \tag{6.1}$$

The goal is to keep $\mathbf{o}_{\mathcal{S}}(\mathbf{q})$ close to $\mathbf{o}(\mathbf{q})$ for *future* queries $\mathbf{q}$ (which are unknown), at all heads and layers, with the downstream effect measured on the model's end task. Three design choices distinguish methods: (i) the *scoring signal* that estimates a token's future importance; (ii) the *granularity and budget allocation* (global, per-layer, per-head, per-group); (iii) the *timing*: whether a policy is applied once after prefill, continuously during decoding, or at every step as a selection (without dropping).

6.1.1 The exact effect of removing one token

Proposition 6.1 (Single-token removal)**.** *Let* $\mathbf{o} = \sum_s p_s \mathbf{v}_s$ *with* $\sum_s p_s = 1$ *and* $p_i < 1$*. If token* i *is removed and the remaining weights renormalised, the new output is* $\mathbf{o}' = (\mathbf{o} - p_i \mathbf{v}_i)/(1 - p_i)$ *and*

$$\mathbf{o}' - \mathbf{o} = \frac{p_i}{1 - p_i} (\mathbf{o} - \mathbf{v}_i), \qquad \left\| \mathbf{o}' - \mathbf{o} \right\| = \frac{p_i}{1 - p_i} \left\| \mathbf{v}_i - \mathbf{o} \right\|. \tag{6.2}$$

Proof. Removing i and renormalising divides the remaining weights by $1 - p_i$, so $\mathbf{o}' = \sum_{s \neq i} \frac{p_s}{1 - p_i} \mathbf{v}_s = \frac{\mathbf{o} - p_i \mathbf{v}_i}{1 - p_i}$. Subtracting $\mathbf{o}$ and putting over the common denominator gives $\frac{\mathbf{o} - p_i \mathbf{v}_i - (1 - p_i)\mathbf{o}}{1 - p_i} = \frac{p_i(\mathbf{o} - \mathbf{v}_i)}{1 - p_i}$. □

The proposition says that the damage of evicting a token is proportional to its attention weight *and* to how far its value is from the current output. A token with large weight but a value equal to the average does no harm to remove; a token with small weight and a very atypical value can still matter, especially when many such tokens are evicted. Nearly all scoring rules below

use only p_s, ignoring the value term; value-aware scores are an under-explored refinement, and the ℓ_2-norm heuristic [58] is a crude proxy.

A unifying view: scores estimate expected future attention

H2O, Scissorhands, TOVA, SnapKV, PyramidKV and KVzip differ mainly in *how they estimate* $\mathbb{E}_{\mathbf{q}}[p_s(\mathbf{q})]$, the expected attention weight of token s under future queries: by accumulating past weights (H2O), counting low-weight events (Scissorhands), using the latest weight (TOVA), using a window of recent prompt queries (SnapKV) or using the weights during a synthetic reconstruction task (KVzip). Their relative merits depend on how well each estimator predicts the future query distribution.

6.2 Attention sinks and StreamingLLM

6.2.1 The phenomenon

Xiao et al. observed that when a pretrained dense model is run with a sliding window, perplexity explodes as soon as the first tokens leave the window, even though those tokens are semantically uninformative [46]. Large fractions of attention mass in many heads are assigned to the first few tokens regardless of content. The authors call this the *attention sink*. Their explanation: softmax forces $\sum_s p_s = 1$, so a head with nothing relevant to attend to must place its mass *somewhere*; the initial tokens, visible to every later token during training, become a learned place for surplus mass. A learned sink logit (Equation (5.2)) provides the same service explicitly.

Subsequent work links sinks to *massive activations*: a few specific hidden-state dimensions take values orders of magnitude larger than typical in a few tokens (often the first token or delimiter tokens), which makes the attention logit to those tokens large and stable, and which act like fixed biases [189]. Gu et al. study when sinks emerge, finding they appear in many pretrained models and depend on optimisation and data and on the softmax normalisation itself [190]. Gated attention models that put a sigmoid gate on the attention output show far less sink behaviour and fewer massive activations [40], which indicates that sinks are a workaround for a missing "output nothing" option rather than an inherent feature of language.

6.2.2 StreamingLLM

StreamingLLM retains the keys and values of the first s_0 tokens (the sinks; $s_0 = 4$ suffices in the authors' experiments) and of a rolling window of the most recent w tokens [46]. A crucial implementation detail with rotary embeddings is that positions are assigned *within the cache*, not by original text position: after evicting the middle, the cached tokens are treated as contiguous. The authors report stable perplexity over streams of millions of tokens (up to four million in their evaluation) and up to 22.2× speedup over a sliding-window baseline that recomputes states for each window [46].

What StreamingLLM does not do

StreamingLLM keeps the model *stable* on an unbounded stream; it does not extend the model's *memory*. Anything outside the sinks and the window is gone: a fact stated 10K tokens ago cannot be recalled after eviction. The authors state this limitation explicitly. Us-

Algorithm 3 StreamingLLM cache update at each decode step (per layer and head).

Require: cache $\mathcal{K}, \mathcal{V}$ of length m; sink size s_0; window w; new pair $(\mathbf{k}_t, \mathbf{v}_t)$ with *pre-rotation* key

append $(\mathbf{k}_t, \mathbf{v}_t)$; $m \leftarrow m + 1$
if $m > s_0 + w$ **then**
    remove the entries at cache indices $s_0, \ldots, m - w - 1$ ▷ evict the oldest non-sink entries
    $m \leftarrow s_0 + w$
end if
apply RoPE with *cache-relative* positions $0, \ldots, m - 1$ to the stored pre-rotation keys when computing scores
return attention over the whole retained cache

ing StreamingLLM as a "long-context" method for question answering over a long document is a misuse and fails predictably.

Storing keys *before* rotation and rotating on the fly (as in the algorithm) is what allows re-assignment of positions; engines that store post-rotation keys must instead re-rotate on eviction, which costs bandwidth.

6.3 Accumulated-score methods

6.3.1 Heavy hitters: H_2O

H2O observes that a small set of tokens, the *heavy hitters*, receive a disproportionate share of attention across steps and that these are well predicted by *accumulated attention score* $a_s = \sum_{t' \le t} p_s^{(t')}$ [47]. It keeps a budget of k tokens split between heavy hitters and the most recent ones, evicting at each step the retained token with the lowest accumulated score outside the recency window. The authors give a theoretical argument that greedy eviction under a submodular-reward formulation is near-optimal, and report, for OPT-6.7B/30B and other models, an implementation with 20% heavy hitters improving throughput by up to 29× over DeepSpeed Zero-Inference and Hugging Face Accelerate and up to 3× over FlexGen on OPT-6.7B and OPT-30B, and reducing latency by up to 1.9× at equal batch size [47]. Those baselines are offload-based and weak relative to modern engines, and the speedup should be read in that light.

Practical issues with the accumulated score deserve attention. (i) It is *biased toward old tokens*, which have had more steps to accumulate; normalising by the number of observations helps. (ii) It requires explicit attention probabilities, which fused FlashAttention kernels never materialise; implementations either compute probabilities in a separate pass, or use methods (like SnapKV) that need them only for a small window. (iii) With GQA, the score must be pooled over the g query heads of a group, or each head's retention must be aligned. (iv) During *prefill* the accumulated scores over a long prompt cost $\mathcal{O}(n^2)$ memory unless computed blockwise.

Algorithm 4 H_2O-style decode-time eviction (one head; budget k, recent window r).

Require: cache of $m \le k$ entries with accumulated scores $a_1, \dots, a_m$; new query $\mathbf{q}_t$, new pair $(\mathbf{k}_t, \mathbf{v}_t)$

append the new pair with $a_{m+1} \leftarrow 0$
compute $\mathbf{p} \leftarrow \mathrm{softmax}(\mathbf{q}_t^\top \mathbf{K}^\top / \sqrt{d_h})$ over the $m+1$ entries
$\mathbf{o}_t \leftarrow \mathbf{p}^\top \mathbf{V}$
$a_s \leftarrow a_s + p_s$ for all s ▷ accumulate
if $m + 1 > k$ **then**
    $\mathcal{E} \leftarrow$ entries outside the last r positions
    $s^\star \leftarrow \arg\min_{s \in \mathcal{E}} a_s$; remove entry $s^\star$ ▷ evict lowest accumulated score
end if
return $\mathbf{o}_t$

6.3.2 Scissorhands, TOVA, Keyformer, norm-based scores

Scissorhands proposes the *persistence of importance* hypothesis: tokens that were influential in the recent past tend to remain influential, so one can keep the tokens that received attention above average in a history window and drop the rest. It reports compressing the cache by up to 5× without loss in its evaluation, and up to 20× when combined with 4-bit quantization [48]. TOVA recasts a transformer with a bounded cache as a *multi-state RNN* and evicts the token with the lowest attention weight at the current step, with no accumulation, showing that a very simple policy approaches full-cache quality on several tasks [56]. Keyformer identifies "key tokens" and regularises the score distribution with injected noise to counter the fact that evicting tokens changes the softmax distribution [57]. A simple norm-based rule keeps tokens whose keys have *low* ℓ_2 norm, since in several models low key norm correlates with high attention; it needs no attention scores and is thus compatible with fused kernels [58]. By Equation (2.10) the sign of the correlation depends on how key norms enter the logit and is model-dependent; the rule should be validated per model.

6.4 Prefill-time compression

A dominant use of eviction is to compress the cache *once*, after the prompt has been processed, so that decoding runs with a small cache. Two properties make this attractive: the prompt is processed with exact attention (no approximation during prefill) and compression is applied before any generation.

6.4.1 SnapKV

SnapKV observes that the attention that the *last* part of the prompt (an observation window of w_o tokens, which usually contains the question) pays to earlier prompt tokens is already predictive of which tokens the generated answer will use [49]. For each head it sums these attention weights over the window, smooths the resulting scores with a pooling kernel so that clusters of neighbouring tokens (rather than isolated tokens) are kept, and retains the top-k positions together with the window itself. The authors report, on their setup, a 3.6× generation speedup and 8.2× memory-efficiency improvement at 16K input length, and the ability to

Algorithm 5 SnapKV prefill compression for one head (budget k, window w_o, pooling kernel width κ).

Require: prompt keys/values $\mathbf{K}, \mathbf{V} \in \mathbb{R}^{n \times d_h}$; observation-window queries $\mathbf{Q}_{\text{obs}} \in \mathbb{R}^{w_o \times d_h}$ (the last w_o prompt positions)

$\mathbf{P} \leftarrow \text{softmax}\left(\mathbf{Q}_{\text{obs}} \mathbf{K}_{1:n-w_o}^{\top} / \sqrt{d_h}\right)$ ▷ window's attention over the prefix

$\mathbf{a} \leftarrow \sum_{\text{rows}} \mathbf{P}$ ▷ vote: one score per prefix position

$\tilde{\mathbf{a}} \leftarrow \text{AvgPool1D}(\mathbf{a}; \kappa)$ ▷ cluster neighbouring positions

$\mathcal{I} \leftarrow \text{TopK}(\tilde{\mathbf{a}},\ k - w_o)$

return $\mathbf{K}[\mathcal{I} \cup \{n - w_o + 1..n\}],\ \mathbf{V}[\mathcal{I} \cup \{n - w_o + 1..n\}]$

process up to 380K-token contexts on a single 80 GB GPU with a small drop on a needle test [49].

SnapKV needs the question inside the prompt at prefill time: it is *query-aware*. If the question arrives later (multi-turn use, shared document prefix with many questions), the observation window does not contain it and the retained set may omit what the next question needs. KVzip addresses this below.

6.4.2 Layer and head budgets: PyramidKV, Ada-KV, HeadKV

Uniform budgets ignore that different layers and heads have different attention concentration (Proposition 2.4). PyramidKV observes that attention in lower layers is broad and funnels to a few tokens in upper layers, and so assigns *larger budgets to lower layers and smaller ones to higher layers*, decreasing arithmetically with depth; the authors report matching full-cache performance on LongBench with about 12% of the cache and a clear advantage over uniform allocation at extreme compression (0.7% retention) [50]. PyramidInfer applies a related reasoning during prefill itself, shrinking the number of retained tokens as layers deepen [51]. Ada-KV allocates budgets *across heads within a layer* by minimising an upper bound on the eviction loss, which depends on the sharpness of each head's attention [53]; HeadKV scores heads by their retrieval and reasoning importance and allocates budgets accordingly [54]. ReasonAlloc extends budget allocation to decoding: an offline layer-wise "wave" pattern combined with online head reallocation, reporting largest gains at budgets of 128–512 tokens on reasoning benchmarks [71].

Proposition 6.2 (Why per-head budgets help)**.** *Let head h have attention mass $\epsilon_h(k_h)$ outside its top-k_h tokens, with ϵ_h convex and decreasing, and suppose the output error of head h is bounded by $2V\epsilon_h(k_h)$ (Proposition 11.1 below). Then for a total budget $\sum_h k_h = K$ the allocation minimising $\sum_h \epsilon_h(k_h)$ equalises marginal gains, $-\epsilon_h'(k_h) = \lambda$ for all heads with $k_h > 0$; equal budgets $k_h = K/H$ are optimal only if all ϵ_h are identical.*

Proof. This is a standard convex resource-allocation problem; the Lagrangian condition $-\epsilon_h'(k_h) = \lambda$ is necessary and sufficient under convexity, and with identical ϵ_h the symmetric point satisfies it. For heterogeneous heads (e.g., $\epsilon_h(k) = (k+1)^{1-\alpha_h}$ with different exponents) equal k_h gives unequal marginals and is therefore not optimal. □

6.4.3 KVzip: query-agnostic compression by reconstruction

KVzip scores each cached entry by how much the model attends to it when asked to *reconstruct the context itself* (a "repeat the previous text" task), and keeps entries with the highest maximum attention across the reconstruction pass [69]. Because the scoring prompt does not depend on any downstream question, the compressed cache is reusable across queries, which the authors verify by evaluating several different questions on the same compressed context. The authors report cache reductions of about three to four times and decoding-latency reductions of about two times with negligible degradation on question answering, retrieval, reasoning and code tasks, over contexts up to 170K tokens [69]. The authors also report that query-aware eviction methods degrade even at a 90% cache budget in multi-query scenarios, which is the failure mode KVzip is designed to avoid. The cost is an additional pass over the context (for the scoring), so it pays off when the compressed cache is reused. The method is empirical: it exploits the observation that tokens the model needs to reproduce are tokens it can answer questions about, which cannot hold for arbitrary queries (Section 11.4); it is, in effect, a better prior on the query distribution.

6.5 Adaptive and head-specialised policies

6.5.1 FastGen

FastGen profiles each head's attention during prefill and assigns it one of a handful of policies: keep only special tokens, keep punctuation, keep only local tokens, keep frequently attended tokens, or keep everything [52]. The choice is made per head to recover a target fraction of the attention mass. It is training-free and adaptive but its decisions are fixed after profiling.

6.5.2 Retrieval versus streaming heads: DuoAttention and relatives

Mechanistic work found that a small subset of heads, *retrieval heads*, perform the copy-from-context operations that long-context question answering needs, and that most heads attend locally [187, 188]. DuoAttention turns this into a policy: each head h gets a learned scalar gate $\alpha_h \in [0, 1]$ and the mixed output

$$\mathbf{o}_h = \alpha_h \,\mathrm{Attn}_{\mathrm{full}} + (1 - \alpha_h)\,\mathrm{Attn}_{\mathrm{stream}}, \tag{6.3}$$

where $\mathrm{Attn}_{\mathrm{stream}}$ attends only to sink and recent tokens. The gates are optimised (with the model frozen) on synthetic retrieval data by a distillation loss between the full model's and the mixed model's outputs plus an ℓ_1 penalty on α_h; after training, heads with α_h above a threshold are labelled retrieval heads and keep a full cache while the others keep only sinks and a window [68]. The authors report, as measured on Llama-2/3 style models, memory reductions of up to 2.55× for MHA and 1.67× for GQA models, decoding speedups of up to 2.18× and 1.50×, pre-filling speedups of up to 1.73× and 1.63×, and, in combination with quantization, a context of 3.3 million tokens for Llama-3-8B on one 80 GB GPU [68]. RazorAttention similarly keeps retrieval heads whole and compresses the rest, adding a "compensation" token that summarises the dropped entries [55].

The headline lesson of this line is that heads are heterogeneous in function, so *head-level* policy outperforms uniform policy (Proposition 6.2). A risk is that the retrieval/streaming labels are

Algorithm 6 DuoAttention: offline head classification and online cache policy.

Require: frozen model; synthetic long-context retrieval data $\mathcal{D}$; sparsity weight λ; threshold τ

initialise gate $\alpha_h \leftarrow 1$ for every KV head h
repeat ▷ offline, only the gates are trained
  sample sequence $x \in \mathcal{D}$
  full-model hidden states $\mathbf{h}^{\text{full}}$ with all heads full
  mixed-model hidden states $\mathbf{h}^{\text{mix}}$ with head h using $\alpha_h \text{Attn}_{\text{full}} + (1-\alpha_h)\text{Attn}_{\text{stream}}$
  minimise $\left\|\mathbf{h}^{\text{full}} - \mathbf{h}^{\text{mix}}\right\|_2^2 + \lambda \sum_h |\alpha_h|$ over $\{\alpha_h\}$
until converged
$\mathcal{R} \leftarrow \{h : \alpha_h > \tau\}$ ▷ retrieval heads
for each decode step **do**
  heads in $\mathcal{R}$: standard full-cache attention
  other heads: keep s_0 sinks and w recent entries (Algorithm 3)
end for

determined on a specific synthetic distribution; heads that are inert on that data but needed for other skills (long-range reasoning, aggregation) may be mislabelled streaming and lose their long-range access. DuoAttention's authors evaluate on several benchmarks beyond needle tests; the independent diagnostic study of Chapter 12 finds that mechanism-level failures remain common for compression methods in general [168].

6.6 Query-aware selection: keep everything, read little

Selection methods keep the full cache but choose, at each decode step, a subset to read. They reduce the bandwidth bill but not the capacity bill, and (when the unselected entries live in CPU memory) turn the capacity bill into a transfer bill.

6.6.1 Quest: page-level upper bounds

Quest partitions the cache into pages of P tokens and stores, for each page, the elementwise minimum $\mathbf{m}$ and maximum $\mathbf{M}$ of its keys [61]. For a query $\mathbf{q}$ the *criticality score* of a page is

$$U(\mathbf{q}; \text{page}) = \sum_{j=1}^{d_h} \max\left(q_j m_j,\ q_j M_j\right), \tag{6.4}$$

and the top-K pages by U are read.

Proposition 6.3 (Quest score is an upper bound)**.** *For every key* $\mathbf{k}$ *in the page,* $\mathbf{q}^\top \mathbf{k} \le U(\mathbf{q}; \textit{page})$.

Proof. Each coordinate satisfies $m_j \le k_j \le M_j$. Hence $q_j k_j \le \max(q_j m_j, q_j M_j)$, because $q_j k_j$ is linear in k_j and so is maximised at an endpoint of $[m_j, M_j]$. Summing over j gives the claim. □

The bound is tight only when the coordinatewise extremes are attained by a single key, so it can overestimate the true best key in the page substantially, which makes the criterion conservative:

a page is read if it *could* contain a high-scoring key. The authors report up to 7.03× self-attention speedup and 2.23× end-to-end latency reduction at long context with small accuracy loss on language modelling, passkey retrieval and long-context QA [61]. The page granularity matters: smaller pages tighten the bound but add metadata (two extra vectors per page, a metadata overhead of $2/P$ of the cache).

6.6.2 Other selectors

- **SparQ attention** approximates the scores with only the r largest-magnitude channels of the query, reads those channels of all keys, selects the top tokens, then runs exact attention on them [62]. It reduces bandwidth by reading a fraction of key channels first.
- **InfLLM** divides the context into blocks, represents each by a few salient tokens, and retrieves relevant blocks from a context memory, which may be held on the CPU [63].
- **RetrievalAttention** builds an approximate-nearest-neighbour index over keys and observes that queries are out-of-distribution relative to keys, which makes naive ANN search inefficient and motivates attention-aware indexes [65].
- **MagicPIG** argues that top-k is a poor estimator of the full attention output because keys outside the top-k still carry non-negligible mass, and replaces it by locality-sensitive-hashing-based *sampling* with an unbiased estimator, run on CPU for the cache [66].
- **ShadowKV** keeps a low-rank representation of the keys on the GPU, stores values off-device, and reconstructs selected keys on the fly using landmark chunks to select tokens [67].
- **InfiniGen** speculatively prefetches the entries that will be needed in the next layer using a partial computation of the next layer's query, hiding offload latency [64].

6.6.3 Selection versus eviction

Proposition 6.4 (Selection is at least as accurate as eviction at equal read budget)**.** *Fix a per-query read budget k. A selector that chooses the k entries with the largest softmax weights for* each *query has output error no larger than any eviction policy that fixes its retained set $\mathcal{S}$, $|\mathcal{S}| = k$, before seeing the query.*

Proof. For any query, the set of the k largest weights minimises the dropped mass $\epsilon(\mathcal{S}) = \sum_{s \notin \mathcal{S}} p_s$ over all sets of size k. By Proposition 11.1 the output error is governed by ϵ, so choosing the minimiser per query is no worse than any fixed $\mathcal{S}$. □

The caveat is cost: the selector must *find* the top-k without reading all keys, and approximate search (page bounds, channel sparsity, ANN, LSH) introduces its own errors and overheads. Exact top-k over all keys reads all keys and so loses the bandwidth benefit. Selection also keeps capacity unchanged: it does not enable larger batches unless combined with offload.

6.7 Decode-time compression for reasoning traces

Reasoning models produce long generations whose cache grows during decoding, not only at prefill. Prefill-time methods cannot help with it. Several methods adapt eviction to this regime.

- **R-KV** scores tokens by a combination of *importance* (attention) and *redundancy* (similarity of keys to other tokens), since reasoning traces repeat, rephrase and verify; redundant tokens are dropped first [70]. The authors evaluate on mathematical reasoning with distilled R1-style models and report near full-cache accuracy at a small fraction of the cache.
- **ThinKV** classifies segments of the trace by the type of thought and allocates compression accordingly [72].
- **ReasonAlloc** reallocates budgets across layers and heads during decoding [71].

These results are primarily on short-answer mathematical benchmarks (MATH-500, AIME) with distilled models up to 14B parameters. We discuss the evidence base and its limits in Chapters 12 and 14: long reasoning traces contain *intermediate results that later steps depend on*, and failures can be silent.

6.8 Learned eviction and merging-by-gating

Dynamic Memory Compression (DMC) retrofits a pretrained model with a learned, per-head decision at each step to *append* the new key–value pair or *merge* it into the last cache entry by weighted averaging, achieving cache compression of several fold after a short continued-pretraining phase [60]. Dynamic context pruning learns a sparse gating that drops uninformative tokens during training of the language model itself [59]. These are the bridge to the trainable sparse designs of Chapter 7: once the policy is learned jointly with the model rather than imposed afterward, the train–inference mismatch disappears.

6.9 Comparison

6.10 Engineering notes

1. **Ragged caches.** Per-head and per-layer budgets make cache lengths differ across heads, which breaks the regular-tensor assumption of batched kernels. Systems pad to the longest head (wasting the saving), flatten into variable-length segments (as Ada-KV does), or use page tables (Chapter 9).
2. **Position handling.** After eviction, absolute positions of retained keys no longer form a contiguous range. Options are to keep original positions (the relative distances remain true but can exceed trained range), or to reassign cache-relative positions (StreamingLLM), which compresses distances. The choice interacts with the RoPE scaling scheme in use (Section 10.7).
3. **GQA.** Eviction scores should be aggregated over the query heads sharing a KV head, or the group will retain different sets for different queries. Early methods developed on MHA models needed adaptation when GQA became the norm; the benefit of eviction relative to the already-reduced GQA cache is correspondingly smaller (the 4–16× saved by GQA is not available to stack with eviction on the same tokens).
4. **Prefix sharing.** Methods that prune per request break prefix caching (Chapter 9).

Table 6.1: Comparison of training-free token-level methods. Headline claims are *as reported by the authors* in their own setups and are not comparable across rows. FA = compatible with fused FlashAttention-style kernels without materialising attention probabilities (author's assessment).

Method	Signal	Timing	Query-aware	Headline claim (cited)
StreamingLLM [46]	position (sinks + recent)	decode	no	stable at 4M tokens; up to 22.2× over window recompute
H_2O [47]	accumulated attention	decode	no	20% heavy hitters; up to 29× (3× vs. FlexGen) throughput
Scissorhands [48]	persistence of importance	decode	no	up to 5× (20× with 4-bit)
TOVA [56]	latest attention	decode	no	multi-state RNN view
SnapKV [49]	observation-window votes	prefill	yes	3.6× speed, 8.2× memory at 16K
PyramidKV [50]	window votes + layer pyramid	prefill	yes	∼12% cache matches full on LongBench
Ada-KV [53]	head-adaptive budgets	prefill	yes	improves SnapKV/PyramidKV
FastGen [52]	per-head profiled policy	prefill	no	adaptive per head
KVzip [69]	reconstruction attention	prefill	no	3–4× cache, ∼2× latency
DuoAttention [68]	head gate (retrieval vs. streaming)	prefill+decode	no	up to 2.55× memory (MHA)
Quest [61]	page min/max bound	decode (select)	yes	up to 7.03× attention speedup
R-KV [70]	importance + redundancy	decode	no	reasoning traces at fraction of cache

Chapter summary

- Removing a token changes the output by $\frac{p_i}{1-p_i}(\mathbf{o} - \mathbf{v}_i)$ (Proposition 6.1); all scores estimate expected future attention; few use the value term.
- Sinks are an artefact of softmax normalisation; StreamingLLM stabilises streams but does not remember.
- Prefill-time methods (SnapKV, PyramidKV, KVzip) are strongest when the query is known (or, for KVzip, when the context is reused); head-adaptive allocation is consistently beneficial.
- Selection is more accurate than eviction at equal read budget, but keeps capacity and needs a cheap, accurate top-k finder.
- Reasoning traces motivate new decode-time methods; the evidence is short-answer mathematics on mid-size distilled models.

Chapter 7

Trainable Sparse Attention

Chapter 6 treated sparsity as something imposed on a model trained to be dense. This chapter treats sparsity as something the model is *trained* to use. The motivation is a mismatch argument: a model pretrained with dense attention learns to rely on all of its context in ways that no post-hoc selector can fully recover, whereas a model trained with a selection mechanism in the loop learns to put what it needs where the selector will find it, and to train the selector itself. Since 2025 this has moved from research to flagship deployment: DeepSeek-V3.2 introduced DeepSeek Sparse Attention [6], and DeepSeek-V4 extended it with compression to a one-million-token window [7].

7.1 Design requirements

A trainable sparse attention layer must satisfy four requirements that dense attention does not.

1. **Differentiability or a surrogate.** Selecting the top-k keys is a hard operation. Either gradients flow through a soft part (NSA's compression branch), the selector is trained by distillation from dense attention (DSA's indexer), or the selection is simply non-differentiable and only the attention over selected entries is trained (MoBA).
2. **Hardware alignment.** Token-granular gathers waste memory bandwidth on partial cache lines; block-granular selection amortises metadata and gives contiguous reads. Selection shared across the query heads of a GQA group (or all heads of an MQA-form MLA layer) lets one gather serve many heads, preserving arithmetic intensity (Proposition 2.2).
3. **A cheap scorer.** If scoring all n keys costs as much as dense attention, nothing is saved; the scorer must use fewer heads, lower precision, or a coarser representation.
4. **Coverage of local context.** Almost all designs include a sliding-window or local branch because local dependencies are dense and selection is inefficient for them.

7.2 Native sparse attention (NSA)

NSA [73] combines three attention branches with a learned gate, for each query position t:

$$\mathbf{o}_t = \sum_{c \in \{\text{cmp},\text{slc},\text{win}\}} g_t^c \, \text{Attn}\big(\mathbf{q}_t, \tilde{\mathbf{K}}_t^c, \tilde{\mathbf{V}}_t^c\big), \qquad g_t^c = \sigma(\text{MLP}(\mathbf{h}_t)) \in [0,1]. \tag{7.1}$$

- **Compression.** The sequence is split into overlapping blocks (length l, stride d) and each block is mapped to *one* key and *one* value by a learnable MLP with intra-block positional information. Attention over these $\approx n/d$ block summaries provides a coarse global view.
- **Selection.** The attention scores $\mathbf{p}_t^{\text{cmp}}$ of the compression branch double as importance scores for *selection blocks* of length l' (these are mapped from compression blocks). For GQA, the scores are summed over the query heads of the group so that all of them select

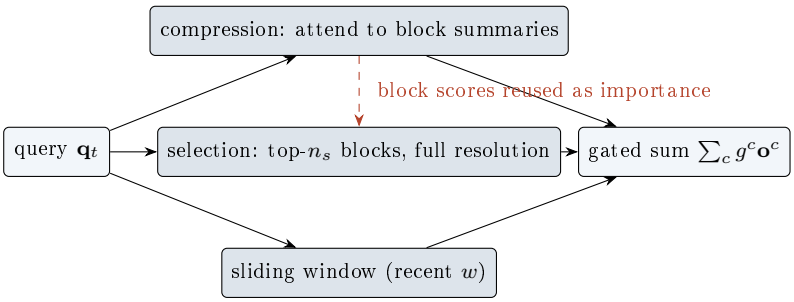


Figure 7.1: Schematic of native sparse attention [73]. The compression branch both provides a coarse global view and scores blocks for the selection branch.

Algorithm 7 NSA attention for one query (single head group; simplified).

Require: query $\mathbf{q}_t$; keys/values $\mathbf{K}, \mathbf{V}$ up to position t; block length l, stride d; selection block l'; count n_s; window w

1: $\tilde{\mathbf{K}}, \tilde{\mathbf{V}} \leftarrow \mathrm{MLP}_{\mathrm{cmp}}$(blocks of $\mathbf{K}, \mathbf{V}$) ▷ cached incrementally: one summary per stride
2: $\mathbf{p}^{\mathrm{cmp}} \leftarrow \mathrm{softmax}(\mathbf{q}_t^\top \tilde{\mathbf{K}}^\top / \sqrt{d_h})$; $\mathbf{o}^{\mathrm{cmp}} \leftarrow \mathbf{p}^{\mathrm{cmp}\top} \tilde{\mathbf{V}}$
3: map compression scores to selection-block scores $\mathbf{p}^{\mathrm{slc}}$ (sum over overlapping compressed blocks); sum over query heads in the group
4: $\mathcal{B} \leftarrow$ indices of the top-n_s selection blocks by $\mathbf{p}^{\mathrm{slc}}$ (including initial and local blocks)
5: $\mathbf{o}^{\mathrm{slc}} \leftarrow \mathrm{Attn}(\mathbf{q}_t, \mathbf{K}[\mathcal{B}], \mathbf{V}[\mathcal{B}])$
6: $\mathbf{o}^{\mathrm{win}} \leftarrow \mathrm{Attn}(\mathbf{q}_t, \mathbf{K}_{t-w+1:t}, \mathbf{V}_{t-w+1:t})$
7: $g^c \leftarrow \sigma(\mathrm{MLP}(\mathbf{h}_t))$ for $c \in \{\mathrm{cmp}, \mathrm{slc}, \mathrm{win}\}$
8: **return** $g^{\mathrm{cmp}}\mathbf{o}^{\mathrm{cmp}} + g^{\mathrm{slc}}\mathbf{o}^{\mathrm{slc}} + g^{\mathrm{win}}\mathbf{o}^{\mathrm{win}}$

the same blocks. The top-n_s blocks are gathered at full resolution (original keys and values) and attended to.

- **Sliding window.** The most recent w tokens are attended directly.

In the paper's configuration, as we recall it from the paper body and did not re-verify, compression blocks have 32 tokens with stride 16, selection blocks 64 tokens, 16 selected blocks per query and a 512-token window [73]; each query therefore attends to a few thousand "tokens-worth" of keys independent of n plus the $\mathcal{O}(n/d)$ compressed ones.

Evidence. The NSA authors pretrain a mixture-of-experts model with NSA (we did not re-verify its exact size from the primary text; our recollection is a model of tens of billions of total and a few billion active parameters) and report that it matches or exceeds a dense-attention model on general, long-context and reasoning benchmarks, and that their kernels give substantial speedups over full attention on 64K-token sequences in decoding, forward and backward passes [73]; the abstract states no single speedup factor, and larger factors appear in the paper's figures, which we do not quote. These are the authors' own numbers on a single model scale; the test of adoption is whether the design appears in flagship models.

Capacity. NSA's selection branch needs full-resolution keys and values of *any* block that might be selected; the original cache must therefore be stored (on device or offloaded). NSA lowers compute and decode bandwidth, not capacity, unless coupled with offloading.

7.3 DeepSeek Sparse Attention and the lightning indexer

7.3.1 Mechanism

DSA was introduced in the V3.2-Exp preview and described in the V3.2 report [6]. It is built on MLA in its MQA (absorbed) form, so that one cached latent per token is shared by all query heads. For each query token t a *lightning indexer* computes an index score against every previous token s:

$$I_{t,s} = \sum_{j=1}^{H^I} w^I_{t,j}\ \mathrm{ReLU}\big(\mathbf{q}^I_{t,j} \cdot \mathbf{k}^I_s\big), \tag{7.2}$$

where H^I is a small number of indexer heads, the indexer queries $\mathbf{q}^I_{t,j}$ and head weights $w^I_{t,j}$ are computed from the hidden state, and the indexer key $\mathbf{k}^I_s$ is a single shared vector per token. The ReLU replaces softmax (avoiding exponentials and normalisation) and the computation is run in low precision. The top-k tokens (the paper uses $k = 2048$) by $I_{t,\cdot}$ are selected and the main attention is computed *only* over their latent entries. The main attention costs $\mathcal{O}(nk)$ per sequence (rather than $\mathcal{O}(n^2)$); the indexer still scores all previous tokens, $\mathcal{O}(n^2)$ overall but with a small constant: few heads, small dimension, low precision [6].

7.3.2 Training

DSA is added to an already pretrained dense MLA model by *continued pretraining* in two stages [6]: (1) a short *dense warm-up* with the main model frozen, in which only the indexer is trained so that its normalised scores match the main attention distribution (aggregated over heads) by a KL-divergence loss; (2) a *sparse stage*, in which the full model is trained with top-k selection active, with the indexer receiving a KL loss restricted to the selected set and the rest of the model receiving only the language-modelling loss. The indexer is thus *distilled* from the dense attention rather than trained end-to-end through the selection. The V3.2 report states that quality on standard and long-context benchmarks was not materially degraded relative to its dense predecessor, and that inference cost at long context fell substantially [6].

Proposition 7.1 (Selection error is bounded by the indexer's miss mass)**.** *Let p_s be the true main-attention weights of a head for a query, and let the indexer select a set $\mathcal{S}$ of k tokens. Define the* miss mass $\mu = \sum_{s \notin \mathcal{S}} p_s$*. If $\|\mathbf{v}_s\| \le V$ for all s, the output of attention restricted to $\mathcal{S}$ satisfies $\|\mathbf{o}_{\mathcal{S}} - \mathbf{o}\| \le 2V\mu$, and in particular $\mu \ge \mu^\star := 1 - \sum_{s \in \mathrm{Top}_k(p)} p_s$ with equality iff the indexer selects exactly the top-k tokens by true weight.*

Proof. The bound is Proposition 11.1 applied with the dropped mass μ. For the second statement, the top-k set maximises $\sum_{s \in \mathcal{S}} p_s$ over all sets of size k, so any other set has smaller captured mass and therefore larger μ. □

The KL objective of the indexer is a proxy for minimising μ over the data distribution. A KL divergence is mass-covering in the wrong direction for this purpose when the indexer distribution is flat (it can match the *shape* of the main distribution on average but still rank the individually critical token below k); this is the typical failure of learned indexers on rare, single-token retrievals. Whether this matters in agentic workloads is an open evaluation question (Chapter 12).

7.3.3 A cost model for DSA

Let the main cache hold L tokens with c_{main} bytes per token per layer, and the indexer hold c_{idx} bytes of indexer key per token per layer. A dense decode reads Lc_{main} bytes per layer. A DSA decode reads $Lc_{\text{idx}} + kc_{\text{main}}$:

$$\frac{\text{bytes}_{\text{DSA}}}{\text{bytes}_{\text{dense}}} = \frac{c_{\text{idx}}}{c_{\text{main}}} + \frac{k}{L}. \tag{7.3}$$

Example 7.1 (Illustrative DSA byte count)**.** Assume (from the V3.2 design as we understand it) an indexer key of 128 dimensions in 8-bit precision, i.e., $c_{\text{idx}} = 128$ B, and a bf16 main entry of $c_{\text{main}} = 1152$ B. With $k = 2048$ and $L = 131{,}072$: $128/1152 + 2048/131072 = 0.111 + 0.016 = 0.127$, a 7.9× reduction in bytes read per layer (derived under these assumptions; the real figure depends on the exact indexer width, precision and the cache layout, and the indexer keys must themselves be stored: $128/1152 \approx 11\%$ extra capacity).

The formula exposes the structure of the saving: for large L, the *indexer's scan* of the full context becomes the bottleneck (the $c_{\text{idx}}/c_{\text{main}}$ term), which is why DeepSeek-V4 compresses the sequence before indexing (Section 7.5). On the FLOP side, with 64 indexer heads of width 128 (assumed), scoring costs $2 \cdot 64 \cdot 128 = 16{,}384$ FLOPs per token against $\approx 2.8 \times 10^5$ for dense absorbed MLA attention (Proposition 2.3), a 17× ratio; the sparse main attention then runs on k tokens instead of L (derived).

7.3.4 What DSA does and does not reduce

DSA stores the same MLA latent cache as V3 plus the indexer keys, so *capacity does not fall*; what falls is the bandwidth and compute of attention at long context (and, via the cheaper per-token cost, the price per token). Its significance is that it demonstrated that token-level sparse selection could be bolted onto a frontier dense model by a modest amount of continued pretraining with preserved quality, a recipe other labs can follow; reports indicate that GLM-5 adopted DSA with MLA [214].

7.4 Mixture of block attention (MoBA)

MoBA [74] applies the routing idea of mixture-of-experts to attention. The context is divided into blocks of fixed size; for query $\mathbf{q}_t$ and block i with keys $\mathbf{K}^{(i)}$, the routing score is the inner product between the query and the *mean-pooled key of the block*,

$$s_{t,i} = \mathbf{q}_t^\top \operatorname{mean}\big(\mathbf{K}^{(i)}\big), \tag{7.4}$$

and the query attends (with exact softmax over all selected tokens) to the top-k blocks together with its own current block, with causality preserved by masking future blocks. The scoring cost is $\mathcal{O}(n/\text{block size})$ per query. Two properties are emphasised by its authors: (i) it can switch between sparse and full attention layers (for example, full attention in a few layers, MoBA in the others, or sparse during most of training and dense at the end), enabling a smooth transition from dense models; (ii) it is implemented on top of FlashAttention-style block kernels. The abstract reports superior long-context performance and states that the method has already been deployed to support Kimi's long-context requests [74]; the paper body reports efficiency gains at long lengths, but we did not verify a specific speedup factor and do not quote one.

Proposition 7.2 (Mean-pooled routing is a lossy proxy for max-pooled routing)**.** *For a block with keys* $\mathbf{k}_1, \dots, \mathbf{k}_P$*, the routing score* $\mathbf{q}^\top \bar{\mathbf{k}}$ *with* $\bar{\mathbf{k}} = \frac{1}{P}\sum_i \mathbf{k}_i$ *equals the* average *logit of the block, whereas the attention mass assigned to the block is governed by the* log-sum-exp *of its logits. By Jensen's inequality* $\frac{1}{P}\sum_i s_i \le \log\left(\frac{1}{P}\sum_i e^{s_i}\right)$ *with strict inequality unless all* s_i *are equal, and the gap grows with the within-block logit variance (to second order,* $\frac{1}{2}\operatorname{Var}_i(s_i)$*).*

Proof. Jensen's inequality for the convex function exp gives $e^{\bar{s}} \le \frac{1}{P}\sum_i e^{s_i}$; taking logs gives the claim. A second-order expansion of $\log \mathbb{E}e^s$ around the mean gives $\bar{s} + \frac{1}{2}\operatorname{Var}(s) + \cdots$. □

The consequence is that a block containing one very relevant token among many irrelevant ones has a low mean score and may not be routed to, although it carries high attention mass. Mean-pooled routing therefore favours blocks that are *uniformly* relevant over blocks with a single sharp match; compression-based scoring (NSA) and token-level indexers (DSA) are more robust to this. Quest's min/max bound (Proposition 6.3) is a conservative alternative with the opposite bias.

7.5 DeepSeek-V4: compressed sparse and heavily compressed attention

DeepSeek-V4, in two sizes (V4-Pro with 1.6T total/49B active parameters and V4-Flash with 284B/13B), is designed for a native one-million-token window [7]. According to the report and a companion summary [213], its attention alternates two layer types.

- **Compressed Sparse Attention (CSA).** Key–value entries are compressed 4× along the sequence by a softmax-gated pooling with a learned positional bias; a lightning indexer (low-precision, ReLU-scored, multi-head) selects the top-k *compressed* entries per query; a sliding-window branch covers the most recent uncompressed tokens.
- **Heavily Compressed Attention (HCA).** Entries are compressed 128× and attended densely (no selection), again with a sliding window for recent tokens.

In the V4-Pro configuration the first layers are HCA and the remainder alternate CSA and HCA; KV entries are mostly stored in 8-bit floating point with higher precision reserved for the rotary dimensions, and the indexer runs in 4-bit floating point [213]. Relative to DeepSeek-V3.2 at a one-million-token context, V4-Pro is reported to need about 27% of the single-token inference FLOPs and about 10% of the KV cache, and V4-Flash about 10% of the FLOPs and 7% of the cache; the report's companion states the cache is roughly 2% of that of a bf16 GQA model with 8 KV heads [7, 213].

Plausibility check. A back-of-envelope calculation helps place these numbers. If half the layers store 1/4 as many entries as dense MLA (CSA) and half store 1/128 (HCA), the average entry count is $\frac{1}{2}(\frac{1}{4}+\frac{1}{128}) \approx 0.129$ of dense, before the sliding-window entries (constant in n), the indexer's keys and the change of storage precision. The reported ≈7–10% is the same order of magnitude, as expected given the precision changes and unknown head-width differences from V3.2. We stress that this is a plausibility check, not a reproduction.

What is new. V4 is the first widely documented design in which *both* capacity and compute are cut by learned compression of the sequence dimension, combined with learned sparse selec-

tion. The compression is differentiable and trained jointly (unlike post-hoc merging, Chapter 8), and the extreme 128× layers behave like a very coarse, dense summary, with the CSA layers and window branches supplying fine-grained detail. It parallels NSA's compression-plus-selection-plus-window triad, with MLA-style latent entries and an 8-bit/4-bit pipeline.

7.6 Training-free dynamic sparse attention for prefill

For prefill, where the quadratic cost bites, a family of methods finds sparse patterns *without* training.

- **MInference** observes that long-context attention matrices fall into a few recurrent patterns (A-shaped, vertical-slash, block-sparse), assigns each head a pattern offline and estimates its indices online with a cheap approximation, then computes with sparse kernels [76].
- **FlexPrefill** chooses patterns and budgets per head and per input adaptively, using a measure of the divergence between pooled and full attention [77].
- **XAttention** scores blocks by summing along antidiagonals of the attention matrix, a cheap but informative proxy for block importance, then runs block-sparse attention [78].
- **SeerAttention** *learns* a lightweight gate (a plug-in module trained by distillation from dense attention) that predicts block-level sparsity, with the base model frozen [75].
- **LServe** combines static sparsity (streaming heads) and dynamic sparsity (page selection with hierarchical paging) in a unified system [79].

These methods are *post-hoc* with respect to the dense model but *pattern-aware*; they are the sparse-attention analogue of the eviction methods of Chapter 6, applied to prefill compute rather than to cache size, and their accuracy is generally reported on retrieval and long-context benchmarks rather than long-generation tasks.

7.7 Comparison of trainable sparse designs

7.8 Why training the selector helps: a simple model

Consider a one-dimensional caricature in which a head must retrieve a "needle" value placed at an unknown position among n distractors. A post-hoc selector with a fixed scoring function $f(\mathbf{q}, \mathbf{k})$ succeeds only if the pretrained representation makes the needle score in the top-k. A model trained jointly with the selector can change the representation to make this true (for instance, by learning a keying scheme in which retrieval-relevant tokens have large indexer scores). Formally, post-hoc selection solves $\min_{\mathcal{S}} \text{loss}(\theta_0, \mathcal{S})$ for a fixed θ_0 trained under dense attention, while joint training solves $\min_{\theta,\phi} \text{loss}(\theta, \mathcal{S}_\phi)$, and since (θ_0, ϕ) is feasible for the latter, its optimum is no worse. In practice the comparison is confounded by compute and data budgets, and by the fact that DSA is added to a dense model by continued pretraining, so the gap is not cleanly measurable; the NSA authors' pretraining-from-scratch result is the cleanest published evidence that native sparsity can match dense quality at their scale [73].

Table 7.1: Trainable sparse attention designs. "Selection" is the granularity of what the query attends at full resolution; "Cap." indicates whether the *stored* cache shrinks. Author's summary of the cited papers.

Design	Selection signal	Training of selector	Cap. reduced?	Notes
NSA [73]	compression-branch scores, block level	end-to-end, via compression branch	no	three branches with gating; Triton kernels; pretrained from scratch
DSA [6]	lightning indexer, token level (top-2048)	KL distillation from dense attention	no (adds indexer keys)	continued pretraining from dense MLA model
MoBA [74]	query · mean key of block	none (parameter-free gate); model trained with it	no	switchable dense/sparse; FlashAttention-based
CSA/HCA (V4) [7]	indexer over 4×-compressed entries; 128× dense	joint	yes (4×/128× compression)	alternating layers, 1M context
SeerAttention [75]	learned block gate	distillation, frozen base	no	plug-in for existing models
MInference/ FlexPrefill/ XAttention	pattern or antidiagonal scores	none	no	prefill compute only

Chapter summary

- Trainable sparse attention removes the train–inference mismatch of post-hoc eviction by training with selection in the loop; designs differ in how the selector is trained (end-to-end, distilled, parameter-free).
- NSA, DSA and MoBA reduce compute and decode bandwidth but not stored capacity; V4's CSA/HCA also reduce capacity by compressing the sequence axis, and report ∼10% of V3.2's cache at 1M tokens.
- DSA's byte count is $c_{\text{idx}}/c_{\text{main}} + k/L$ of dense; its indexer scan becomes the bottleneck at very long context, hence the move to compress first.
- Selection error is controlled by the miss mass (Proposition 7.1); mean-pooled routing is biased against sharp single-token matches (Proposition 7.2).
- Evidence for native sparsity is concentrated in a few labs' reports; independent replication at comparable scale and on long-generation workloads is limited.

Chapter 8

Merging, Low-Rank Compression, Model Conversion and the Quantization Interface

Eviction discards tokens. This chapter surveys methods that keep information about all tokens but represent it more compactly: by *merging* similar tokens or layers, by projecting keys and values into a low-dimensional subspace, and by *converting* an existing model into one whose architecture natively uses such a subspace. We then discuss, briefly and without duplicating the quantization paper of this series, how quantization interacts with each of these and with eviction.

8.1 KV merging and token merging

8.1.1 Motivation and a correctness principle

If two cached tokens have similar keys, they will receive similar attention weights from any query, and their values can be averaged without much error. Token merging was popularised in vision transformers [87]; for language-model caches the same idea appears in several forms. To merge correctly one must keep the attention mass:

Proposition 8.1 (Size-corrected merging)**.** *Suppose tokens i, j have logits s_i, s_j against a query and are replaced by a single token with key $\bar{\mathbf{k}}$ and value $\bar{\mathbf{v}}$ standing for a multiplicity $m = 2$. If $\mathbf{k}_i = \mathbf{k}_j = \bar{\mathbf{k}}$ and $\mathbf{v}_i = \mathbf{v}_j = \bar{\mathbf{v}}$, the original attention output is reproduced exactly provided the merged token's logit is $\bar{s} + \ln m$ (equivalently, its softmax weight is multiplied by m).*

Proof. Before merging, the two tokens contribute $e^{\bar{s}}\bar{\mathbf{v}} + e^{\bar{s}}\bar{\mathbf{v}} = 2e^{\bar{s}}\bar{\mathbf{v}}$ to the numerator and $2e^{\bar{s}}$ to the denominator. After merging with logit $\bar{s} + \ln 2$, they contribute $e^{\bar{s}+\ln 2}\bar{\mathbf{v}} = 2e^{\bar{s}}\bar{\mathbf{v}}$ and $2e^{\bar{s}}$. □

This is the *proportional attention* correction of token merging [87]. Without it, merging halves the combined mass and distorts the softmax, an error that grows with the number of merges. If keys and values differ slightly the merge is approximate, and the error for a different query $\mathbf{q}'$ is controlled by the logit shift $\left|\mathbf{q}'^{\top}(\mathbf{k}_i - \bar{\mathbf{k}})\right| / \sqrt{d_h}$ (Proposition 11.2 in Chapter 11).

8.1.2 Methods

- **KVMerger** clusters consecutive key states by cosine similarity into merge sets, chooses a pivot per set and merges the others into it by Gaussian-kernel weights, applying merging

in addition to or instead of eviction on long-context tasks [85]. Its premise is that adjacent tokens' keys are often similar (the local-similarity observation).

- **Dynamic Memory Compression** (Chapter 6) learns the append-or-merge decision per head and trains the model to cope with the merged cache [60].
- **MiniCache** merges the caches of *adjacent layers* in the deeper half of the network, observing that their states are highly similar in direction: it interpolates directions spherically, retains magnitudes, and keeps unmergeable tokens separately [86]. This is the post-hoc counterpart of cross-layer attention (CLA) [9].
- **CSA/HCA compression** in DeepSeek-V4 is a *learned, differentiable* merge along the sequence axis (4× and 128×) with a learned positional bias, trained jointly with the model [7]; NSA's compression branch is similar [73]. These show that merging can be competitive when trained, even at high ratios, while post-hoc merging is typically used at 2–4×.

Merging blurs exactly what retrieval needs

Averaging two keys produces a key between them; a query that matches *one* of them sharply now matches the average only weakly, and retrieval of the specific item degrades (cf. the mean-versus-log-sum-exp gap of Proposition 7.2). Merging is safest for *redundant* content (repetitive reasoning traces, boilerplate) and worst for distinct facts, identifiers and numbers.

8.2 Low-rank key–value compression

8.2.1 Why the optimal projection is activation-aware

Section 2.5 showed that the best rank-r approximation of a key matrix is its truncated SVD. But what matters is error in *attention logits*, which depend on the queries. Moreover, one wants a *fixed projection per head* computed from calibration data, so that new tokens are compressed online. Consider the key projection $\mathbf{W}^K \in \mathbb{R}^{d \times d_h}$ applied to hidden states $\mathbf{h} \in \mathbb{R}^d$ with second-moment matrix $\Sigma = \mathbb{E}[\mathbf{h}\mathbf{h}^\top]$. We seek a rank-$r$ matrix $\hat{\mathbf{W}}$ minimising the expected squared error $\mathbb{E}\left\|\mathbf{h}^\top(\mathbf{W}^K - \hat{\mathbf{W}})\right\|^2$.

Theorem 8.1 (Activation-aware low-rank approximation)**.** *Let $\Sigma = \mathbb{E}[\mathbf{h}\mathbf{h}^\top] \succ 0$ with a symmetric square root $\Sigma^{1/2}$. Then*

$$\min_{\operatorname{rank}\hat{\mathbf{W}} \le r} \mathbb{E}\left\|\mathbf{h}^\top(\mathbf{W}^K - \hat{\mathbf{W}})\right\|^2 = \sum_{j>r} \sigma_j^2(\Sigma^{1/2}\mathbf{W}^K),$$

attained by $\hat{\mathbf{W}} = \Sigma^{-1/2}\,[\Sigma^{1/2}\mathbf{W}^K]_r$, where $[\cdot]_r$ is the truncated SVD.

Proof. $\mathbb{E}\left\|\mathbf{h}^\top\mathbf{D}\right\|^2 = \operatorname{tr}(\mathbf{D}^\top\Sigma\mathbf{D}) = \left\|\Sigma^{1/2}\mathbf{D}\right\|_F^2$ with $\mathbf{D} = \mathbf{W}^K - \hat{\mathbf{W}}$. Substituting $\mathbf{Z} = \Sigma^{1/2}\hat{\mathbf{W}}$ (a bijection on rank-$\le r$ matrices because $\Sigma^{1/2}$ is invertible) turns the problem into

$$\min_{\operatorname{rank}\mathbf{Z} \le r} \left\|\Sigma^{1/2}\mathbf{W}^K - \mathbf{Z}\right\|_F^2,$$

which Theorem 2.1 solves by truncated SVD. □

The theorem justifies calibration-based low-rank methods (the same idea as in whitened SVD for weight compression): the reconstruction error is measured in the metric induced by the activations that actually occur. Writing $[\Sigma^{1/2}\mathbf{W}^K]_r = \mathbf{U}_r\mathbf{S}_r\mathbf{V}_r^\top$, the compressed projection factorises as $\hat{\mathbf{W}} = \mathbf{AB}$ with $\mathbf{A} = \Sigma^{-1/2}\mathbf{U}_r\mathbf{S}_r \in \mathbb{R}^{d\times r}$ and $\mathbf{B} = \mathbf{V}_r^\top \in \mathbb{R}^{r\times d_h}$. The cache then stores the r-dimensional latent $\mathbf{z} = \mathbf{h}^\top\mathbf{A}$ per token and reconstructs $\hat{\mathbf{k}} = \mathbf{zB}$ when needed; the cache shrinks by r/d_h.

Proposition 8.2 (Joint compression dominates separate compression)**.** *Let* $\mathbf{W}^{KV} = [\mathbf{W}^K\ \mathbf{W}^V] \in \mathbb{R}^{d\times 2d_h}$. *The best rank-$r$ approximation of* $\Sigma^{1/2}\mathbf{W}^{KV}$ *has error no larger than the error obtained by compressing* $\Sigma^{1/2}\mathbf{W}^K$ *to rank* r_K *and* $\Sigma^{1/2}\mathbf{W}^V$ *to rank* r_V *separately with* $r_K + r_V = r$.

Proof. Separate compression yields an approximation $[\hat{\mathbf{W}}^K\ \hat{\mathbf{W}}^V]$ of the concatenation whose rank is at most $r_K + r_V = r$. The joint solution minimises the error over *all* matrices of rank at most r (Theorem 2.1), hence is at least as good. □

Proposition 8.2 is the statistical justification for MLA's *joint* latent for keys and values: sharing one latent lets the model allocate rank where it is needed (keys versus values, head versus head) rather than fixing the split.

8.2.2 Palu

Palu [80] applies low-rank projection to the cache of pretrained models and addresses two difficulties of the naive approach: reconstruction cost and rank allocation. (i) Reconstructing $\hat{\mathbf{k}} = \mathbf{zB}$ for *every* cached token at every step would cost $\mathcal{O}(nrd_h)$ FLOPs per head; Palu's *group-head* decomposition shares a latent across a group of heads, balancing reconstruction cost against accuracy, and fuses reconstruction into the attention kernel. (ii) It allocates ranks across matrices using a Fisher-information-based sensitivity measure. (iii) It notes that the low-rank latent is amenable to quantization, and for keys that are rotated by RoPE the reconstruction is performed before rotation, which costs an extra computation compared with MLA's decoupled design. The authors report that Palu halves the cache while keeping accuracy strong, with up to 1.89× speedup on the RoPE-based attention module, and, combined with quantization, up to 2.91× on that module with comparable or better accuracy than quantization alone [80]; these are not comparable to figures for methods that need no calibration.

Algorithm 8 makes the connection to MLA explicit: with the absorption trick, a post-hoc low-rank KV compression *is* an MLA-like layer, except that RoPE prevents full absorption unless positional dimensions are separated out. This is precisely the gap that conversion methods fill.

8.2.3 Eigen-decompositions in selection

Low-rank structure is also used for *scoring* rather than storage. ShadowKV stores low-rank keys on the GPU and uses them to select landmarks and tokens while values stay off-device [67]. SparQ's use of the top-r query channels to estimate scores [62] is a coordinate-subset analogue of projecting onto a dominant subspace. Both use the same observation as Theorem 8.1: a few directions dominate the logits.

Algorithm 8 Activation-aware low-rank KV compression (Palu-style, single layer; simplified).

Require: key/value projections $\mathbf{W}^K, \mathbf{W}^V$ for a head group; calibration hidden states $\{\mathbf{h}\}$; target rank r

$\Sigma \leftarrow \frac{1}{N}\sum \mathbf{h}\mathbf{h}^\top + \lambda \mathbf{I}$ ▷ regularised second moment

$\mathbf{W}^{KV} \leftarrow [\mathbf{W}^K\ \mathbf{W}^V]$; compute SVD of $\Sigma^{1/2}\mathbf{W}^{KV} = \mathbf{U}\mathbf{S}\mathbf{V}^\top$

$\mathbf{A} \leftarrow \Sigma^{-1/2}\mathbf{U}_r\mathbf{S}_r^{1/2}$; $\mathbf{B} \leftarrow \mathbf{S}_r^{1/2}\mathbf{V}_r^\top$ ▷ $\mathbf{B} = [\mathbf{B}^K\ \mathbf{B}^V]$

Inference, per new token: $\mathbf{z}_t \leftarrow \mathbf{h}_t^\top \mathbf{A}$; append $\mathbf{z}_t$ to the cache ▷ r numbers

Attention: absorb $\mathbf{B}^K$ into the query: $\tilde{\mathbf{q}} \leftarrow \mathbf{B}^K\mathbf{q}$ so that $\mathbf{q}^\top\hat{\mathbf{k}}_s = \tilde{\mathbf{q}}^\top\mathbf{z}_s^\top$ (valid without RoPE)

output $\mathbf{o} \leftarrow \big(\sum_s p_s\mathbf{z}_s\big)\mathbf{B}^V$ ▷ value up-projection after the weighted sum

With RoPE: reconstruct $\hat{\mathbf{k}}_s = \mathbf{z}_s\mathbf{B}^K$, rotate by position s, then score ▷ extra cost; avoided by decoupled RoPE

8.3 Converting existing models to latent attention

Pretraining MLA from scratch is expensive; many open models use GQA. Two 2025 papers show that one can *convert*.

MHA2MLA. Ji et al. convert MHA and GQA models into MLA by (i) partially retaining RoPE: choosing a subset of rotary dimensions to keep (by a contribution criterion) and removing rotation from the rest, so that the unrotated part is compatible with absorption; (ii) jointly compressing the unrotated key part and the value part with a low-rank SVD; and (iii) fine-tuning on a small fraction of the original pretraining data. They report that very little data (a fraction of one percent of the original) suffices to recover performance and that the cache of Llama2-7B can be cut by 92.19% with a 0.5% drop in LongBench performance, recovering performance with roughly 0.3–0.6% of the training data [81], and that the conversion composes with quantization.

TransMLA. Meng et al. build on the observation of Proposition 4.2, that GQA is a special case of MLA with the same cache size, to convert GQA models to MLA, then use the extra expressive power (distinct per-head up-projections) during fine-tuning; they report gains over the original GQA models on benchmarks after fine-tuning on a modest number of tokens, and argue that the converted model gains the hardware benefits of MLA's high-intensity decoding [82].

Caveats. Both papers evaluate on a limited set of models and benchmarks and with fine-tuning budgets that, while small relative to pretraining, are not negligible for large models; neither establishes that conversion matches a natively trained MLA of the same size. The conversion changes the *function* of the model, so any alignment, safety behaviour or calibration of the original must be re-verified (Chapter 14).

8.4 Linearisation as conversion

At the far end of the spectrum, attention layers can be *replaced* by recurrent layers through distillation: Mamba-in-the-Llama distils a transformer into a hybrid with Mamba layers [89];

Algorithm 9 Generic GQA/MHA → MLA conversion recipe (synthesised from [81, 82]; details differ by paper).

Require: pretrained attention layer with $\mathbf{W}^Q, \mathbf{W}^K, \mathbf{W}^V, \mathbf{W}^O$; calibration data; target latent width d_c; RoPE budget d^R

1: select d^R rotary dimensions to retain per head (or share across heads) ▷ e.g., those contributing most to logits
2: split keys into rotary part $\mathbf{W}^{K,R}$ and non-rotary part $\mathbf{W}^{K,N}$; remove rotation from the latter
3: $\mathbf{M} \leftarrow [\mathbf{W}^{K,N}\ \mathbf{W}^V]$; compute (activation-aware) rank-d_c factorisation $\mathbf{M} \approx \mathbf{D}\,[\mathbf{U}^K\ \mathbf{U}^V]$ ▷ Theorem 8.1
4: set $\mathbf{W}^{DKV} \leftarrow \mathbf{D}$, $\mathbf{W}^{UK} \leftarrow \mathbf{U}^K$, $\mathbf{W}^{UV} \leftarrow \mathbf{U}^V$; keep $\mathbf{W}^{K,R}$ as the decoupled rotary key
5: fine-tune (distillation from the original model and/or language-modelling loss) on a small token budget
6: at inference use the absorbed form (Algorithm 1)

SUPRA uptrains transformers to RNN-like linear attention [90]; LoLCATs linearises with low-rank adaptation and attention transfer [88]; Bick et al. distil quadratic into subquadratic models in stages [211]. These use the same pattern as the MLA conversions (initialise from the teacher's projections, then train briefly) and share the same caveat: they are weakest on recall.

8.5 Interface with quantization

KV quantization has its own paper in this series; we restrict ourselves to the interactions that matter for the other families.

8.5.1 What the main methods do

KIVI quantizes keys *per channel* and values *per token* to 2 bits, because keys have a few outlier channels and values do not [92]. KVQuant quantizes keys before rotary embedding, uses non-uniform codebooks and isolates outliers, targeting very long contexts [91]. QJL applies a Johnson–Lindenstrauss transform followed by a 1-bit quantization of the sketch with an unbiased inner-product estimator and no per-block scale overhead [93]. TurboQuant applies a random rotation and an optimal scalar quantizer per coordinate, adds a 1-bit QJL on the residual to make inner products unbiased, proves distortion within a small constant factor (about 2.7) of information-theoretic lower bounds, and reports quality neutrality for KV caches at 3.5 bits per channel with marginal degradation at 2.5 bits [94]. All numbers are the authors' claims from their own setups.

8.5.2 Effective bits and the real saving

Group-wise quantization stores a scale and zero-point (say 16 bits each) per group of g elements, so the effective bits per element is

$$b_{\text{eff}} = b_q + \frac{32}{g}. \tag{8.1}$$

At $b_q = 2$ and $g = 32$, $b_{\text{eff}} = 3$, a 5.3× reduction relative to bf16 rather than 8×. Methods that avoid per-group metadata (QJL, TurboQuant's rotation-based design) save this overhead [93, 94]. Applying 4-bit group-32 quantization ($b_{\text{eff}} = 5$) to the Llama-3-8B 128K cache of 16 GiB gives $5/16 \times 16 = 5$ GiB, and with 4× eviction on top roughly 1.25 GiB (derived), a 12.8× total.

Proposition 8.3 (Errors from eviction and quantization add)**.** *Let* $\mathbf{o}$ *be the exact attention output,* $\mathbf{o}_{\mathcal{S}}$ *the output after evicting to set* $\mathcal{S}$ *with dropped mass* ϵ*, and* $\hat{\mathbf{o}}_{\mathcal{S}}$ *the output after additionally quantizing the retained keys and values so that every logit changes by at most* η *and every retained value by at most* δ *in norm. Then*

$$\|\hat{\mathbf{o}}_{\mathcal{S}} - \mathbf{o}\| \le 2V\epsilon + \delta + V\big(e^{2\eta} - 1\big),$$

where $V = \max_s \|\mathbf{v}_s\|$.

Proof. Decompose $\hat{\mathbf{o}}_{\mathcal{S}} - \mathbf{o} = (\hat{\mathbf{o}}_{\mathcal{S}} - \mathbf{o}_{\mathcal{S}}) + (\mathbf{o}_{\mathcal{S}} - \mathbf{o})$. The second term is bounded by $2V\epsilon$ (Proposition 11.1). For the first, the weights over $\mathcal{S}$ are perturbed by factors in $[e^{-2\eta}, e^{2\eta}]$ after renormalisation (Proposition 11.2), contributing at most $V \,\mathrm{TV} \cdot 2 \le V(e^{2\eta} - 1)$, and the value perturbation contributes at most δ since the weights sum to one. □

The additive form says *budgets can be traded* but also that stacking does not self-correct; errors from both stages accumulate and the worst of them dominates. In practice errors are not worst-case aligned, and published combinations (Scissorhands with 4-bit; DuoAttention with quantization; Palu with quantization [48, 68, 80]) work well on their benchmarks, but a combined method must be evaluated as a unit.

8.5.3 Rotations unify low-rank and quantization

Orthogonal rotations leave attention logits unchanged when applied to both queries and keys, $\mathbf{q}^\top \mathbf{k} = (\mathbf{R}\mathbf{q})^\top (\mathbf{R}\mathbf{k})$. A rotation can therefore be inserted for free to make the cache easier to compress. A *random* rotation spreads outlier channels evenly (the TurboQuant/QJL route); a *PCA* rotation concentrates variance in a few leading channels, so the trailing ones can be dropped (low-rank) or quantized with fewer bits (variable bit allocation, the reverse water-filling solution of Section 11.5). Truncation after PCA *is* the low-rank method; quantization after PCA with variance-proportional bit allocation is the rate–distortion optimal transform code for Gaussian sources. This suggests a design principle for combined compressors: apply a data-aware rotation, then allocate bits across rotated channels by their variance, with zero bits meaning "dropped".

8.5.4 Interaction with selection and eviction

Eviction and selection scores computed on *quantized* keys can rank tokens differently from scores on full-precision keys; if the quantization noise has standard deviation comparable to the logit gaps of Proposition 2.4, top-k membership changes near the boundary. Hence, scoring on full-precision metadata (as Quest's min/max do) or scoring before quantization is common practice. In the opposite direction, eviction *lowers* the sensitivity to quantization noise in a favourable way: removing low-mass tokens removes the contributions that would have summed many small noise terms.

Table 8.1: Merging, low-rank and conversion methods. Claims are the authors' reports; none is a like-for-like comparison.

Method	Axis compressed	Needs training?	Reported effect
KVMerger [85]	tokens (merge)	no	merging beats evicting at same budget on long-context tasks (per authors)
DMC [60]	tokens (learned merge)	continued pre-training	several-fold compression with small loss
MiniCache [86]	layers (merge)	no	compresses deep-layer caches by interpolation
Palu [80]	channels (low rank)	calibration only	∼50% reduction; up to 2.91× attention speedup with quantization
MHA2MLA [81]	channels (to MLA)	light fine-tuning	92.19% cache reduction on Llama2-7B, 0.5% LongBench drop
TransMLA [82]	channels (GQA→MLA)	fine-tuning	equal cache, improved expressivity
ShadowKV [67]	keys low-rank on GPU, values offloaded	no	enables larger batch and longer context with offload

8.6 Summary table

Chapter summary

- Merging must preserve attention mass (size-corrected logits); it is safest on redundant text and worst on distinct facts.
- The optimal low-rank projection is activation-aware (Theorem 8.1) and joint key–value compression dominates separate (Proposition 8.2), the statistical case for MLA.
- Post-hoc low-rank compression is an MLA-like layer whose obstacle is RoPE; conversion methods (MHA2MLA, TransMLA) solve it by partial RoPE and short fine-tuning.
- Quantization composes with all of these multiplicatively in size but additively in error (Proposition 8.3); effective bits include group metadata; rotations connect quantization and low-rank compression.

Chapter 9

Memory Management and Serving Systems

The previous chapters change what is stored. This chapter changes *where, when and how it is stored and computed* without altering the model's function: exact attention kernels, paged memory, prefix reuse, chunked prefill, tiered offload, disaggregated serving, and speculative decoding as an adjacent technique. These are the techniques that make every other method deployable, and, as Chapter 16 argues, those with the most stable return.

9.1 Exact attention kernels: the FlashAttention lineage

9.1.1 Online softmax

Standard attention materialises the $n \times n$ score matrix in HBM. FlashAttention [110] avoids this by tiling and by a streaming softmax that processes keys in blocks while maintaining a running maximum m, normaliser ℓ and unnormalised output $\mathbf{u}$ [116, 117].

Proposition 9.1 (Online softmax)**.** *Let logits $s_1, \dots, s_n$ be split into consecutive blocks. Define for the first j blocks $m^{(j)} = \max$ of all logits so far, $\ell^{(j)} = \sum_s e^{s-m^{(j)}}$ and $\mathbf{u}^{(j)} = \sum_s e^{s-m^{(j)}}\mathbf{v}_s$ over the same entries. On receiving block $j+1$ with maximum $\tilde{m}$, local sum $\tilde{\ell} = \sum e^{s-\tilde{m}}$ and local weighted sum $\tilde{\mathbf{u}}$, set $m' = \max(m^{(j)}, \tilde{m})$ and*

$$\ell^{(j+1)} = e^{m^{(j)}-m'}\ell^{(j)} + e^{\tilde{m}-m'}\tilde{\ell}, \qquad \mathbf{u}^{(j+1)} = e^{m^{(j)}-m'}\mathbf{u}^{(j)} + e^{\tilde{m}-m'}\tilde{\mathbf{u}}. \tag{9.1}$$

After the last block, $\mathbf{u}/\ell = \sum_s p_s \mathbf{v}_s$ is the exact softmax-weighted output.

Proof. By definition $\ell^{(j)}e^{m^{(j)}} = \sum_{s\in\text{first } j} e^s$ and likewise for $\mathbf{u}$ and the new block. Adding the two partial sums of the true (unshifted) exponentials and re-expressing them with the new shift m' gives (9.1). The ratio $\mathbf{u}/\ell$ cancels the common shift and equals $\sum_s e^s \mathbf{v}_s / \sum_s e^s$. □

The merge (9.1) is associative and commutative, which has two consequences: blocks can be processed in any order, and partial results computed on *different processors* can be combined afterwards. This is the foundation of the parallel decoding kernels below and of several cache-sharing methods (prefix/suffix decomposition) later in this chapter.

9.1.2 IO complexity and the lineage

FlashAttention reduces HBM traffic from $\Theta(nd + n^2)$ to $\Theta(n^2d^2/M)$ for on-chip memory of size M (the authors' IO-complexity analysis [110]), by never writing the score matrix. FlashAttention-2 improves parallelism and work partitioning across thread blocks and

Algorithm 10 Split-KV decode attention (one query, one KV head, P parallel splits).

Require: query $\mathbf{q}$; keys/values partitioned into splits $\{(\mathbf{K}^{(i)}, \mathbf{V}^{(i)})\}_{i=1}^{P}$

for $i = 1, \dots, P$ **in parallel do**

  $\mathbf{s} \leftarrow \mathbf{K}^{(i)}\mathbf{q}/\sqrt{d_h}$; $m_i \leftarrow \max \mathbf{s}$; $\ell_i \leftarrow \sum e^{\mathbf{s}-m_i}$; $\mathbf{u}_i \leftarrow \mathbf{V}^{(i)\top} e^{\mathbf{s}-m_i}$

end for

$m \leftarrow \max_i m_i$

$\ell \leftarrow \sum_i e^{m_i - m}\ell_i$; $\quad \mathbf{u} \leftarrow \sum_i e^{m_i - m}\mathbf{u}_i$ ▷ reduction, Equation (9.1)

return $\mathbf{u}/\ell$

warps [111]; FlashAttention-3 exploits Hopper-generation asynchrony (overlapping data movement and matrix operations), warp specialisation, and low-precision (FP8) matrix multiplication [112]. These kernels are exact, and are the baseline against which sparse methods are compared. Sequence-parallel variants distribute the keys and values of a very long prompt across devices: Ring Attention circulates K/V blocks around a ring of devices, overlapping communication with the blockwise computation [118], and DeepSpeed-Ulysses redistributes sequence and head dimensions with all-to-all communication [119].

9.1.3 Decoding: split-KV

During decode a query has length one, so the standard parallelisation over query blocks yields a single block and leaves most of the GPU idle at small batch and long context. *Flash-Decoding* [113] parallelises *over the keys*: the cache is split into chunks processed by different thread blocks, each producing a partial $(m, \ell, \mathbf{u})$, and a final reduction combines them with (9.1). FlashDecoding++ further reduces synchronisation in the softmax merge [114], and FlashInfer provides a customisable engine with block-sparse and composable formats for paged, ragged and shared-prefix caches [115].

9.2 Paged memory management

9.2.1 The fragmentation problem

Before paged allocation, serving systems reserved a contiguous region of the *maximum* possible sequence length for each request, because the output length is unknown. This wastes memory three ways: internal fragmentation (reserved but unused), external fragmentation (gaps between regions) and the inability to share memory between sequences that share a prefix. PagedAttention [95] borrows the operating-system solution: the cache of each sequence is divided into fixed-size *blocks* (for example 16 tokens), which can live anywhere in memory; a per-sequence *block table* maps logical to physical blocks, and the attention kernel gathers through it. The vLLM system built on it reports near-zero waste (under 4% in the authors' analysis) and 2–4× higher throughput than the prior state-of-the-art systems at the same latency in its evaluation [95, 217].

Proposition 9.2 (Internal fragmentation of paged allocation)**.** *If sequence lengths are independent and the last block of a sequence is uniformly filled, the expected wasted tokens per sequence is* $(P_b - 1)/2$ *for block size* P_b*, i.e., a waste fraction of* $(P_b - 1)/(2\bar{n})$ *for mean length* $\bar{n}$*. A*

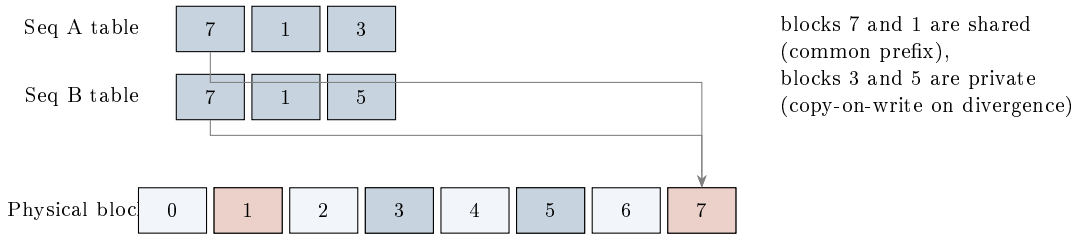


Figure 9.1: Block-table indirection in paged attention (schematic). Two sequences share the physical blocks that hold a common prefix; private suffix blocks are allocated on demand.

contiguous allocation reserving $n_{\max}$ tokens wastes $n_{\max} - \bar{n}$ on average.

Proof. The unused slots in the last block are uniformly distributed on $\{0, \dots, P_b - 1\}$, with mean $(P_b - 1)/2$. For contiguous allocation, the reserved $n_{\max}$ minus the used $\bar{n}$ is unused. □

For $P_b = 16$ and $\bar{n} = 1000$ the waste is 0.75%; for contiguous allocation at $n_{\max} = 8192$ it is 88% (derived, illustrative lengths). This is why paging, rather than any compression, produced the largest single throughput improvement in production serving between 2022 and 2024.

Block size trade-off. Smaller blocks reduce fragmentation and improve sharing granularity but increase table size and the number of non-contiguous reads per kernel invocation; larger blocks improve memory-access efficiency. vAttention [96] argues that the indirection costs kernel complexity and instead uses virtual-memory mappings to keep the cache virtually contiguous while physically paged, allowing unmodified kernels; both approaches are in use.

Interaction with compression. Paging absorbs variable cache lengths per head and per layer (eviction, DuoAttention's two head classes) without padding if the page table is kept per head; engineering effort is required to support non-uniform page tables, and not all engines support it. Hybrid models need *two* allocators, for growing KV pages and for fixed-size recurrent state.

9.3 Prefix caching and shared computation

9.3.1 Radix trees

Because a token's key and value depend on all earlier tokens (causality), the cache of a prefix can be reused by any later request that starts with the *identical token sequence*. RadixAttention (in SGLang) stores cached blocks in a radix tree keyed by token sequences, and uses an LRU policy on leaves together with a cache-aware scheduler that orders requests to maximise sharing [97]. vLLM's automatic prefix caching achieves a similar effect by hashing each block's content together with the hash of its predecessor block (a hash chain), so that a block is identified by the whole prefix up to it. Typical wins: multi-turn chat (each turn re-sends the conversation), shared system prompts, few-shot templates, retrieval-augmented prompts with common documents, and agent loops that append to a growing transcript.

Proposition 9.3 (Prefix reuse validity)**.** *Reusing the cached keys and values of a prefix $x_{1:m}$ for a request $x_{1:m}x'$ is exact (equals recomputation) if and only if (i) the model is causal, (ii) positions of the reused tokens are identical, (iii) no cache-modifying transformation (eviction, merging, quantization with data-dependent scales, sparse selection state) was applied that depended on tokens after m, and (iv) the numerical kernels are deterministic for the reuse path versus the recompute path (otherwise equality holds only up to floating-point reordering).*

Proof sketch. By causality, layer-ℓ keys and values at positions $\leq m$ are functions of $x_{1:m}$ only; positions enter through RoPE and must match. If cache entries had been modified using information from later tokens (as in query-aware eviction with the question appended), they are no longer functions of $x_{1:m}$ alone. Floating-point non-associativity makes recomputation and reuse differ in the last bits unless kernels coincide, which is why bit-exact reproducibility is not guaranteed across cache hits and misses. □

Condition (iii) is the formal statement of the incompatibility flagged in Table 3.2: *query-aware per-request compression is incompatible with prefix reuse*, and conversely eviction applied to a shared prefix *before* the question is known (query-agnostic, as KVzip) is compatible.

9.3.2 Non-prefix reuse

Retrieval-augmented prompts concatenate documents whose caches would be reusable if only the prefix condition were relaxed. Prompt Cache precomputes caches for reusable prompt modules with explicitly assigned position ranges and splices them together [98]; CacheBlend reuses precomputed caches of retrieved chunks and *selectively recomputes* the keys and values of a small fraction of tokens whose cache deviates most from what full prefill would produce, to restore cross-chunk attention, reporting 2.2–3.3× lower time-to-first-token and 2.8–5× higher throughput than full recomputation in its evaluation [99]; RAGCache organises the knowledge cache across GPU and host memory for RAG [108]. All are *approximations*: cross-chunk attention is missing or partially repaired. They trade fidelity for latency and therefore need the same evaluation scrutiny as eviction.

9.3.3 Shared-prefix decoding

Hydragen and ChunkAttention observe that when many sequences share a long prefix, attention over the prefix can be computed *once for the whole batch* as a matrix–matrix product, with the suffix handled separately and the two combined with the merge (9.1) [100, 101].

Proposition 9.4 (Shared-prefix intensity)**.** *For B sequences sharing a prefix of n_p tokens and a KV head serving g query heads each, batched prefix attention reads the prefix K/V once and performs $4Bgn_pd_h$ FLOPs, so its arithmetic intensity is $2Bg/b$ FLOP/B, B times that of independent decode attention (Proposition 2.2).*

Proof. The prefix K/V is $2n_pd_hb$ bytes read once; the Bg queries each perform $4n_pd_h$ FLOPs. The quotient is $2Bg/b$. □

With $B = 64$ and $g = 8$ the intensity is 512 FLOP/B, above the ridge point of Section 2.3: shared-prefix attention is *compute-bound*, not memory-bound. The authors of Hydragen report

end-to-end throughput improvements of up to 32× for CodeLlama-13b against competitive baselines, with throughput falling by less than 15% as the shared prefix grows from 1K to 16K tokens, whereas the baselines' throughput dropped by over 90% [100]. This is the one situation where long context *improves* hardware utilisation.

9.4 Chunked prefill and scheduling

Prefill and decode interfere: a long prefill occupying the GPU stalls the decode steps of other requests, inflating inter-token latency for them. Sarathi-Serve splits prompts into chunks of a fixed token budget and forms each iteration's batch from all ongoing decodes plus one prefill chunk ("stall-free batching"), tuning the chunk size to meet latency targets [102]. Chunking is mathematically exact: chunk j's queries attend to the cache of chunks $< j$ plus causally to their own chunk, which yields the same result as monolithic prefill. It interacts with other techniques as follows.

- With paged caches it is natural: each chunk appends pages.
- With hybrid recurrent layers it works with a chunkwise state carry.
- With prompt-dependent eviction (SnapKV) the observation window comes from the *last* chunk, so scoring must be deferred until all chunks have arrived, increasing peak memory unless the policy is made chunk-incremental.
- With sparse prefill (MInference-style) the per-head patterns must be computed per chunk against the accumulated cache.

Chunk size c trades time-to-first-token (smaller chunks delay the prompt's completion because it shares iterations with decodes) against decode-latency stability.

9.5 Tiered storage and offloading

9.5.1 What the links allow

Accelerator memory bandwidth is of order 3–8 TB/s for current datacenter parts, while the host link (PCIe Gen5 x16) offers on the order of 64 GB/s per direction and a good NVMe drive several to ∼14 GB/s (round, illustrative figures). Reading cache from host memory is therefore roughly 50× slower than from HBM, and from NVMe a few hundred times slower. Two usage patterns result.

Pattern 1: per-step reads from the lower tier. If at each decode step only a fraction ρ of the cache of size S is read from the host, the transfer time $\rho S/\beta_{\text{link}}$ must be below the step time $S_{\text{HBM-resident}}/\beta$ to stay hidden: $\rho \lesssim \beta_{\text{link}}/\beta \approx 2\%$. This is why per-step offload systems require very sparse selection (Quest-style top-k, InfiniGen's prefetch, RetrievalAttention, MagicPIG, ShadowKV) [61, 64–67] and why FlexGen, designed for throughput with huge batches rather than latency, overlaps transfers with computation across batches [109].

Pattern 2: prefix reuse across requests. Cache computed for a prefix is stored in a lower tier and *loaded back* when a matching request arrives. Here the comparison is load time versus recompute time.

Proposition 9.5 (Load-versus-recompute break-even)**.** *Loading n tokens of cache from a tier with bandwidth β_{tier} takes $nc_{\text{tok}}/\beta_{\text{tier}}$ with $c_{\text{tok}} = b\sum_{\ell} c_{\ell}$ bytes per token, while recomputing them takes $\approx 2P_{\text{act}}\, n/(\mu P)$ with P_{act} the active parameters, P peak FLOP/s and μ the achieved utilisation (attention's quadratic term ignored). Loading is faster iff $c_{\text{tok}}/\beta_{\text{tier}} < 2P_{\text{act}}/(\mu P)$, a condition independent of n.*

Example 9.1. Take a Llama-3-8B-class model ($c_{\text{tok}} = 128$ KiB, $P_{\text{act}} = 8 \times 10^9$) on illustrative hardware with $P = 10^{15}$, $\mu = 0.5$. Recompute costs $2 \cdot 8 \times 10^9/(5 \times 10^{14}) = 32\,\mu$s per token; loading over a 50 GB/s link costs $131{,}072/(5\times10^{10}) = 2.6\,\mu$s, a 12× advantage for loading; from a 5 GB/s NVMe drive it costs $26\,\mu$s, close to break-even. For a 70B model ($c_{\text{tok}} = 320$ KiB) recompute costs $280\,\mu$s per token while a 50 GB/s load costs $6.6\,\mu$s (42×), and an NVMe load $66\,\mu$s (4×). The model with more parameters per cache byte benefits more from stored prefixes. All values are derived and illustrative.

Proposition 9.5 has an important corollary: architectural reductions of c_{tok} (MLA, hybrids) make *loading cheaper*, and thus make large external caches practical: for an MLA model with 68.6 KiB/token the 70B-class calculation above improves by another ∼5×. LMCache implements a tiered cache layer (GPU, CPU memory, disk, remote) that integrates with serving engines and shares caches across engine instances [107].

9.6 Disaggregated prefill and decode

Prefill is compute-bound and favours high-FLOP configurations; decode is memory-bound and favours high bandwidth and large memory. Running both on the same GPUs makes them interfere (Section above) and forces one parallelism configuration for two different bottlenecks. *Disaggregation* assigns them to separate pools of machines: a prefill node computes the cache and *ships it* to a decode node. Splitwise characterises the two phases and proposes phase splitting to improve cost and power [104]; DistServe optimises *goodput* (requests meeting both time-to-first-token and per-token latency targets) by choosing separate parallelism and placement for each phase [103]; Mooncake, the serving platform of Kimi, goes further with a KV-cache-centric design in which the cache is a global resource pooled in CPU and storage tiers, with scheduling driven by cache locality [105].

9.6.1 The network bill

Disaggregation turns the cache into network traffic. For a sequence of n tokens the transfer is nc_{tok} bytes. Table 9.1 computes the transfer times for sample models over a 400 Gb/s (50 GB/s) link. Layer-by-layer pipelining overlaps transfer of early layers with computation of later ones, hiding a large part of the latency, but the volume matters for network capacity and cost.

CacheGen compresses the cache for network transfer using an encoding that exploits the token-wise locality of K/V values, delta coding against anchor tokens, quantization with layer-dependent precision, and arithmetic coding, reporting several-fold smaller transfers and corresponding reductions in time-to-first-token when contexts are loaded over bandwidth-limited links [106]. This is lossy compression of the *stored/transmitted* cache, applied outside the attention path, so it composes with all in-model reductions.

Table 9.1: Time to transfer the cache of a 128K-token sequence over an illustrative 50 GB/s link (*derived* from Table 2.1; no compression, no overlap with computation; recurrent states of hybrids are small and omitted). Illustrates how architecture reduces the cost of disaggregation.

Model	Cache at 128K (GiB)	Transfer time (s)
Llama-3.1-405B (GQA)	63.0	1.35
Llama-3-70B (GQA)	40.0	0.86
Qwen3-8B (GQA)	18.0	0.39
DeepSeek-V3 (MLA)	8.6	0.18
gpt-oss-120b (local–global)	4.5	0.10
Qwen3-Next (hybrid)	3.0	0.06
Kimi-Linear (hybrid)	1.0	0.02

9.7 Parallelism and data layout for long contexts

Tensor parallelism shards heads; with 8 KV heads on 8 GPUs each holds one, but beyond that degree KV heads must be replicated (Section 2.2). MLA's single latent cannot be sharded by head, so MLA deployments typically use data-parallel attention (each attention replica holds complete caches for its own requests) with expert parallelism for the feed-forward layers; the fraction of memory holding cache is then not shared across devices. Context parallelism splits *one* long sequence across devices for prefill (ring or all-to-all schemes [118, 119]) and, for decode, partitions the cache by sequence blocks with the split-KV merge of Algorithm 10 across devices.

9.8 Batching, mixture-of-experts and the cache-bound regime

Equation (2.8) assumed W constant. For mixture-of-experts models each token activates k_e of E experts; the experts *read* per step grow with batch as $\approx E\big(1-(1-k_e/E)^B\big)$ in expectation under uniform routing, so a decode step at small batch reads only a few experts and at large batch nearly all. The weight term $W(B)$ is therefore increasing and concave in B; the amortisation of weight reads by batching is weaker than in dense models at small B, and cache-bound behaviour sets in at *smaller* batch for MoE models. This is one reason MoE frontier models benefit disproportionately from cache reductions.

9.9 Speculative decoding and the cache (adjacent method)

Speculative decoding proposes γ tokens with a cheap draft (a small model, extra heads, or feature-level drafting) and verifies all of them with one forward pass of the target model [124, 125]; with acceptance probability α per token the expected number of tokens produced per target pass is $(1-\alpha^{\gamma+1})/(1-\alpha)$ [124]. For $\alpha = 0.8$ and $\gamma = 4$ this is 3.36 (derived). Variants include Medusa heads (no separate draft model, tree attention over multiple candidates), EAGLE (feature-level autoregressive drafting), and tree-based verification in SpecInfer [126–128].

Its interaction with the cache is favourable and subtle.

Proposition 9.6 (Speculative decoding amortises cache reads)**.** *In the cache-bound regime, verifying $\gamma+1$ positions in one pass reads the target model's cache once for all of them. If the*

expected accepted length per pass is $\bar{\tau} = (1 - \alpha^{\gamma+1})/(1 - \alpha)$*, the cache bytes read per generated token are reduced by a factor* $\bar{\tau}$ *(before draft overhead), because arithmetic intensity increases by up to* $\gamma + 1$ *(the* $\gamma + 1$ *queries share the cache read, as in Proposition 9.4).*

Speculative decoding and cache compression are therefore *complementary* in the cache-bound regime: one reduces bytes per step, the other steps per token. Frictions: (i) the draft model has its own cache (small for small drafts; EAGLE-style heads reuse target features); (ii) draft tokens must be appended to the cache and *rolled back* on rejection, which is a pointer truncation for KV caches but requires saved states for recurrent layers (Chapter 5); (iii) with per-query sparse selection (DSA, Quest) the $\gamma + 1$ queries select different tokens, so the shared-read benefit is reduced; (iv) with eviction, tentative tokens must be excluded from scoring until accepted.

9.10 Kernel support for compressed and sparse caches

A recurring practical barrier is that an algorithm without a fast kernel does not deliver its theoretical savings. Table 9.2 lists, qualitatively and to the best of the author's knowledge as of the survey date, the kernel status of the method families. Engine support changes rapidly; the table is a guide to *kinds* of kernel risk rather than a statement about particular releases.

Chapter summary

- The online-softmax merge (Proposition 9.1) underlies FlashAttention, Flash-Decoding, shared-prefix decoding and multi-device attention; it is exact.
- Paging removes fragmentation (near-zero waste) and enables sharing; prefix caching is exact only if cache entries depend on the prefix alone (Proposition 9.3).
- Shared-prefix batching turns decode attention compute-bound (intensity $\propto B$); speculative decoding cuts cache reads per token by the expected accepted length.
- Offloaded prefix caches pay whenever load time beats recompute time; big models and small per-token caches win.
- Disaggregation converts cache size into network traffic, so cache-reducing architecture lowers its cost directly.
- Kernel availability, not the algorithm, often determines realised gains.

Table 9.2: Kernel-level considerations by method family (author's qualitative assessment; verify against engine documentation).

Family	Kernel requirement	Risk
GQA / MQA	standard grouped decode	low; universally supported
MLA	absorbed MQA-form decode, expanded prefill, decoupled RoPE	medium; specialised kernels needed (FlashMLA-type); head-dim 576/512 unusual
Local–global	windowed prefill and ring-buffer decode	low–medium; two cache layouts per model
Hybrid linear / SSM	chunkwise scan prefill, recurrent decode, state checkpoints	medium–high; new allocators; prefix-cache semantics differ
Token eviction (H_2O etc.)	needs attention probabilities or scoring side kernels; ragged caches	high; incompatible with fused kernels without extra pass
Page / block selection (Quest)	metadata kernels, gather of selected pages	medium
Token-level sparse (DSA)	indexer kernel (low precision), sparse gather decode	high; hardware-specific low-precision paths
Block sparse (NSA, MoBA)	block-wise gather, group-shared selection	medium–high; training kernels too
KV quantization	dequantize-in-kernel	medium; format proliferation
Low rank (Palu)	fused reconstruct + attention	medium–high; RoPE re-rotation
Paging / prefix cache	block-table gather	low; mature

Chapter 10

Prompt and Context Compression, Memory, Agents and Retrieval

The previous chapters compress the *cache* that the model builds from its input. This chapter compresses the *input itself*, or replaces the growing transcript with some other form of memory. The distinction matters operationally: input-side compression reduces prefill compute *and* the cache simultaneously, since fewer tokens are ever processed, but it changes the text the model sees and is therefore a change to the task.

10.1 A taxonomy of input-side compression

Form of compressed context	Representative methods	Characteristics
Hard prompt (text)	LLMLingua, LongLLMLingua, LLMLingua-2 [129–131]	still text; works with any (even API) model; interpretable; compression 2–20×
Soft prompt (vectors)	prompt/prefix tuning, gist tokens, AutoCompressor, ICAE [132–136]	learned vectors or cache entries; need model access and a trained compressor; higher ratios
Recurrent / segment memory	Transformer-XL, Compressive Transformer, RMT, Memorizing Transformers, Infini-attention, Titans [137–142]	memory carried across segments; architecture-level
Summary memory	recursive summarisation, MemGPT-style paging [146, 147]	text summaries; lossy; agent-friendly
Agent context policies	context folding, typed eviction [144, 145]	learned or rule-based management of the agent's own transcript

10.2 Hard prompt compression: the LLMLingua family

10.2.1 Information-theoretic basis

A language model assigns each token a *self-information* (surprisal) $I(x_t) = -\log p(x_t \mid x_{<t})$. Tokens with low surprisal are predictable from context, and deleting them removes little information, in the same sense that a Huffman code spends few bits on likely symbols. LLMLingua exploits this with a small language model: it estimates the surprisal of each token (or segment) under the small model and drops the most predictable ones, subject to a token budget [129]. Its

Algorithm 11 Budgeted surprisal-based prompt compression (simplified LLMLingua-style).

Require: prompt segments $\{(\text{type}_j, \mathbf{x}^{(j)})\}$ (instruction, demonstrations, context, question); small LM q; target token budget T

assign per-type retention rates ρ_j (e.g., instruction and question ≈ 1; demonstrations lower) so that $\sum_j \rho_j |\mathbf{x}^{(j)}| \approx T$
for each segment j with $\rho_j < 1$ **do**
    compute $I(x_t) = -\log q(x_t \mid x_{<t})$ for tokens in the segment (in chunks, conditioning on already compressed preceding text)
    keep tokens with $I(x_t) \geq \tau_j$, where τ_j is the $(1-\rho_j)$-quantile of surprisal in the segment
end for
return concatenation of kept tokens in original order

components are a budget controller that allocates different compression rates to instructions, demonstrations and questions (instructions and questions are kept more intact than demonstrations), a coarse-to-fine procedure (sentence/demonstration level first, then token level), and an alignment step to reduce the distribution mismatch between the small compressor model and the target model. The authors report compression ratios up to about 20× with little performance loss on reasoning and in-context-learning benchmarks [129].

10.2.2 Question-aware and learned variants

LongLLMLingua adapts the approach to long contexts by scoring document relevance *conditional on the question*, reordering documents to counter the position sensitivity documented in [156], and adjusting compression rates per document, with the aim of both shortening and improving the prompt [130]. LLMLingua-2 replaces the unsupervised surprisal rule with a token-level *classifier* (a small bidirectional encoder) trained on data distilled from a stronger model that was asked to compress texts without losing information; since a bidirectional encoder sees full context, it avoids the causal-surprisal bias, and it is faster and task-agnostic [131].

10.2.3 Limits

(i) Deleting tokens can drop the one fact needed: numbers, names and negations have low surprisal in context but are exactly what questions ask about. (ii) The compressed text is *new text*; it breaks prefix reuse across requests unless the compression is deterministic and applied to the shared document in isolation. (iii) Surprisal under a small model is not importance under the target model; mismatch grows with target-model capability. (iv) For long outputs (reasoning, agents), input compression does nothing about the cache growth from the model's own generations.

10.3 Soft compression: gist tokens, AutoCompressor, ICAE

Prompt tuning and prefix tuning showed that a small number of *continuous* vectors prepended to the input can condition a frozen model as strongly as a task prompt [135, 136]. Soft context compression asks for the converse: given a *specific* long text, produce a few vectors that stand in for it.

Gist tokens. Mu et al. insert k special *gist tokens* after a prompt and modify the attention mask so that tokens after the gist tokens cannot attend to the prompt but only to the gist tokens; the model is fine-tuned so that the gist tokens' activations carry what later tokens need. At inference only the gist tokens' keys and values are retained as a compressed cache. They report prompt compression of up to 26× with minimal loss on instruction-following prompts [132].

AutoCompressor. A model is fine-tuned to produce *summary vectors* at the end of each segment; summary vectors of earlier segments are prepended as soft prompts when processing the next segment, so context accumulates recursively through a fixed number of vectors per segment, enabling contexts of tens of thousands of tokens (the paper reports up to about 30K) at a fraction of the cache [133].

ICAE. The in-context autoencoder uses a LoRA-adapted copy of the LLM as an *encoder* that produces a small number of memory slots from a long input, and a frozen decoder (the original LLM) that reads those slots in place of the text; it is pretrained with autoencoding and language-modelling objectives and then fine-tuned on prompt-response data, with roughly 4× compression reported [134].

Soft compression is learned KV compression on the sequence axis

The key–value entries of the gist or memory tokens *are* the compressed cache. Soft prompt compression is therefore the same idea as the learned sequence-axis compression of NSA's compression branch and DeepSeek-V4's CSA/HCA layers [7, 73], but trained as a separate stage on a frozen or lightly adapted model rather than inside the architecture. The architectural versions compress *every layer's* cache by a fixed ratio at every position; the soft-prompt versions compress a designated span by a trained ratio.

Capacity versus learnability. Is the compression ratio limited by information capacity? A span of n tokens over a vocabulary of V contains at most $n \log_2 V$ bits, e.g., a 4000-token span at $\log_2 V \approx 17$ is $\approx 6.8 \times 10^4$ bits. A memory of k vectors of width d at 16 bits stores up to $16kd$ bits, e.g., 64 slots at $d = 4096$ is $\approx 4.2 \times 10^6$ bits, sixty times more. So capacity does not bind at the ratios reported (4–30×); what binds is *learnability* and the trained decoder's ability to extract specific details. This explains why such methods do well on gist-like tasks (summary-like understanding, following instructions) and worse on tasks requiring verbatim recall of specifics, and it suggests that failures can be reduced by training objectives that force detail retention.

10.4 Recurrent and external memory

Transformer-XL caches the hidden states of the previous segment (without gradient) and lets the current segment attend to them with relative positions, extending the effective context to roughly segment length times depth [138]; the **Compressive Transformer** compresses evicted memories into a secondary, smaller memory by learned convolutions [139]. The **Recurrent Memory Transformer** (RMT) adds memory tokens whose output states at the end of one segment become the inputs of the next, creating an explicit recurrence trained by back-propagation through segments [137]. **Memorizing Transformers** store past key–value pairs

in an external memory searched by approximate k-nearest-neighbour lookup at one layer [141]. **Infini-attention** adds to local attention a per-head compressive memory updated and read in the manner of linear attention (an associative matrix), with a learned gate mixing local and memory readouts [140]. **Titans** treat a neural memory as a module whose parameters are updated at test time by a surprise-driven gradient rule with momentum and forgetting [142]; test-time-training layers are a related line [143].

Structurally these are the recurrent-state methods of Chapter 5: a fixed-size memory updated by writes. They inherit the counting bound of Section 11.4 and the interference of Proposition 2.7. Evidence: reasoning-in-a-haystack evaluations such as BABILong show that models with recurrent memory can handle very long streams but with limits in the number of facts that can be tracked [163].

10.5 Summary-based memory and agent context management

10.5.1 Summaries

The simplest memory for a long interaction is a running natural-language summary. Recursive summarisation (summarise chunks, then summarise the summaries) was used for book-length tasks [147], and agentic "compaction" (replace the oldest part of the transcript by a model-written summary when the context nears its limit) is now standard in coding and research agents. Its failure modes are well known: information loss that the summariser cannot foresee, hallucinated details, and drift when summaries are summarised repeatedly (summary of summary). Because the summariser does not know future queries, summarisation is query-agnostic compression and is bound by the lower bound of Section 11.4.

10.5.2 OS-style paging: MemGPT

MemGPT treats the context window like RAM and external storage like disk, and gives the model functions to move information between them; the model itself decides what to write to or recall from external memory [146]. This makes compression a *tool-use* problem: reliability depends on the model's judgement about future relevance.

10.5.3 Context folding

Context folding [144] lets an agent *branch* into a sub-trajectory to perform a subtask and then *fold* it: on return, the intermediate steps are removed and replaced by a concise result, so the working context contains only the outcomes of finished sub-trajectories plus the current branch. The folding behaviour is trained with reinforcement learning (FoldGRPO, with process rewards that shape the branching and summarising). On deep-research and software-engineering tasks, the authors report that the folding agent matches or exceeds standard ReAct baselines while using an active context about ten times smaller, and clearly outperforms summarisation-based context management [144].

Algorithm 12 Context-folding agent loop (schematic, after [144]).

Require: task x; policy π with actions `think`, `tool`, `branch`(d), `return`(r); context stack $\mathcal{C} = [x]$

while not finished **do**
  $a \sim \pi(\cdot \mid \mathcal{C})$
  if $a =$ `branch`(d) **then**
    push a new frame with description d onto $\mathcal{C}$ ▷ sub-trajectory begins
  else if $a =$ `return`(r) **then**
    pop the frame, *discarding its intermediate steps*; append only r to the parent frame ▷ fold
  else
    execute a; append the action and observation to the current frame
  end if
end while
return final answer

Table 10.1: Context-management strategies for long-running agents. "Cache effect" refers to the model cache; qualitative assessment by the author, evidence from the cited papers only.

Strategy	Who decides what to drop	Cache effect	Main risk
Truncation (FIFO)	recency	bounded	drops early instructions and goals
Summarisation	LLM (query-agnostic)	bounded; prefix changes	lossy, hallucinated, compounding
Context folding [144]	learned policy	bounded active context	policy errors on when to fold; training cost
Typed eviction [145]	deterministic rules	bounded	type labelling errors; dependency tracking
External memory tools [146]	model via function calls	bounded	model forgets to save or recall
Cache-level eviction (H_2O etc.)	attention statistics	bounded	silent failures; breaks prefix sharing

10.5.4 Typed, deterministic eviction

A complementary approach avoids model-written summaries entirely: Context Window Lifecycle labels the agent's trajectory as typed, dependency-linked episodes and uses a deterministic, non-LLM policy to evict episodes in priority order when a budget is exceeded, keeping user turns and still-needed exploratory context while dropping action episodes whose effects are already stored in the environment [145]. The authors report one agent session completing 89 sequential tasks over about 80 million tokens with no measurable accuracy loss relative to isolated sessions [145]. This is a single preprint with a specific task suite; the structure it exploits (that the *environment* is a durable external memory, so many transcript steps are redundant with state) is real and general for tool-using agents.

10.6 Retrieval versus long context

Retrieval-augmented generation retrieves relevant passages at query time [191]; long-context models read everything. Empirical findings are nuanced.

- Xu et al. found that retrieval augmentation can match or exceed fine-tuned long-context models of much larger window on several tasks, and that retrieval benefits long-context models too (a 70B model with retrieval and a 32K window performed best in their comparison) [193].
- Li et al. found that, when resources suffice, long-context models generally outperform RAG on average, but RAG is much cheaper; most queries produce the same prediction under both, which motivates *Self-Route*: let the model decide, from retrieved chunks, whether it can answer, and fall back to full context otherwise, reducing cost while keeping comparable performance [192].
- Yu et al. report that retrieving more passages improves recall at first but then *hurts* answer quality because of distractors (an inverted-U), and that preserving the original order of retrieved chunks helps [194].
- Models use long contexts unevenly: performance is best when relevant information is at the beginning or end and degrades in the middle [156].

10.6.1 A simple cost model

Let a corpus have n tokens, a query have q tokens, and a RAG prompt include k retrieved tokens plus retrieval cost r (in token-equivalents). Let a be the cost per prefilled token and d the extra cost per query of reading a cache of n tokens during decoding (cache-bound, per Equation (2.8)). If the corpus cache is prefilled once and *reused* across Q queries (prefix caching), the long-context total is $an + Q(aq + d)$; RAG costs $Q(a(k + q) + r)$. Long context is cheaper when

$$Q > Q^\star = \frac{an}{ak + r - d}. \tag{10.1}$$

Example 10.1 (Illustrative)**.** Take $n = 200{,}000$, $k = 4{,}000$, $r = 1{,}000$ token-equivalents and per-query cache-read overhead d equivalent to 20,000 token-prefills (illustrative values reflecting that reading 200K tokens of cache per output token adds up over a few hundred output tokens). Then $ak + r - d$ in units of a is $4000 + 1000 - 20000 < 0$: long context never breaks even at these values, because decode-time cache reads dominate. If cache reduction brings d down to the equivalent of 2,000 prefills, $Q^\star = 200000/(4000 + 1000 - 2000) \approx 67$ queries. Changing d by an order of magnitude flips the conclusion, which is the point: cache-reducing architecture and prefix caching are what make long-context-instead-of-retrieval economically viable (derived from the illustrative model).

Quality, of course, does not follow the cost model: RAG can fail when retrieval misses a needed passage or when the question is global ("summarise the themes of all documents"), while long context can fail from distraction and from the compression methods of the earlier chapters. A pragmatic stance is the hybrid: retrieve to narrow, keep as much context as is affordable, and place the most important material at the ends.

10.7 Positional encoding as it bears on the cache

We treat positional encoding only where it touches cache design.

RoPE basics. Rotary embeddings [148] rotate each consecutive pair of coordinates of $\mathbf{q}$ and $\mathbf{k}$ at position m by angle $m\theta_i$ with $\theta_i = \text{base}^{-2i/d_h}$, $i = 0, \ldots, d_h/2 - 1$, so that $\mathbf{q}_m^\top \mathbf{k}_{m'}$ depends on $m - m'$. The wavelength of pair i is $\lambda_i = 2\pi/\theta_i$: high-frequency pairs (short wavelength) encode local offsets; low-frequency pairs encode long-range position.

Extension. A model trained on length L_{train} degrades beyond it because rotation angles at larger positions were not seen. Positional interpolation divides positions by a factor $s = L_{\text{new}}/L_{\text{train}}$ so all angles stay in the trained range, then fine-tunes briefly [150]. This compresses the resolution of high-frequency pairs, which the NTK-aware and YaRN refinements avoid by interpolating *only the low-frequency dimensions*, those with $\lambda_i > L_{\text{train}}$ (where a full period was never seen), leaving high-frequency dimensions untouched and adding an attention-temperature correction [151]. LongRoPE searches for non-uniform per-dimension rescaling factors and reports extension to over two million tokens [152]; Self-Extend remaps relative positions beyond a neighbour window into grouped positions at inference without tuning [153]; ALiBi replaces rotations by distance-proportional biases [149]. Training recipes for long context (data mix, staged extension) matter as much as the encoding [154, 155].

Cache consequences.

1. **Pre- versus post-rotation storage.** Storing keys after rotation is cheapest at decode but fixes positions; eviction schemes that reassign positions (StreamingLLM, Algorithm 3), low-rank compression (which should operate on pre-rotation keys) and MLA's decoupled design all prefer pre-rotation or separate rotary dimensions.
2. **Dynamic scaling breaks caching.** Some implementations change the scaling factor as the sequence grows ("dynamic NTK"). A key cached at position m under one scale is inconsistent with the scale used later; implementations recompute, which defeats the cache and prefix reuse. Serving systems should use a *static* scaling.
3. **Claimed versus effective context.** Extension methods let a model *accept* a long input; whether it *uses* it is a separate question measured by RULER-style benchmarks (Chapter 12) [157].
4. **No-position layers.** Several hybrid designs omit positional encoding in some (global) layers; those layers' caches need no rotation, simplifying compression of exactly the layers that matter at long range.

Chapter summary

- Input-side compression reduces prefill and cache together but changes the task; hard methods (LLMLingua) are model-agnostic, soft methods (gist, ICAE, AutoCompressor) are learned KV compression on the sequence axis.
- Soft compression is limited by learnability and detail extraction, not by information capacity at current ratios.
- Agent context management is moving from LLM summaries to learned folding and typed deterministic eviction; evidence is promising but from few preprints.

- RAG versus long context is a cost–quality trade whose break-even depends strongly on cache reuse and decode-time cache reads (Equation (10.1)).
- Positional encoding matters to cache design through pre/post-rotation storage, position reassignment after eviction, and the need for static scaling in serving.

Chapter 11

Theory: Error Bounds, Sketching, Lower Bounds and Information

This chapter collects the mathematical statements that explain *why* the methods of earlier chapters work, *when* they must fail, and *how well* any method could possibly do. Results are labelled as proved here (with proof), standard (cited with a proof sketch) or cited without proof. All are idealised models: they quantify what is possible under stated assumptions and should not be read as predictions of benchmark scores.

11.1 Error bounds for eviction and perturbation

11.1.1 Eviction

Proposition 11.1 (Eviction error). *Let* $\mathbf{o} = \sum_{s=1}^{n} p_s \mathbf{v}_s$ *with* $\sum p_s = 1$, $\|\mathbf{v}_s\| \le V$. *Let* $\mathcal{E}$ *be an evicted set of total mass* $\epsilon = \sum_{s\in\mathcal{E}} p_s < 1$ *and* $\mathcal{S}$ *its complement. The output after eviction and renormalisation is* $\mathbf{o}_{\mathcal{S}} = \frac{1}{1-\epsilon}\sum_{s\in\mathcal{S}} p_s \mathbf{v}_s$ *and*

$$\mathbf{o} - \mathbf{o}_{\mathcal{S}} = \epsilon\,(\mathbf{o}_{\mathcal{E}} - \mathbf{o}_{\mathcal{S}}), \qquad \|\mathbf{o} - \mathbf{o}_{\mathcal{S}}\| \le \epsilon\,\|\mathbf{o}_{\mathcal{E}} - \mathbf{o}_{\mathcal{S}}\| \le 2V\epsilon, \tag{11.1}$$

where $\mathbf{o}_{\mathcal{E}} = \frac{1}{\epsilon}\sum_{s\in\mathcal{E}} p_s \mathbf{v}_s$ *is the mean value of the evicted tokens. Hence the error is linear in the evicted mass, and vanishes if the evicted values equal the retained mean.*

Proof. Write $\mathbf{o} = (1-\epsilon)\mathbf{o}_{\mathcal{S}} + \epsilon\mathbf{o}_{\mathcal{E}}$, the decomposition of the weighted average into the retained and evicted parts. Then $\mathbf{o} - \mathbf{o}_{\mathcal{S}} = \epsilon(\mathbf{o}_{\mathcal{E}} - \mathbf{o}_{\mathcal{S}})$. The norms of $\mathbf{o}_{\mathcal{E}}$ and $\mathbf{o}_{\mathcal{S}}$ are at most V (convex combinations of vectors of norm at most V), giving the final bound. □

Corollary 11.1 (Budget–error trade-off under power-law attention). *Under Proposition 2.5 with exponent* $\alpha > 1$, *retaining the top-*k *tokens leaves* $\epsilon_k \approx \frac{k^{1-\alpha}}{(\alpha-1)\zeta(\alpha)}$ *and thus* $\|\mathbf{o} - \mathbf{o}_{\mathcal{S}}\| \lesssim 2V\epsilon_k$, *which decays polynomially in* k *and is independent of* n.

Example 11.1 (Numbers). For $\alpha = 1.5$ ($\zeta(1.5) \approx 2.612$): $\epsilon_k \approx k^{-0.5}/(0.5\cdot 2.612) = 0.766\,k^{-0.5}$. Keeping $k = 100$ gives $\epsilon \approx 0.077$; $k = 1000$ gives 0.024; $k = 10^4$ gives 0.0077 (derived from the model). For $\alpha = 1.1$ the tail is much heavier: $\epsilon_k \approx k^{-0.1}/(0.1 \cdot 10.58) = 0.945\,k^{-0.1}$, so even $k = 10^4$ leaves $\epsilon \approx 0.38$. Whether real heads look like $\alpha = 1.5$ or $\alpha = 1.1$ is the central empirical question for each model, head and workload.

11.1.2 Logit perturbation

Proposition 11.2 (Logit perturbation). *Let* $p = \mathrm{softmax}(s)$ *and* $p' = \mathrm{softmax}(s')$ *with* $\max_i |s'_i - s_i| \le \eta$. *Then* $p'_i/p_i \in [e^{-2\eta}, e^{2\eta}]$ *for all* i, *the total variation distance satisfies* $\mathrm{TV}(p, p') \le \frac{1}{2}(e^{2\eta} - 1)$, *and for values bounded by* V *the outputs differ by at most* $V(e^{2\eta} - 1)$.

Proof. $p'_i/p_i = e^{\delta_i}\, Z/Z'$ with $|\delta_i| \le \eta$ and $Z'/Z = \sum_j p_j e^{\delta_j} \in [e^{-\eta}, e^{\eta}]$, hence the ratio lies in $[e^{-2\eta}, e^{2\eta}]$. Then $|p'_i - p_i| = p_i\,|p'_i/p_i - 1| \le p_i(e^{2\eta} - 1)$ and $\mathrm{TV} = \frac{1}{2}\sum_i |p'_i - p_i| \le \frac{1}{2}(e^{2\eta} - 1)$. Finally $\|\sum_i (p'_i - p_i)\mathbf{v}_i\| \le V \sum_i |p'_i - p_i| = 2V\,\mathrm{TV}$. □

This covers any approximation that perturbs logits by a bounded amount: quantization of keys, low-rank projection of keys, merging of nearby keys, and sketching. For a low-rank projection with key residual $\mathbf{r}_s = \mathbf{k}_s - \hat{\mathbf{k}}_s$, $\eta = \max_s \left|\mathbf{q}^\top \mathbf{r}_s\right| / \sqrt{d_h} \le \|\mathbf{q}\| \max_s \|\mathbf{r}_s\| / \sqrt{d_h}$.

11.1.3 Propagation through layers and time

The bounds above hold per head and per step. Two propagation effects make the end-to-end behaviour different. *Across layers*, a perturbation in layer ℓ's attention output is passed through the remaining layers; in a worst-case Lipschitz analysis the final perturbation is $\le \sum_\ell \delta_\ell \prod_{j>\ell} K_j$ where K_j bounds the Lipschitz constant of layer j, which is loose but shows that earlier-layer errors can be amplified. *Across time*, a model that evicts during decoding writes later tokens' keys and values using *perturbed* hidden states, so the error is stored in the cache and persists.

Proposition 11.3 (A toy model of error accumulation)**.** *Let $e_t \ge 0$ measure the deviation of the compressed model's cache state from the dense model's after t generated tokens, and suppose $e_{t+1} \le \rho\, e_t + \delta$, where δ is the new error injected per step by eviction and $\rho \ge 0$ is the gain with which earlier deviation propagates into later states. Then $e_T \le \delta \frac{1-\rho^T}{1-\rho}$ for $\rho \ne 1$ and $e_T \le \delta T$ for $\rho = 1$. The deviation stays bounded as $T \to \infty$ iff $\rho < 1$, in which case $e_\infty \le \delta/(1-\rho)$.*

Proof. Unroll the recursion: $e_T \le \rho^T e_0 + \delta \sum_{j=0}^{T-1} \rho^j$ with $e_0 = 0$, and sum the geometric series. □

The model is too crude to estimate ρ for real systems, but it explains qualitatively why long-generation tasks (chain-of-thought with thousands of steps) are harder for decoding-time compression than short-answer tasks: injected error compounds unless the dynamics are contractive. Whether language-model reasoning is contractive in this sense is unknown and likely depends on whether later steps *re-derive* information (contractive) or *depend on a single stored intermediate* (non-contractive). Chapter 12 reports empirical evidence consistent with the second case for some tasks.

11.2 Heavy hitters and streaming algorithms

11.2.1 Space-Saving applied to accumulated attention

Treat each decode step as distributing one unit of weight over cached tokens, and let $f_s = \sum_t p_s^{(t)}$ be token s's accumulated attention. A cache of k slots that must keep tokens with high f_s is a *weighted heavy-hitters* problem, studied in streaming algorithms [204–206].

Proposition 11.4 (Space-Saving guarantee for weighted updates)**.** *Maintain k counters keyed by token. On an update that adds weight w to token s: if s has a counter, add w; otherwise, if a free slot exists, create a counter with value w; otherwise replace the token with the* minimum *counter by s and set s's counter to $c_{\min} + w$. Let N be the total weight inserted. Then (i) every counter overestimates its token's true accumulated weight by at most $c_{\min}$ at the time of*

insertion; (ii) $c_{\min} \le N/k$ *at all times; (iii) any token with true accumulated weight greater than* N/k *is present at the end.*

Proof. (ii) The sum of the counters equals the total inserted weight N (each update adds exactly w to the sum, whether the token existed, a slot was free, or a replacement occurred), and the minimum of k non-negative numbers is at most their mean, N/k. (i) A new token's counter is initialised at $c_{\min} + w \ge w$ so counters never underestimate, and overestimate by at most the $c_{\min}$ of the replacement event. (iii) A token's counter is at least its true weight, hence greater than $N/k \ge c_{\min}$, so it is never the minimum and is never replaced. □

Accumulated-attention eviction in the style of H_2O is a Space-Saving-like heuristic: it keeps the tokens with the highest accumulated score, though it does not initialise new entries with $c_{\min}$ (a new token has had little time to accumulate and is protected by the recency window instead). The proposition shows two things. (1) If importance *were* stationary and equal to the accumulated mass, a cache of k slots retains every token whose mass exceeds a $1/k$ fraction of the total, regardless of the sequence length, which is the streaming analogue of Proposition 2.5. (2) It guarantees nothing about *conditional* importance (a token irrelevant until one specific future query), which is the case that matters for retrieval.

11.2.2 Why sketching does not trivially solve the cache

Johnson–Lindenstrauss projections [202] preserve inner products among n points to relative accuracy ε in dimension $m = \mathcal{O}(\varepsilon^{-2} \log n)$, independently of d. One could therefore hope to compress each head's keys from $d_h = 128$ to a smaller m for any n. The constants are unforgiving at the dimensions of interest: with the standard bound $m \ge 4 \ln n / (\varepsilon^2/2 - \varepsilon^3/3)$ and $n = 10^5$, $\varepsilon = 0.1$ gives $m \approx 10^4 \gg 128$ (derived). Worst-case sketches do not compress 128-dimensional keys; success of low-rank methods relies on *data-dependent* structure (fast-decaying spectra, Figure 2.3), not on worst-case geometry. By contrast, one-bit sketches such as QJL [93] and rotation-based quantization [94] use JL-type transforms to *randomise* rather than to reduce dimension, and their guarantees concern the unbiasedness and variance of the inner-product estimator.

11.3 Hashing and sampling: Reformer, HyperAttention, MagicPIG

11.3.1 Locality-sensitive hashing for attention

Reformer [174] shares queries and keys and groups them by an angular LSH so that tokens with similar directions land in the same bucket; attention is computed within sorted, chunked buckets, giving $\mathcal{O}(n \log n)$ cost; multiple hash rounds reduce the chance that a high-attention pair is split. Locality-sensitive hashing was introduced for approximate nearest-neighbour search [203]; the basic collision probability for random-hyperplane hashing (SimHash) is classical:

Proposition 11.5 (SimHash collision). *For unit vectors* $\mathbf{a}, \mathbf{b}$ *with angle* θ*, and a random hyperplane through the origin with normal* $\mathbf{g} \sim \mathcal{N}(0, \mathbf{I})$*,* $\Pr[\operatorname{sign}(\mathbf{g}^\top \mathbf{a}) = \operatorname{sign}(\mathbf{g}^\top \mathbf{b})] = 1 - \theta/\pi$.

With K independent bits per table and L tables, the probability that $\mathbf{a}, \mathbf{b}$ collide in at least one table is $1 - \left(1 - (1 - \theta/\pi)^K\right)^L$.

Proof. The two signs differ iff the hyperplane separates the vectors; restricted to the plane spanned by $\mathbf{a}$ and $\mathbf{b}$ the normal's projection has a uniformly random direction, so separation has probability θ/π. Independence across bits and tables gives the second formula. □

A pair with small angle ($\theta = \pi/8$: per-bit collision 0.875) collides in a single $K = 8$-bit table with probability $0.875^8 \approx 0.34$, and in at least one of $L = 10$ tables with probability $1 - 0.66^{10} \approx 0.98$, whereas a pair at $\theta = \pi/2$ (per-bit 0.5) collides per table with $0.5^8 = 0.0039$ and across 10 tables with ≈ 0.038 (derived). This is the amplification that makes LSH useful and the reason hash-based attention needs several tables: the retrieval probability for a moderately similar but important pair can be low.

11.3.2 Sampling instead of top-k: a self-normalised importance estimator

MagicPIG [66] argues that attention need not be approximated by the top-k entries: it can be *estimated by sampling*. If key i is included in a sample with probability π_i (LSH collision probabilities provide π_i increasing in $\mathbf{q}^\top \mathbf{k}_i$), then the self-normalised estimator

$$\hat{\mathbf{o}} = \frac{\sum_{i \in \mathcal{M}} w_i \mathbf{v}_i / \pi_i}{\sum_{i \in \mathcal{M}} w_i / \pi_i}, \qquad w_i = e^{\mathbf{q}^\top \mathbf{k}_i / \sqrt{d_h}}, \tag{11.2}$$

where $\mathcal{M}$ is the sampled set, is a ratio of two unbiased estimators of the numerator $\sum_i w_i \mathbf{v}_i$ and denominator $\sum_i w_i$ (Horvitz–Thompson). The ratio is consistent and has bias $\mathcal{O}(1/|\mathcal{M}|)$. The variance is smallest when $\pi_i \propto p_i \|\mathbf{v}_i - \mathbf{o}\|$ (the quantity of Proposition 6.1), so LSH-based inclusion probabilities, which are monotone in the logit, approximate the optimal importance distribution. The advantage over top-k is that tokens outside the top-k still contribute in expectation, which matters when the attention mass is spread (Proposition 2.5 with α near 1, or Proposition 2.6 in the high-temperature phase).

11.3.3 HyperAttention

HyperAttention [175] combines (i) an LSH-based sort that places large entries of the attention matrix near the diagonal of a permuted matrix, where a block-diagonal computation captures them, with (ii) uniform sampling of the remaining columns to estimate the rest. Its guarantee is a near-linear running time under two fine-grained parameters of the attention matrix (a bound on the largest column norm after normalisation and on the ratio of row sums), *without* assuming bounded entries; the authors report speedups at long sequence lengths in the prefill/forward pass and note use in causal settings through a recursive scheme [175]. Like all worst-case-avoiding methods, its guarantee is conditional on the structure of the input matrix, consistent with the lower bounds below.

11.4 Lower bounds

11.4.1 Time: the fine-grained complexity of attention

Let $\mathbf{A} = \mathrm{diag}(\mathbf{D})^{-1}\exp(\mathbf{Q}\mathbf{K}^\top)$ be the attention matrix. Alman and Song study the problem of approximating $\mathbf{AV}$ up to additive error $1/\mathrm{poly}(n)$ with $d = \mathcal{O}(\log n)$ and entries of $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ bounded by B [176].

Theorem 11.1 (Alman–Song, cited)**.** *If $B = o(\sqrt{\log n})$ there is an algorithm running in time $n^{1+o(1)}$. If $B = \Omega(\sqrt{\log n})$, no algorithm runs in time $\mathcal{O}(n^{2-\delta})$ for any constant $\delta > 0$ unless the Strong Exponential Time Hypothesis (SETH) fails.*

A related result of Keles et al. shows that exact self-attention requires quadratic time under SETH [177]; the gradient computation has analogous thresholds [199]. The threshold $\sqrt{\log n}$ for the *entry* bound corresponds to logits of magnitude $B^2 \approx \log n$, the same scale as the logit gap $\Delta \approx \ln(n/k)$ of Proposition 2.4 and the freezing threshold $\sqrt{2\ln n}$ for logit standard deviation in Proposition 2.6. The coincidence is not accidental: the hard regime is exactly where attention must be able to single out one key against n distractors, so sparsity cannot be assumed away. *Worst-case subquadratic attention is impossible at high precision; every practical method exploits structure in the data*, which is why evaluation on adversarially structured inputs (multi-hop, aggregation, very flat attention) is where such methods are most likely to fail.

11.4.2 Space: query-agnostic compression and the INDEX problem

Suppose a model reads a sequence and must retain a summary (a cache, a recurrent state, a gist vector) from which *any one of many possible questions* can later be answered. The following is the cleanest statement of what this costs.

Proposition 11.6 (Memory lower bound for query-agnostic recall)**.** *Let $X = (X_1, \dots, X_n)$ be i.i.d. uniform bits, and let a (possibly randomised) procedure read X once and produce a state S of s bits. Suppose that for every $i \in [n]$ a decoder, given S and i, outputs X_i correctly with probability at least $2/3$. Then*

$$s \;\geq\; n\,\big(1 - H_2(1/3)\big) \;\approx\; 0.082\,n,$$

where H_2 is the binary entropy function.

Proof. By independence of the X_i, $I(X;S) \geq \sum_i I(X_i;S)$. (Indeed $I(X;S) = \sum_i I(X_i;S \mid X_{<i})$ and, since X_i is independent of $X_{<i}$, $I(X_i;S \mid X_{<i}) \geq I(X_i;S)$.) For each i, decoding X_i from S with error probability $\leq 1/3$ implies, by Fano's inequality, $H(X_i \mid S) \leq H_2(1/3)$, hence $I(X_i;S) = 1 - H(X_i \mid S) \geq 1 - H_2(1/3)$. Finally $s \geq H(S) \geq I(X;S) \geq n(1 - H_2(1/3))$. □

This is the one-way communication complexity of the INDEX problem, $\Omega(n)$, in an information-theoretic form. It applies to *any* fixed-size summary: a recurrent state of s bits, a gist memory, an evicted cache with s bits of storage. If $s = o(n)$ bits, some index is answered wrongly with high probability. For recurrent layers with state of $d_v d_k$ numbers of b bits, it says that the number of *arbitrary facts* recallable is at most $\mathcal{O}(d_v d_k b/\text{bits per fact})$ per head and layer, regardless of gating or the delta rule.

Proposition 11.7 (Copying requires state proportional to length)**.** *A deterministic machine with a state of s bits that reads an arbitrary string in Σ^n and must later output it exactly needs $s \geq n \log_2 |\Sigma|$.*

Proof. There are $|\Sigma|^n$ possible inputs, each of which must lead to a distinct output, hence a distinct state after reading (the output depends on the state only). A state of s bits takes at most 2^s values, so $2^s \geq |\Sigma|^n$. □

Jelassi et al. prove a related separation: a small transformer can copy exponentially long strings, whereas a fixed-state model cannot [181]; empirical recall limits of recurrent and linear-attention models appear in [182, 183, 185]. The caveat: a *real* sequence is highly compressible (language has far fewer than $\log_2 V$ bits per token), and a model is not required to retain arbitrary random strings.

Remark 11.1 (The cache is a time–memory trade-off, and its floor is the text's entropy). Propositions 11.6 and 11.7 bound the memory needed to answer arbitrary queries by the *information content of the input*, about $n \log_2 V \approx 17n$ bits for a 130K-vocabulary text, i.e., roughly two bytes per token. A KV cache of 128 KiB per token stores $\sim 6 \times 10^4$ times more bits than this floor. The raw token ids are themselves a perfect query-agnostic summary of the context: keep them, and *recompute* the cache on demand at the cost of a prefill. Every method in this paper therefore lies on a curve between two ends, "store the cache" (memory-heavy, compute-free) and "store the tokens" (memory-minimal, recompute-heavy), and the load-versus-recompute analysis of Proposition 9.5 locates the practical optimum. Compression of the cache can reduce memory far below 128 KiB per token without violating information bounds; what it cannot do is guarantee that the *model's own computation* can still extract the needed bit.

11.4.3 Attention is efficient in memory only with enough heads and dimension

Sanford, Hsu and Telgarsky prove representational separations between transformers and other architectures on tasks of retrieval-like type (such as sparse averaging), where attention uses an embedding dimension logarithmic in n while recurrent or feed-forward alternatives require substantially more [178]. The qualitative lesson matches Proposition 11.6: attention is a content-addressable memory whose capacity is the cache itself; any attempt to bound it must pay by reducing what can be retrieved.

11.5 Rate–distortion view of KV compression

Treat each key (or value) vector as a draw from a source and ask for the minimum number of bits needed to reproduce it to a given distortion. For a Gaussian source with independent components of variances $\lambda_1 \geq \cdots \geq \lambda_d$ and squared-error distortion, the rate–distortion function, in the framework of Shannon's source-coding theory [208] and with scalar quantizers in the tradition of Lloyd [210], is given by reverse water-filling:

Theorem 11.2 (Reverse water-filling, standard [207])**.** *For $X \sim \mathcal{N}(0, \mathrm{diag}(\lambda_1, \ldots, \lambda_d))$ and distortion budget D, $R(D) = \sum_{j=1}^{d} \frac{1}{2} \log_2 \frac{\lambda_j}{D_j}$ bits with $D_j = \min(\theta, \lambda_j)$ and θ chosen so that $\sum_j D_j = D$. Components with $\lambda_j \leq \theta$ receive zero bits (they are dropped).*

Proof sketch. For independent components the problem decouples as $\min \sum_j \frac{1}{2} \log \frac{\lambda_j}{D_j}$ subject to $\sum D_j = D$ and $0 \le D_j \le \lambda_j$, a convex problem whose Lagrangian condition equalises $D_j = \theta$ on active components and clips at λ_j. □

Reverse water-filling formalises the design principle of Chapter 8: in a PCA basis, bits (or rank) should go to high-variance directions and zero bits to low-variance ones, which is exactly low-rank truncation combined with quantization. For attention the relevant distortion is on logits, not on keys:

Proposition 11.8 (Query-weighted distortion)**.** *If* $\mathbb{E}[\mathbf{q}\mathbf{q}^\top] = \Sigma_q$ *and key error* $\mathbf{r} = \mathbf{k} - \hat{\mathbf{k}}$*, then the mean-square logit error is* $\mathbb{E}(\mathbf{q}^\top \mathbf{r})^2 = \mathbb{E}[\mathbf{r}^\top \Sigma_q \mathbf{r}]$ *for independent* $\mathbf{q}$ *and* $\mathbf{r}$*. Applying Theorem 11.2 to the transformed source* $\Sigma_q^{1/2}\mathbf{k}$ *gives the optimal rate allocation for this distortion; its relevant "variances" are the eigenvalues of* $\Sigma_q^{1/2}\Sigma_k\Sigma_q^{1/2}$*.*

Proof. $\mathbb{E}(\mathbf{q}^\top \mathbf{r})^2 = \mathbb{E}[\mathbf{r}^\top \mathbf{q}\mathbf{q}^\top \mathbf{r}] = \mathbb{E}[\mathbf{r}^\top \Sigma_q \mathbf{r}] = \mathbb{E}\left\|\Sigma_q^{1/2}\mathbf{r}\right\|^2$ under independence, which is squared-error distortion on $\Sigma_q^{1/2}\mathbf{k}$ and its reconstruction. □

Directions of key space that queries rarely probe (Σ_q small) can be dropped even if keys vary a lot there; this is the information-theoretic content of activation-aware low-rank methods (Theorem 8.1) and of the observation that query and key subspaces need not coincide.

Example 11.2 (Synthetic spectrum)**.** Take $d = 128$ and eigenvalues $\lambda_j = j^{-2}$ (total variance ≈ 1.637). With water level $\theta \approx r^{-2}$, about r components are active and the distortion is $D \approx 2/r$ and the rate $R \approx r \log_2 e = 1.44r$ bits per vector. For a relative distortion D/tot of 1%, 5% and 10% this gives $r \approx 122, 24, 12$ and rates of ≈ 176, 35, 18 bits per vector, against $128 \times 16 = 2048$ bits in bf16, i.e., compression factors of about 12×, 58× and 116× (derived from the idealised Gaussian model). These factors are *upper limits* for one fast-decaying spectrum within a single vector. Real keys have outlier channels and flatter tails, and logit errors are not mean-square errors of vectors. Practical quantizers (3.5 bits per channel in [94] corresponds to 448 bits) are far from this limit, and for isotropic worst-case sources there is no spectrum to exploit; TurboQuant proves it is within a factor of about 2.7 of the lower bound for such sources [94].

11.6 The information bottleneck and what to keep

The information bottleneck [209] frames context compression as finding a representation T of the context X that retains information about future quantities Y (the future queries and answers) at minimum rate: $\min_{p(t|x)} I(X;T) - \beta\, I(T;Y)$. Every method in this paper chooses T implicitly: eviction (a subset of tokens), low rank (a subspace), merging (cluster centroids), gist tokens (learned vectors), summaries (text). What differs is the assumed $p(y \mid x)$. Query-aware methods plug the actual question in as Y; query-agnostic methods must average over a distribution of questions, and Proposition 11.6 shows that for worst-case question distributions the rate cannot fall below the information content of the context. The practical consequence is that *the quality of a compressor is the quality of its prior over future questions*: KVzip's reconstruction prior [69], reasoning-trace priors [70], and agentic dependency graphs [145] are all priors, and each fails when the actual question is outside it.

Table 11.1: Theoretical statements of this chapter and their practical meaning.

Statement	Assumption	Implication
Eviction error $\leq 2V\epsilon$ (Prop. 11.1)	bounded values; renormalised softmax	drop mass, not tokens, is what matters; value-aware scores better
Logit perturbation (Prop. 11.2)	ℓ_∞ logit error η	quantization/low-rank/merging errors enter via $e^{2\eta} - 1$
Error accumulation (Prop. 11.3)	linear toy dynamics	long generation compounds unless contractive
Space-Saving (Prop. 11.4)	stationary accumulated importance	heavy tokens retained with k counters; no conditional guarantee
SimHash (Prop. 11.5)	random hyperplanes	LSH needs several tables to find moderate-similarity keys
Alman–Song (Thm. 11.1)	SETH, $d = \mathcal{O}(\log n)$	worst-case subquadratic attention impossible; structure required
INDEX bound (Prop. 11.6)	i.i.d. bits; all queries	query-agnostic summaries need $\Omega(n)$ bits; floor is entropy of input
Copy bound (Prop. 11.7)	exact output	fixed-state models cannot copy long strings
Reverse water-filling (Thm. 11.2)	Gaussian, MSE	PCA + bit allocation is optimal; low-rank is the zero-bit case

11.7 Summary of theoretical statements

Chapter summary

- Eviction error is exactly $\epsilon\,(\mathbf{o}_{\mathcal{E}} - \mathbf{o}_{\mathcal{S}})$, linear in the dropped mass; every approximation that perturbs logits enters through $e^{2\eta} - 1$.
- Heavy-hitter retention has streaming guarantees for stationary importance, none for conditional importance.
- Worst-case subquadratic attention and sublinear-memory query-agnostic recall are impossible (SETH; INDEX); practical methods work because real attention is structured and real text is compressible.
- The information floor of a context is its entropy (a couple of bytes per token), orders of magnitude below a KV cache, so compression is limited by what the model can *decode*, not by storage.
- A compressor is only as good as its prior over future questions.

Chapter 12

Evaluation Methodology

A compression method is only as trustworthy as the evaluation that supports it. Long-context evaluation is unusually treacherous: tasks are expensive, easy to saturate, and often solvable by shortcuts that compression methods happen to preserve. This chapter surveys the benchmarks, explains why needle-in-a-haystack tests are insufficient, describes what is known about failures specific to cache compression, and proposes a layered evaluation protocol.

12.1 What must be measured

A complete report on a cache-reduction method includes four families of measurements.

1. **Quality**, on tasks representative of the *deployment workload* and not only on public leaderboards, compared with the same model uncompressed ("FullCache").
2. **Memory**, as bytes per token *including all metadata* (scales, indexer keys, page tables, summaries), peak and steady-state, for named batch sizes and context lengths.
3. **Latency**: time to first token (TTFT, dominated by prefill), time per output token (TPOT or inter-token latency), at stated batch size, and tail percentiles, not only means.
4. **Throughput and cost**: tokens per second per accelerator at a stated latency target (*goodput* [103]), against a *tuned* baseline (a fused FlashAttention-class or FlashInfer-class kernel [112, 115]), not against a naive eager implementation.

Three definitional traps recur. First, "compression ratio" is sometimes the fraction of *tokens* retained, sometimes the fraction of *bytes*; these differ when quantization, merging or low-rank projection is involved. Second, budgets may be per head, per layer, or *averaged*, and average budgets can hide that some layers keep nearly everything. Third, a 4× reduction relative to an MHA cache is not a 4× reduction relative to a GQA cache that already saves 8×. A fair comparison states the baseline model, its cache size in bytes per token, and the compressed cache in the same unit.

12.2 Needle-in-a-haystack and why it is not enough

The needle-in-a-haystack (NIAH) test inserts a short fact ("the magic number is 7421") at a chosen depth in a long distractor text and asks the model to retrieve it [158]. It popularised the heat-map plots of retrieval accuracy as a function of context length and depth. It has real value as a smoke test: a model or compression method that fails NIAH is broken. But passing it is weak evidence, for four reasons.

1. **Single lookup with literal overlap.** The question shares words with the needle, so a retrieval head (Chapter 6 [188]) copying the matching string suffices; the task does not test reasoning, aggregation or multi-hop use of the context. NoLiMa constructs needles

with minimal lexical overlap with the question and finds that performance drops steeply as context grows for many models that look strong on literal-match tests [164].

2. **Saturation.** Strong models pass at all depths up to their claimed window, leaving no signal to distinguish methods.
3. **The needle is easy to preserve.** Methods that score tokens by attention from a final question (SnapKV and relatives) preserve exactly the tokens that match the question, i.e., the needle. A synthetic benchmark of this form rewards what such methods do best.
4. **Needle tests do not test the dependency structure of real tasks**, in which many facts must be used jointly, or an intermediate result produced by the model itself must be reused later.

12.3 Beyond NIAH: structured long-context benchmarks

RULER. RULER extends NIAH to multi-needle and multi-value retrieval, *variable tracking* (follow a chain of assignments, a multi-hop task), *aggregation* (count or list the most frequent words, which requires attending to the whole context and cannot be answered from a few tokens) and question answering with distractor documents. Its central finding is that models with a claimed context window of 32K or more often fail to maintain performance at that length on such tasks, so claimed and *effective* context lengths differ [157]. Aggregation tasks are particularly instructive for compression: they *require* that mass be spread over many tokens (the flat-attention regime of Proposition 2.6), which sparse methods are least equipped to serve.

HELMET argues that synthetic long-context tasks are poor predictors of performance on application-style tasks and evaluates models over several categories with controlled lengths, recommending to use diverse application tasks together with robust prompting and metrics [162]. **BABILong** embeds reasoning facts into long irrelevant text; its authors report that popular models make effective use of only a small fraction of their stated context on it, and it supports architectures with recurrent memory in comparison [163]. **Michelangelo** introduces latent-structure queries in which the model must discover and use structure distributed across the context rather than retrieve a literal needle [165].

12.4 Cache-specific benchmarks and findings

12.4.1 SCBench: the life cycle of the cache

SCBench evaluates methods across the *life cycle* of a KV cache (generation, compression, retrieval, loading) in scenarios where the context is *shared* across multiple requests or turns, which is the setting of real deployments [166]. Its design embodies the key point that single-query benchmarks, where a question is posed once after the context, favour query-aware compression (SnapKV-style), while multi-query settings break it. In the authors' analysis, methods with sub-linear memory struggle in multi-turn use, whereas methods that keep the full cache but reduce prefill compute (dynamic sparse prefill) remain robust; these are qualitative findings of one benchmark study and depend on the evaluated methods and models [166].

12.4.2 Systematic comparisons

Yuan et al. benchmark a range of KV-cache compression and long-context methods on a broad suite and conclude, in broad terms, that the effect of compression is strongly task-dependent and that no single method dominates [167]; surveys catalogue the methods and their evaluation practices [169–172]. We caution that aggregate benchmark averages across tasks can mask large per-task losses: a method can lose a few points on average by losing 30 points on aggregation tasks and none on retrieval.

12.4.3 KVDiagnosis: why do compressed runs fail?

KVDiagnosis (August 2026) organises 25 compression methods into five mechanism families and implements eight of them, comparing each *against its own uncompressed control* so that each method gets its own failure set (cases the FullCache run answers correctly and the compressed run does not) [168]. On Qwen3-8B with four evidence-aware workloads, it produced 59,800 supported compressed runs over 2,600 sources, of which 12,520 were FullCache-correct-but-compressed-wrong rows. Its main findings, as reported by the authors, illustrate the diagnostic gap.

- About 63% of the failure rows have low or partial measurement coverage: for most failures the available measurements cannot say why the run failed. Only 19 rows (0.2%) combine high coverage with strong likelihood drift.
- All ten diagnostics distinguish failed from successful compressions better than chance, with stratified AUROC between 0.684 and 0.871; none is decisive.
- In 96 reproducible failures with a low ratio of attention to the evidence span, boosting evidence attention by a factor of four repaired 29.2% of failures, versus 6.3% for a count-matched sham intervention, and degraded matched success controls by 3.3%. This indicates that insufficient attention to retained evidence is a causal contributor in some failures, but not in most.

The work is a single study on one model size (8B) and one family; its value is methodological: it shows that *failure analysis*, not aggregate accuracy, is needed, and that current instrumentation explains few failures.

12.5 Evaluating long-generation reasoning

Eviction methods were first developed for prompt-heavy tasks, where the cache grows during prefill and generation is short. Reasoning workloads reverse this: the prompt is short and the cache grows during a long chain of thought (Section 6.7). The evaluation literature is thinner. Methods such as R-KV, ThinKV and ReasonAlloc evaluate on short-answer mathematics (MATH-500, AIME) with distilled reasoning models up to 14B parameters [70–72]. Recommendations for such evaluations:

- **Account for the statistics of small benchmarks.** AIME contains 30 problems per year; a single-run accuracy of 50% has a standard error $\sqrt{0.5 \cdot 0.5/30} \approx 9$ percentage points from problem sampling alone. Differences of a few points between a compressed and a full-cache model are not significant unless many problems and many samples per problem are used and a paired test is applied (the same problems and seeds for both

conditions).

- **Report output length.** If compression causes longer, more repetitive generations, the apparent savings per step may be offset by more steps. Report mean and tail generation lengths and total decode time to the answer, not only accuracy.
- **Include tasks with long dependency chains**: multi-step arithmetic whose intermediate results are used much later, code tasks requiring consistent variable names across hundreds of lines, and tasks requiring exact reuse of a given constraint after thousands of tokens. These are the tasks for which the contraction assumption of Proposition 11.3 is least plausible.
- **Test more than one model family and size**, including models trained with long reinforcement-learning reasoning where traces may be more tightly dependent than in distilled models.

12.6 Evaluating agentic, multi-turn and shared-prefix use

Agent trajectories are long, interleave model output with tool observations, reuse the prefix at every step, and are judged by end-task success. For cache methods this introduces failure modes absent from single-shot benchmarks.

1. **Instruction persistence.** Place a constraint in the system prompt ("never reveal X", "respond only in JSON") and test compliance at turn 50 and 200, under the compression policy in use. Eviction policies that drop system-prompt tokens in favour of recent content are a direct risk (Chapter 14).
2. **Tool-output recall.** Require the model at step k to use a precise value (an ID, a file path) returned at step $j \ll k$.
3. **Cache-hit determinism.** Run the same conversation with a cold cache and with a warm prefix cache and compare outputs under greedy decoding; they should be identical up to the numerical reordering of Proposition 9.3; larger divergence indicates a bug or an approximate reuse path.
4. **End-task success rate** on realistic multi-step tasks (software repair, research) rather than proxy metrics, with enough runs to estimate variance.

Context-management methods for agents (folding, typed eviction) report such end-task metrics in their own papers [144, 145], which are encouraging but come from the authors' suites and a small number of base models.

12.7 Mechanistic and cheap diagnostics

End-task evaluation is expensive; cheap diagnostics can screen methods before running it.

- **Attention-mass retention.** For a held-out set of queries, measure per head the mass $1-\epsilon$ retained by the compression policy, compared with the full cache. By Proposition 11.1 this bounds output error; heads with low retention are candidates for a larger budget (Proposition 6.2).
- **Logit and next-token divergence.** KL divergence between the next-token distributions of the full and compressed model over long held-out continuations; a sensitive, task-independent signal.

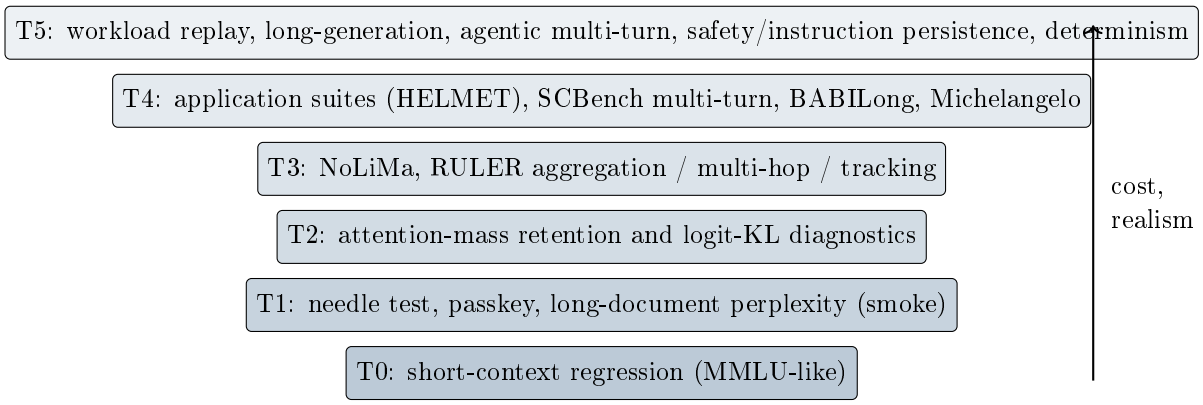


Figure 12.1: A layered evaluation protocol. Lower tiers are cheap screens; a method should not be deployed on the basis of tiers T0–T2 alone.

- **Evidence-attention ratio** for tasks with a known evidence span, as in [168].
- **Perplexity on long documents** as a sanity check; but note that perplexity is dominated by local predictions and is insensitive to missing long-range facts (a model with only a window can have near-normal perplexity).

12.8 A layered protocol

Minimum evidence before deploying a lossy cache reduction

(1) A same-model FullCache control on the *deployment-like* task mix with confidence intervals; (2) results on at least one aggregation-type and one multi-hop-type long-context benchmark; (3) multi-turn tests with shared context if prefix reuse is used; (4) a long-generation test if the workload includes reasoning; (5) an instruction-persistence and safety-prompt test; (6) memory and latency measured at the target batch size against a tuned baseline; (7) a documented *rollback* path.

12.9 Benchmark overfitting and the Goodhart problem

Compression methods are developed and tuned on public benchmarks (LongBench, needle tests), and hyperparameters (observation windows, pooling kernels, budgets, thresholds) are set by validation on those benchmarks. Reported gains therefore partly reflect adaptation to the benchmark's structure. The MiniMax report on why M2 uses full attention makes the same point from the model-design side: a hybrid matched full attention on the standard suite but showed multi-hop reasoning deficits at scale, and proxy metrics built to detect the weakness may not transfer at larger scale or survive attention from the community [36]. Practical mitigations: hold out a private set of tasks from the target workload; change prompt formats and orderings between tuning and evaluation; and monitor production quality signals after deployment (Chapter 15).

Chapter summary

- NIAH is a smoke test; aggregation, multi-hop, low-overlap retrieval, multi-turn and long-generation tasks expose the damage.
- Report bytes per token including metadata, latency tails, goodput, and compare with a tuned baseline and a same-model FullCache control.
- Failure analysis is currently weak: in a diagnostic study, most failures could not be explained by available measurements [168].
- Small reasoning benchmarks have large sampling noise; use paired tests and report output lengths.
- Use a tiered protocol and a held-out workload set; do not deploy on tiers T0–T2 alone.

Table 12.1: Long-context benchmarks relevant to cache compression. "Sensitive to compression?" is the author's qualitative assessment of how likely the benchmark is to reveal damage from cache reduction.

Benchmark	What it tests	Sensitive to compression?	Notes
NIAH [158]	single literal retrieval at depth × length	low	smoke test only
RULER [157]	configurable: multi-key/multi-value NIAH, variable tracking, aggregation (frequent words), QA with distractors	medium–high	reports effective context lengths shorter than claimed
NoLiMa [164]	retrieval with minimal lexical overlap	high	exposes reliance on literal match
HELMET [162]	seven application-oriented categories (RAG, re-ranking, in-context learning, long QA, summarisation, citation, synthetic recall)	high	finds synthetic tasks poorly predict applications
LongBench / v2 [159, 160]	bilingual multitask; v2 stresses deeper reasoning over long realistic inputs	medium	common in eviction papers; v2 harder
∞Bench [161]	tasks beyond 100K tokens	medium	code, math, dialogue, retrieval
BABILong [163]	facts hidden in long background text, reasoning over several facts	high	measures effective use of context; supports recurrent-memory comparisons
Michelangelo [165]	latent-structure queries over long context	high	beyond haystack retrieval
SCBench [166]	KV-cache lifecycle: shared context, multi-request/multi-turn, generation, compression, retrieval, loading	high	designed for cache methods
KVDiagnosis [168]	diagnostic dataset for compression failures with mechanism-level measurements	high	per-method failure sets against own FullCache control

Chapter 13

Recent Advances Timeline and Current Capabilities

13.1 Timeline, 2022 to October 2026

Table 13.1 lists the developments discussed in this paper in chronological order, using the arXiv submission month (or the date stated by the source) for each. Months are those of first public appearance as best we can establish from the cited records; where we could only verify the year, the month is omitted. The list is selective, not exhaustive: it contains items we cite and could verify, and omits many relevant works.

Table 13.1: Timeline of cache, attention and context-compression developments (selective).

Date	Development	Ref.
2019-11	Multi-query attention: one shared K/V head for fast decoding	[2]
2020	Efficient-transformer wave (Longformer, BigBird, Reformer, Performer, Linformer, linear transformers)	[11, 13, 21, 174, 179, 180]
2022-05	FlashAttention: IO-aware exact attention	[110]
2022-07	Recurrent Memory Transformer	[137]
2022-11	Efficiently scaling transformer inference (roofline-style analysis); speculative decoding (2022-11/2023-02)	[120, 124, 125]
2023-03	FlexGen: offloading for throughput	[109]
2023-04	Gist tokens	[132]
2023-05	GQA; Scissorhands; AutoCompressor; RWKV	[3, 23, 48, 133]
2023-06	H_2O heavy hitters; positional interpolation	[47, 150]
2023-07	FlashAttention-2; ICAE; RetNet; lost-in-the-middle	[25, 111, 134, 156]
2023-09	PagedAttention (vLLM); StreamingLLM; YaRN	[46, 95, 151]
2023-10	FastGen; LLMLingua; Ring Attention; HyperAttention; Flash-Decoding; MemGPT	[52, 113, 118, 129, 146, 175]
2023-11	Splitwise; Prompt Cache	[98, 104]
2023-12	Mamba; SGLang/RadixAttention; Gated linear attention; SparQ	[18, 26, 62, 97]
2024-01	DistServe; Medusa; EAGLE; KVQuant; TOVA	[56, 91, 103, 126, 127]
2024-02	KIVI; Hydragen; InfLLM; LongRoPE	[63, 92, 100, 152]

Date	Development	Ref.
2024-03	Sarathi-Serve; Jamba; DMC; Gemini 1.5 (long context to millions of tokens)	[29, 43, 60, 102]
2024-04	SnapKV; RULER; Infini-attention; Mamba-based study	[30, 49, 140, 157]
2024-05	DeepSeek-V2 (MLA); CLA; YOCO; vAttention; Mamba-2	[4, 9, 10, 19, 96]
2024-06	PyramidKV; Quest; Mooncake; BABILong; QJL	[50, 61, 93, 105, 163]
2024-07	MInference; Palu; Ada-KV; Llama 3 herd; FlashAttention-3	[42, 53, 76, 80, 112]
2024-08	Mamba in the Llama (distillation to hybrid)	[89]
2024-10	DuoAttention; MagicPIG; ShadowKV; HELMET; Seer-Attention	[66–68, 75, 162]
2024-12	DeepSeek-V3; Gated DeltaNet; SCBench; LongBench v2	[5, 28, 160, 166]
2025-01	DeepSeek-R1; MiniMax-01 (lightning attention); Titans; TPA; Qwen2.5-1M	[8, 34, 41, 83, 142]
2025-02	NSA; MoBA; TransMLA; MHA2MLA; NoLiMa	[73, 74, 81, 82, 164]
2025-03	Gemma 3 (5:1 local–global, 128K)	[16]
2025-04	Nemotron-H; TurboQuant	[31, 94]
2025-05	Qwen3; KVzip; R-KV (reasoning-trace compression)	[37, 69, 70]
2025-08	gpt-oss (banded + dense, learned sinks); GLM-4.5	[17, 45]
2025-09	Qwen3-Next (3:1 Gated DeltaNet); DeepSeek-V3.2-Exp introduces DSA (stated in V3.2 report)	[6, 38]
2025-10	Kimi Linear (KDA + MLA hybrid); Context-Folding; ThinKV; LMCache	[33, 72, 107, 144]
2025-12	DeepSeek-V3.2 report; Nemotron 3 Nano (hybrid Mamba-Transformer MoE, 1M)	[6, 32]
2026	Qwen3.5 (3:1 pattern), Kimi K2.5 and GLM-5 (MLA; GLM-5 with DSA); MiniMax-M2.5 full attention (as summarised by a secondary source)	[214]
2026-04	DeepSeek-V4 (CSA + HCA, 1M context; ∼10% of V3.2 KV cache at 1M for Pro)	[7]
2026-05	Context Window Lifecycle: typed deterministic eviction for agents	[145]
2026-06	ReasonAlloc: hierarchical decoding-time budget allocation	[71]
2026-08	KVDiagnosis: diagnostic benchmark for KV compression failures	[168]

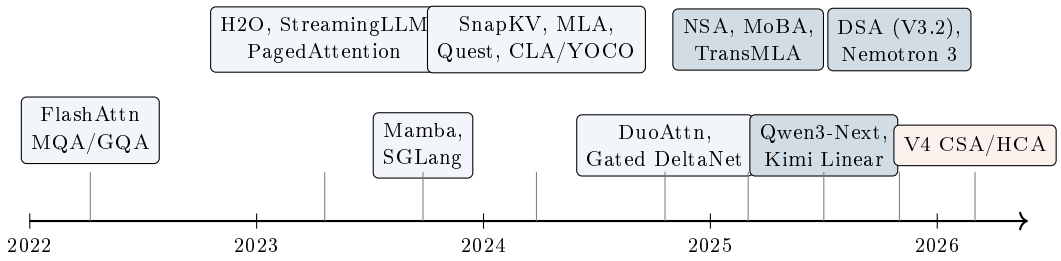


Figure 13.1: Schematic timeline of the main shifts: from exact kernels and head sharing (2022–23), to post-hoc eviction and paging (2023–24), to latent and hybrid architectures (2024), to trainable sparse and production hybrids (2025–26). Positions are approximate and for orientation only.

13.1.1 Four phases

Phase 1 (2019–2022): kernels and head sharing. The bottleneck is identified as memory traffic; MQA and FlashAttention are the main tools. **Phase 2 (2023–early 2024): post-hoc eviction and memory management.** Training-free eviction (H_2O, StreamingLLM, Scissorhands, FastGen), paging, and prefix caching appear within a few months of each other, driven by the open-sourcing of long-context models and by serving costs. **Phase 3 (2024): cache-aware architecture.** MLA, cross-layer sharing, hybrids and local–global patterns shift the effort from compressing the cache to designing the model so that it has less to store; selection methods (Quest) and head-specialised policies (DuoAttention) refine post-hoc approaches. **Phase 4 (2025–2026): trained sparsity and agents.** NSA, MoBA, DSA and V4's compressed attention make sparsity a trained property; hybrids with gated delta rules ship in open flagship models; evaluation matures (RULER, NoLiMa, SCBench, KVDiagnosis); research on reasoning-trace compression and agent context management accelerates.

13.2 What is achievable today

This section tabulates *reported* capabilities. We separate three kinds of entries: architecture-level reductions *derived* from public model configurations (reliable up to the assumptions stated); post-hoc compression results *cited* from the original papers under their own setups; and serving-system results *cited*. None of the cited numbers is a like-for-like comparison, and we strongly recommend reading the original setup before relying on any.

13.2.1 Cache footprints of shipped models (derived)

Table 2.1 gave per-token footprints. Table 13.2 converts them to the number of 128K-token sequences that fit in an illustrative 56 GB cache budget (what remains of an 80 GB accelerator after weights and overhead for a small model).

In the cache-bound regime of Equation (2.8), per-token decode latency at large batch is proportional to cache bytes read, so the relative figures also bound the attainable throughput gains from architecture alone, *before* considering compute, communication or kernel efficiency. For instance, at 32K context the Llama-3-8B-class asymptote is 1.28 ms per token (4.29 GB per sequence over 3.35 TB/s); a cache five times smaller moves it to 0.26 ms, a 5× bound on

Table 13.2: Number of concurrent 128K-token sequences fitting in an illustrative 56 GB cache budget, *derived* from per-token footprints (Table 2.1). For large models the 56 GB is the per-device budget, so the figure is illustrative of relative scale only. Constant-size windows and recurrent states are included where known (gpt-oss: 4.5 MiB; Qwen3-Next: ≈36 MiB; Kimi Linear: ≈20 MiB).

Model (attention)	Cache per 128K seq (GB, decimal)	Sequences in 56 GB	Capacity gain vs. Llama-3-8B
Llama-2-7B (MHA, hypothetical 128K)	68.7	0	—
Llama-3-8B (GQA)	17.2	3	1×
Qwen3-8B (GQA)	19.3	2	0.9×
Llama-3-70B (GQA)	42.9	1	0.4×
DeepSeek-V3 (MLA)	9.2	6	1.9×
gpt-oss-120b (local–global)	4.8	11	3.6×
Qwen3-Next (hybrid)	3.3	17	5.2×
Kimi-Linear (hybrid)	1.1	52	16×

throughput gains (derived; the realised gain is lower because weight reads, attention compute and non-attention layers remain).

13.2.2 Post-hoc compression: reported results

13.2.3 Composition: how far can stacking go? (derived arithmetic)

Table 13.4 shows the *arithmetic* of stacking reductions on one example, a Qwen3-8B-class cache at 128K. This is bookkeeping, not a validated configuration: it assumes the reductions are independent and that quality survives each stage, which must be established by evaluation (Proposition 8.3).

The comparison shows that a hybrid architecture reaches in one design decision about what a three-stage post-hoc stack reaches, with no inference-time quality gamble beyond what the training verified; but it requires a different model, not an existing one.

13.2.4 Context lengths

Open-weight models shipping with 1M-token windows include Kimi Linear, DeepSeek-V4 and Nemotron 3 Nano [7, 32, 33]; Qwen2.5-1M extended Qwen2.5 to 1M tokens [41]; the Gemini 1.5 report describes experiments up to ten million tokens with high needle-retrieval accuracy [43]. The practically usable window is smaller and workload-dependent (Chapter 12).

13.2.5 Summary of capabilities

What is achievable today

- **Architecture:** per-token caches of roughly 8–25 KiB (derived) for open hybrid models, against 128–500 KiB for dense GQA models; one-million-token windows are shipped in open models.
- **Post-hoc, training-free:** 2–4× cache reduction with small loss on retrieval- and summarisation-style benchmarks is reported by many methods; retention of only 12–20% of the cache (5–8×) is reported by PyramidKV and H_2O at their settings, and

up to 20× with 4-bit quantization by Scissorhands, all with the evaluation caveats of Chapter 12.

- **Low-rank/conversion:** ∼50% reduction post-hoc (Palu); >90% with fine-tuning and partial RoPE (MHA2MLA, 92.19% on Llama2-7B).
- **Trained sparse:** substantial compute reductions at 64K–1M tokens are reported (NSA at 64K; V4-Pro at ∼27% of V3.2's single-token FLOPs at 1M); DSA's savings are cost-model-derivable (∼8× fewer bytes at 128K under stated assumptions).
- **Serving:** paging and prefix caching are default features; disaggregation and tiered offload are in production at large operators.
- **Agent context:** learned folding and typed eviction report ∼10× smaller active contexts and very long sessions in single-lab suites.

Table 13.3: Reported capabilities of post-hoc and serving-level techniques. All figures are *claims from the cited source under its own model, benchmark and baseline*; they are not comparable across rows. "Reduction" is of cache or attention cost as the source defines it.

Technique	Reported result	Caveat from the source setup
StreamingLLM [46]	stable streaming to 4M tokens; up to 22.2× over sliding-window recompute	does not extend memory; baseline recomputes
H_2O [47]	20% heavy hitters; up to 29× throughput (3× vs. FlexGen)	baselines are offload systems; OPT-class models
Scissorhands [48]	up to 5× cache; up to 20× with 4-bit	early models; benchmark subset
SnapKV [49]	3.6× faster generation, 8.2× memory efficiency at 16K; 380K on one 80 GB GPU	question at end of prompt (query-aware)
PyramidKV [50]	∼12% cache matches full on LongBench; benefit at 0.7% retention	LongBench aggregate
DuoAttention [68]	up to 2.55× (MHA) / 1.67× (GQA) memory; up to 2.18×/1.50× decode; 3.3M tokens with quantization	Llama-family; synthetic gate training
Quest [61]	up to 7.03× attention speedup; 2.23× end-to-end	keeps full cache (capacity unchanged)
KVzip [69]	3–4× cache; ∼2× decode latency; contexts up to 170K	extra scoring pass; reused contexts
Palu [80]	∼50% cache; up to 1.89× (2.91× with quantization) attention-module speedup	calibration; reconstruction overhead
MHA2MLA [81]	92.19% cache on Llama2-7B with 0.5% LongBench drop; ∼0.3–0.6% of training data	light fine-tuning required
NSA [73]	substantial speed-ups on 64K sequences in decode, forward and backward (abstract gives no factor)	natively pretrained MoE model; exact factors not quoted
MoBA [74]	superior long-context performance; deployed for Kimi's long-context requests (abstract); no speedup factor verified	block routing; authors' deployment claim
Kimi Linear [33]	up to 75% lower KV cache; up to 6× decode throughput at 1M	vs. full MLA, same recipe, 48B/3B active
DeepSeek-V4 [7, 213]	Pro ∼27% FLOPs and ∼10% cache of V3.2 at 1M; Flash ∼10% FLOPs, 7% cache	authors' report; preview models
Nemotron 3 Nano [32]	up to 3.3× throughput vs. similar-size open models; 1M context	MoE hybrid; throughput setups per authors
TurboQuant [94]	quality neutral at 3.5 bit-	KV quantization; vendor benchmarks

Table 13.4: Illustrative stacking arithmetic for a dense GQA model at 128K tokens (18 GiB bf16 baseline, Table 2.1). Reductions are applied multiplicatively; quality is *not* validated.

Stage	Cumulative size (GiB)	Cumulative reduction
bf16 baseline	18.0	1×
4-bit group-32 quantization ($b_{\text{eff}} = 5$)	5.6	3.2×
+ 4× token eviction (e.g., prefill-time)	1.4	12.8×
+ 2× low-rank on keys/values (Palu-like)	0.7	25.7×
Hybrid alternative (Qwen3-Next-like architecture, bf16)	3.0	6×
Hybrid + 4-bit group-32	0.94	19×

Chapter 14

Pros, Cons and Risks

14.1 Pros and cons by method family

Table 14.1 summarises, for the sixteen sub-topics scored in Chapter 16, what each offers and what it costs. Entries are the author's synthesis of the preceding chapters; the evidence is cited there.

Table 14.1: Pros and cons of the sixteen sub-topics.

Sub-topic	Pros	Cons
1. Paging, chunked prefill, scheduling	lossless; near-zero fragmentation; enables continuous batching and sharing; mature in all major engines	does not shrink per-token cache; kernel indirection; tuning of block and chunk sizes
2. Prefix caching / radix reuse	lossless for exact prefixes; large TTFT and cost savings for chats, agents, RAG with fixed context	only exact-prefix hits; memory pressure from cached prefixes; cache-hit nondeterminism; potential timing side channels in shared deployments
3. Disaggregated prefill/decode	isolates compute- and memory-bound phases; better goodput and tail latency at scale	network traffic equals cache bytes; operational complexity; needs fast interconnect and scheduler
4. Tiered offload and KV network compression	capacity beyond HBM; cross-instance reuse; pays when load $<$ recompute	per-step offload needs extreme sparsity; extra moving parts; lossy network compression adds approximation
5. GQA / MQA	simple, universal, uptrainable; reduces cache and raises intensity by g	quality loss at high g; limited headroom (already the baseline)
6. MLA and latent architectures	$\sim$57$\times$ smaller than MHA at DeepSeek scale; absorbed decode compute-bound; conversion papers exist	decoupled RoPE complexity; specialised kernels; replicated latent under TP; conversion needs fine-tuning and is lossy
7. Local–global / windows	constant-size cache for most layers; cheaper prefill; simple	needs global layers and sinks; long-range retrieval limited to global layers

Sub-topic	Pros	Cons
8. Hybrid linear / SSM	lowest per-token cache (8–25 KiB/token derived); better throughput at 1M; shipped in open models	recall limits; non-truncatable state complicates prefix caching and speculation; one report of multi-hop weakness at scale; immature low-precision state
9. Trainable sparse attention	removes train–inference mismatch; large compute savings at 64K–1M; DSA shows retrofit by continued pretraining	most variants keep full cache (capacity unchanged); selector quality risk; kernels hardware-specific; evidence concentrated in few labs
10. Training-free eviction	no training; fast to deploy; reduces capacity and bandwidth	fails silently on unseen queries, multi-hop, long generation; breaks prefix sharing; incompatible with fused kernels without extra work
11. Query-aware selection	near-full quality at small read budgets; compatible with full cache and offload	capacity unchanged; selector overhead; approximate nearest-neighbour recall
12. KV merging	retains information from all tokens; good for redundant text	blurs distinct facts; needs size-correct softmax; limited ratio post hoc
13. Post-hoc low-rank / SVD	principled (Theorem 8.1); composes with quantization	RoPE complicates; reconstruction overhead; spectrum varies; calibration dependence
14. KV quantization (cross-ref.)	simple, widely supported; 2–4× with small loss at 4-bit; composes with everything	metadata overhead at low bits; outlier handling; scale format fragmentation; errors add to eviction errors
15. Prompt / context compression	reduces prefill and cache; works with API models (hard)	changes the text; loses specifics; breaks prefix reuse; learned compressors need training
16. Agent context management	addresses real agent workloads; learned folding gives ∼10× smaller contexts; typed eviction deterministic	policy errors; summary drift; evidence from few papers; hard to evaluate end-to-end

14.2 Risk register

The risks below are those the author considers most consequential for production use of the techniques in this paper. Likelihood and impact ratings are the author's *estimates* informed by the cited literature and general engineering experience; they are not measurements.

14.2.1 R1: Silent retrieval failure

Silent retrieval failure

Mechanism. When a token carrying a needed fact is evicted, merged, overwritten in a recurrent state, or missed by a selector, the model has no signal that information is missing. It continues to generate fluent text, often a plausible but wrong answer. No exception, no log line. **Why it is hard.** Eviction error is bounded by the dropped attention mass (Proposition 11.1), but the mass of a single critical token may be negligible *in aggregate statistics* and decisive for one query.

A quantitative illustration: suppose the compression policy knows nothing about which token will be needed, so a given token survives with probability equal to the retained fraction ρ. A fact held in one token is lost with probability $1 - \rho$, and a task needing m independent facts succeeds with probability ρ^m. At $\rho = 0.25$ a single-fact task loses 75% of the time if the policy is uninformative, and a three-fact task succeeds with probability 1.6% (derived). Informed scoring (query-aware or attention-based) raises survival of *relevant* tokens far above ρ, which is why benchmarks with a question at the end look good; but the model of an uninformative policy is the correct baseline for *queries the scoring did not anticipate*. **Likelihood: high** for training-free query-agnostic methods; **impact: high** in retrieval-critical applications. **Mitigations:** budgets validated on multi-fact and aggregation tasks; canary facts (known strings planted in the context and probed periodically) with alarms when recall falls; attention-mass monitors (Chapter 12); fallback to the full cache for flagged requests; pinning of critical spans.

14.2.2 R2: Multi-turn and prefix-sharing breakage

Multi-turn and prefix-sharing breakage

Query-aware eviction chooses a cache for *this* question. The next turn asks about something else; the evicted tokens are gone. Prefix caches that store the compressed state make the loss *permanent* for every later request that hits it; per-request compression forfeits prefix reuse. Recurrent states in hybrids can only be reused at checkpoints; a mismatch between checkpoint positions and the prefix boundary forces recomputation or, worse, a wrong state if keys are mismatched.

Proposition 9.3 gives the exact conditions for valid reuse. Violating condition (iii) (a cached prefix modified by information from later tokens) is the root cause of cross-turn failures. SCBench [166] was designed to exercise exactly these scenarios. **Likelihood: high** for query-aware eviction in chat/agent use; **impact: medium–high. Mitigations:** compress only query-agnostically (KVzip-like) when the context is shared; keep the uncompressed prefix cache and compress on a per-request copy; version cache entries by (model, position map, compression policy) hash; cold-versus-warm cache equivalence tests in CI.

14.2.3 R3: Reasoning-trace degradation

Reasoning-trace degradation

Decoding-time eviction acts on the model's *own* thoughts. Long chains of thought contain intermediate results (a computed number, a case split, a definition) that later steps use. Evicting such tokens produces inconsistent reasoning that can look locally sound. Errors persist in the cache (Proposition 11.3). Compression may also lengthen outputs, erasing efficiency gains.

Evidence on reasoning-specific compression is mainly from short-answer mathematics with distilled models up to 14B parameters [70–72]. Evidence from models trained with long reinforcement-learning reasoning, from code, or from proofs is scarcer. **Likelihood: medium** (unknown for frontier reasoning models); **impact: high** where correctness matters. **Mitigations:** redundancy-aware rather than recency-only policies; larger budgets for reasoning than for retrieval; test long-dependency tasks; track output length and answer consistency (self-consistency across samples); an uncompressed "verification" pass for high-stakes answers.

14.2.4 R4: Instruction and safety text eviction

Instruction and safety-text eviction

System prompts, safety policies, tool schemas and user constraints sit at the *beginning* of the context. Recency windows drop them; attention-based scores may rate them low because the model attends to them diffusely. Sinks keep only the first few tokens. The model can then "forget" a refusal policy or an output-format constraint while still functioning fluently.

We are not aware of a systematic published study quantifying safety-behaviour loss under KV compression; the mechanism is plausible from the structure of the methods (recency and attention-mass heuristics are indifferent to the *normative* importance of tokens), and from the established importance of instruction hierarchy in robust models [198]. This is a hypothesis to be tested, not an established result. **Likelihood: unknown, plausibly medium**; **impact: high. Mitigations:** *pin* system and developer messages and tool definitions as non-evictable regions; evaluate instruction persistence at turn 100+ under compression; run output-side safety classifiers that do not depend on the cache; include red-team prompts in compression regression suites.

14.2.5 R5: Adversarial and prompt-injection interactions

Adversarial and injection interactions

(i) **Cache flooding:** attacker-controlled text can be crafted (repetition, large-norm keys, delimiter patterns that attract attention) to accumulate score and displace legitimate content under heavy-hitter policies. (ii) **Retention of injections:** injected instructions that attract attention may be retained while benign user constraints are evicted. (iii) **Compressor as attack surface:** an LLM that summarises untrusted tool output can be injected, and the resulting summary persists as trusted-looking memory. (iv) **Longer contexts, larger attack surface:** many-shot jailbreaking exploits long contexts [216]. (v) **Shared-**

cache side channels: prefix-cache hits change latency, which can leak whether another tenant has used the same prefix.

Indirect prompt injection into LLM-integrated applications is well documented [196, 197]; interactions with cache policies have received little study. Items (i)–(iii) and (v) are inferences from mechanism, not demonstrated results in published evaluations that we could verify. **Likelihood: medium** in exposed deployments; **impact: medium–high. Mitigations:** treat the retention policy as a security-relevant component; per-tenant cache salting; mark untrusted spans and exclude them from score accumulation or cap their share of the budget; never let a model-written summary of untrusted text be placed in the privileged part of the context without provenance marking.

14.2.6 R6: Nondeterminism

Nondeterminism

Floating-point addition is not associative. Split-KV reductions (Algorithm 10), kernel-selection heuristics that depend on batch size and sequence length, prefix-cache hits versus recomputation, and tied scores in top-k selection can change the last bits of logits, and with sampling at low temperature or greedy decoding near a tie, can change the generated text. Compression policies amplify this: a flipped top-k membership produces a different cache.

Likelihood: high (it occurs by default); **impact: low–medium** (reproducibility, flaky tests, hard debugging; rarely correctness). **Mitigations:** deterministic kernels and fixed split counts where reproducibility is required; stable tie-breaking by position; log compression decisions; equivalence tests with tolerance.

14.2.7 R7: Kernel and hardware coupling

Kernel and hardware coupling

Gains are realised only with specialised kernels (MLA decode, sparse top-k gathers, linear-attention scans, 8-bit and 4-bit indexer paths). Kernels depend on specific accelerator generations and low-precision formats; engine support lags papers; an optimisation that wins on one GPU generation may not on another; a bug in a custom kernel is a correctness risk.

DeepSeek-V4's use of 8-bit KV and a 4-bit indexer [213] illustrates the coupling to hardware formats. **Likelihood: high**; **impact: medium** (cost, portability, delayed adoption). **Mitigations:** choose methods with maintained open kernels; keep a reference implementation for correctness tests; pin versions; budget engineering time for kernel maintenance.

14.2.8 R8: Benchmark overfitting and evaluation gaps

Compression hyperparameters tuned on public suites inflate reported gains; standard benchmarks can hide multi-hop weaknesses (the MiniMax report [36]); the diagnostic gap (most failures unexplained [168]). See Chapter 12. **Likelihood: high; impact: medium–high.**

14.2.9 R9: Calibration and distribution shift

Calibration-based methods (Palu's projections, DuoAttention's head gates, rank allocation by sensitivity, learned indexers) assume that deployment inputs resemble calibration data. Domain shifts (code, multilingual text, tabular data, long tool outputs) can break head labels or subspaces. **Likelihood: medium; impact: medium. Mitigation:** calibrate on a mix covering the deployment domains; periodic re-validation.

14.2.10 R10: Numerical hazards in recurrent and low-precision state

Recurrent states accumulate rounding error through the recurrence; low-precision storage of state or cache (8-bit, 4-bit) adds outlier-sensitive error. The MiniMax team cites low-precision state storage among open problems for linear architectures [36]. **Likelihood: medium; impact: medium. Mitigation:** higher-precision accumulators and state for recurrent layers; calibrated per-channel scales for quantized K.

14.2.11 R11: Operational and economic risks

Savings may not materialise: if throughput is bound by something else (weights at small batch, communication, scheduling), a 4× cache reduction yields much less than 4× cost savings (Equation (2.8)). *Observability loss*: compressed systems hide the information that would reveal failure (dropped mass) unless monitors are added. *Capacity planning errors*: variable-length caches under per-head policies complicate admission control. *Rollback*: a model whose function has changed (MLA conversion, linearisation, fine-tuned compression) cannot be rolled back by turning off a flag.

14.2.12 R12: Privacy and data retention

Tiered caches in CPU memory, local disk and remote stores persist user content in places not designed for it; cross-request reuse with misconfigured keys can leak content across users; cache entries encode the content (they can be inverted to a large degree). **Likelihood: low–medium; impact: high** where sensitive data is processed. **Mitigations:** tenant-scoped keys and salts, encryption at rest, retention limits, deletion on request.

14.3 Risk by family

Chapter summary

- Lossless serving techniques carry operational and privacy risks; lossy techniques carry correctness risks that are *silent*.
- The highest-impact, highest-likelihood risk is silent retrieval failure; the most understudied are safety-text eviction and adversarial interaction with retention policies.
- Pin privileged text, add canaries and mass monitors, test cold-versus-warm cache equivalence, and keep a rollback path.

Table 14.2: Risk register summary. L and I are *author's estimates* of likelihood and impact in production deployments of the affected methods: H high, M medium, L low.

ID	Risk	Mostly affects	L	I	Primary detection
R1	Silent retrieval failure	eviction, merging, recurrent state, selectors	H	H	canaries; mass monitors
R2	Multi-turn / prefix breakage	query-aware eviction, prefix caches, hybrids	H	M–H	cold vs. warm equivalence tests
R3	Reasoning degradation	decode-time eviction	M	H	long-dependency tests; length tracking
R4	Instruction/safety eviction	recency and score-based eviction	M?	H	persistence tests at depth
R5	Adversarial interaction	score-based eviction, summaries, shared caches	M	M–H	red-team in CI
R6	Nondeterminism	all with split reductions, caches	H	L–M	repeat runs; tolerance tests
R7	Kernel/hardware coupling	sparse, MLA, low-precision paths	H	M	reference-implementation tests
R8	Benchmark overfitting	all lossy methods	H	M–H	held-out workload
R9	Calibration shift	Palu, DuoAttention, learned gates	M	M	drift monitors
R10	Numerical hazards	recurrent, low-precision	M	M	precision ablations
R11	Operational/economic	all	M	M	cost accounting at target load
R12	Privacy	tiered and shared caches	L–M	H	access audits

Table 14.3: Which risks bite which families (author's judgment): (●) primary, (○) secondary, blank negligible.

Family	R1	R2	R3	R4	R5	R6	R7	R8	R9	R10	R11	R12
Paging/scheduling						○					○	
Prefix caching		○			○	○						●
Disaggregation / offload						○	○				○	●
GQA / MLA							○					
Local–global	○			○				○				
Hybrid linear / SSM	●	●	○				●	●		●		
Trainable sparse	○		○				●	○	○			
Training-free eviction	●	●	●	●	●	○	○	●	○			
Query-aware selection	○					○	●	○				
Merging / low-rank	●		○				○	○	●	○		
Quantization	○		○				○		○	●		
Prompt compression	●	○		○	●			○	○			
Agent context mgmt.	●	○	○	●	●			○			○	○

Chapter 15

Practical Decision Guide

This chapter turns the survey into decisions. It offers a flowchart, scenario tables, a bottleneck-to-lever map, and a rollout checklist. The recommendations are the author's synthesis of the cited evidence and are conditional on workload; where evidence is thin we say so.

15.1 Decision flowchart

15.2 Scenario tables

Table 15.1: Recommended stacks by scenario (author's judgment).

Scenario	Recommended stack	Avoid / caution
Chat assistant with shared system prompt	paging + prefix caching with tenant salt; GQA/MLA model; optional 8-bit KV; pinned system prompt	per-request eviction of the shared prefix; dynamic RoPE scaling
RAG over a fixed corpus	prefix-cache the corpus when it fits; otherwise chunk-level reuse with recomputation (CacheBlend-class) for non-prefix; retrieval + moderate context; place key passages at ends	aggressive prompt compression of numeric/entity-heavy passages
Repository-scale coding assistant	long-context model (MLA/hybrid), prefix caching of repository prefix, tiered offload; test exact-identifier recall	recency-only windows (identifiers far away are lost)
Single-shot long-document QA	SnapKV-class prefill compression (question known) at $\geq$25% budget; or sparse prefill (MInference-class) to cut TTFT	reusing the compressed cache for a different question
Multi-question session on one long document	query-agnostic compression (KVzip-class) or selection (Quest-class) on the full cache; offload full cache	SnapKV-class question-conditioned compression

Scenario	Recommended stack	Avoid / caution
Reasoning-model serving (long CoT)	architecture with small cache (MLA/hybrid) and large batch; if post-hoc, redundancy-aware eviction at moderate budget; speculative decoding	recency-only eviction; low budgets without long-dependency tests
Long-horizon agents	context folding or typed eviction, prefix caching of stable prefix, pinned tool schemas; external memory for durable state	repeated LLM summarisation of summaries; untrusted tool output in privileged context
On-device / edge	small GQA model with KV quantization (4-bit), sliding-window/local–global architecture, StreamingLLM-style sinks for streams	expecting long-range recall from window-only caches
Offline batch (throughput-oriented)	large batches, paging, KV quantization, offload à la FlexGen; hybrid architecture for very long inputs	latency-oriented features; per-request tuning
Multi-tenant API at scale	disaggregation, prefix caching with salting, KV transfer compression, admission control with cache-aware scheduling	cross-tenant cache sharing without isolation

15.3 From bottleneck to lever

15.4 Choosing an open model: what the cache says

When a model can be chosen, its cache per token (Table 2.1) and its attention class largely determine long-context economics. A reading guide:

- **Dense GQA** (Llama-3, Qwen3 dense): 128–500 KiB/token; mature kernels and tooling; use paging, prefix caching, 8-bit KV; expect cache-bound at 128K.
- **MLA** (DeepSeek-V3-class, Kimi K2): ∼69 KiB/token; absorbed decode compute-friendly; heavy-weight models need expert parallelism and data-parallel attention; check kernel support in the chosen engine.
- **Local–global** (Gemma 3, gpt-oss): ∼5–6× cache savings at long context in the 5:1 design; good for chat with moderate context; validate long-range retrieval to the extent the global layers deliver.
- **Hybrid linear / SSM** (Qwen3-Next, Kimi Linear, Nemotron 3): 8–25 KiB/token plus fixed state; best capacity and throughput at 128K–1M; validate multi-hop and recall; check prefix-caching and speculative-decoding support for recurrent state.
- **Trained sparse** (DeepSeek V3.2/V4-class): lower attention compute and bandwidth at long context; V4-class compression reduces capacity; engine support and kernels are the gating factors.

In all cases compare models on *your* workload, at your target context length, with a tiered

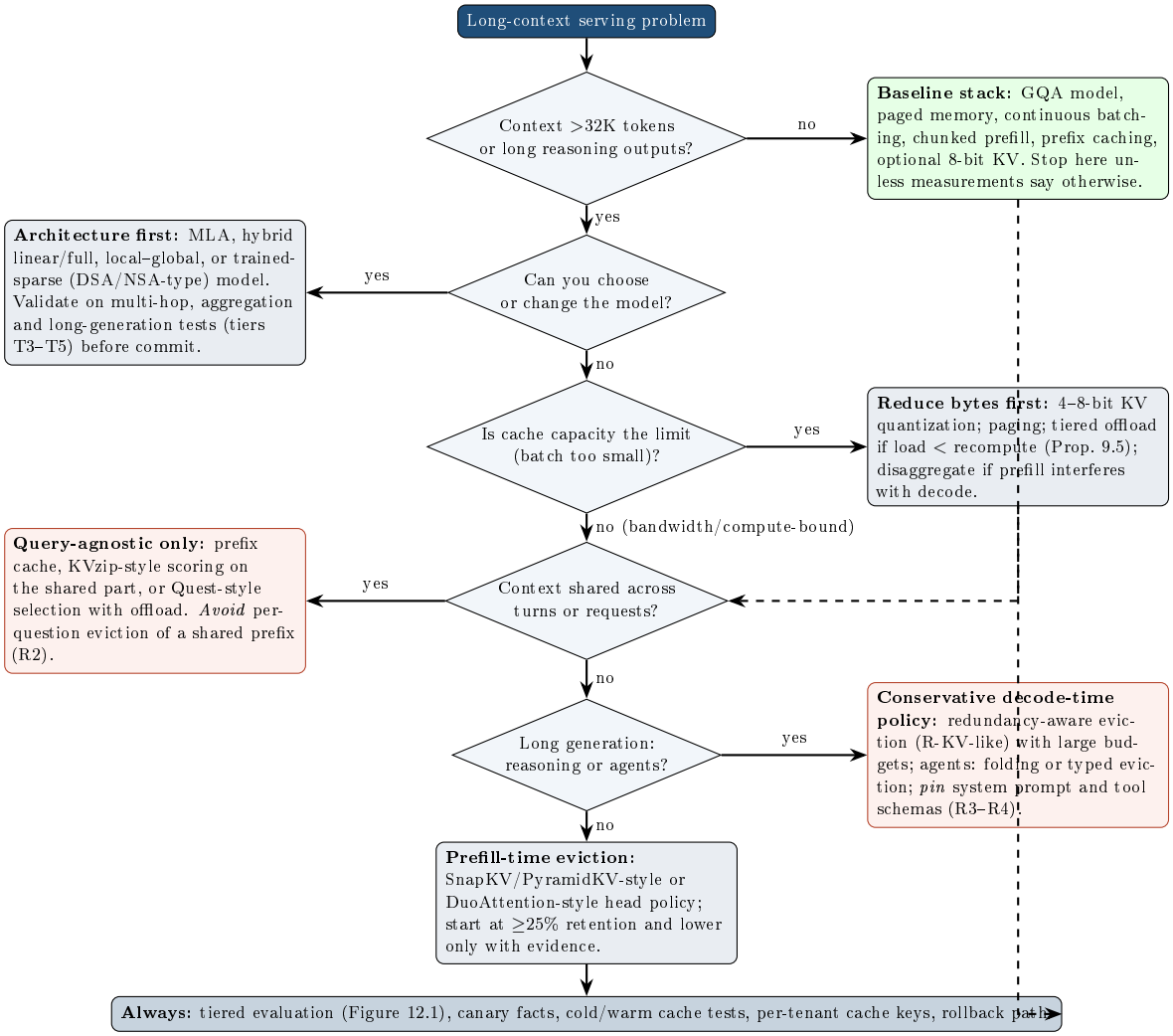


Figure 15.1: Decision flowchart for long-context serving. Architecture choice dominates when available; post-hoc compression is a later, evidence-gated step. Boxes are recommendations, not guarantees.

Table 15.2: Which lever addresses which bottleneck. ✓✓ strong, ✓ moderate, – none; author's assessment.

Lever	HBM capacity	Decode BW	TTFT	Network	Cost	Note
GQA / MLA model	✓✓	✓✓	✓	✓✓	✓✓	needs model change
Hybrid linear model	✓✓	✓✓	✓✓	✓✓	✓✓	recall and serving-complexity risk
Trained sparse model	–	✓✓	✓✓	–	✓✓	cache full unless compressed
KV quantization	✓	✓	–	✓	✓	2–4× at low risk
Token eviction	✓✓	✓✓	–	✓	✓	silent-failure risk
Selection (Quest etc.)	–	✓✓	–	–	✓	needs full cache
Paging	✓ (utilisation)	–	–	–	✓✓	near-universal
Prefix caching	✓	–	✓✓	–	✓✓	exact prefixes only
Chunked prefill	–	–	✓ (tail latency)	–	✓	scheduling
Disaggregation	–	✓	✓	(adds traffic)	✓	at scale
Offload	✓✓	– (hurts)	✓ (reuse)	–	✓	if load < recompute
Speculative decoding	–	✓✓	–	–	✓	better with cache-bound regime
Prompt compression	✓	✓	✓✓	✓	✓	changes the text

evaluation (Figure 12.1); the cache size is a cost metric, not a quality metric.

15.5 Go / no-go checklist for a lossy cache method

1. Quantify the benefit on the real bottleneck (Table 15.2); estimate the realised gain with Equation (2.8), not the paper's headline.
2. Run tiers T0–T4 of Figure 12.1 with a same-model FullCache control; include aggregation and multi-hop.
3. Run T5 for the workload class (reasoning, agents, multi-turn), including instruction persistence.
4. Verify cold-versus-warm cache equivalence and determinism tolerance.
5. Establish monitors in production: attention-mass retention proxy, canary recall, output-length distribution, user-facing quality signals.

6. Roll out in stages: shadow traffic (compare against FullCache offline), then a small percentage with automatic rollback triggers, then ramp.
7. Document the rollback path and keep the uncompressed configuration deployable.

Chapter summary

- Start with the baseline stack (GQA/MLA, paging, prefix caching, 8-bit KV); add architecture choices before post-hoc compression; add lossy token-level compression last and only with evidence.
- Match the method to the query structure: single-shot question (query-aware ok), shared context or multi-turn (query-agnostic or selection), long generation (conservative, redundancy-aware).
- Pin privileged text; monitor; keep rollback.

Chapter 16

Estimated Priority in the LLM Engineering Community

Read this first: what these scores are and are not

The scores in this chapter are the **author's subjective judgment**, informed by the evidence cited throughout this paper (adoption signals in public model configurations and engine documentation, publication momentum, hardware trends, reported headroom and the risk analysis of Chapter 14). They are **not measured data**. There is no survey, no download or deployment statistic, and no citation count behind them. Two informed readers could reasonably assign different scores; the sensitivity analysis shows how much the ranking depends on such disagreement, not that the ranking is right. Where a score rests on thin evidence we say so and give a low confidence.

16.1 What "priority" means

We define the *priority* of a sub-topic as the degree to which, *today*, a competent LLM engineering organisation that serves long-context workloads should direct engineering and research attention to it, weighing how widely it is already used, how fast it is moving, whether the hardware and software ecosystem is pulling in its direction, how much benefit is still unclaimed, and how risky it is. Note the consequences of this definition. A technique that is universal and settled (GQA) has *high importance* but only moderate *priority for new effort*, because its marginal benefit is exhausted; a technique that is promising but unproven (hybrid models) may rank high on momentum and benefit but be held back by risk.

16.2 Rubric

Each sub-topic is scored from 1 (low) to 5 (high), in half-point steps, on five criteria.

The priority score is the weighted mean $P = \sum_k w_k S_k$ on a 1–5 scale. The sixteen sub-topics are the rows of Table 14.1. Speculative decoding and retrieval-versus-long-context choices are discussed in the text as adjacent topics and are not scored.

Table 16.1: Scoring rubric. Weights are the author's default; alternative weightings are explored in the sensitivity analysis.

Code	Criterion	Weight	Scoring guide (5 means ...)
A	Production adoption	0.30	shipped by most serious serving stacks and models; 1 means research-only
B	Research activity and momentum	0.15	many new papers and rapid improvement in 2025–2026; 1 means dormant
C	Hardware and software tailwind	0.20	aligned with hardware trends and with maintained kernels and engines; 1 means fighting the hardware or lacking kernel support
D	Marginal benefit remaining	0.20	large unclaimed cost or capability gains; 1 means essentially exhausted
E	Risk-adjusted safety	0.15	low correctness, operational and security risk (inverse of risk); 1 means high and hard-to-detect risk

16.3 Scores and ranking

16.3.1 Sensitivity to the weighting

Different readers weight the criteria differently. Table 16.3 recomputes the ranking under four alternative weight vectors: *adoption-heavy* ($A = 0.5$, $B = 0.1$, $C = 0.2$, $D = 0.1$, $E = 0.1$), *forward-looking* ($A = 0.1$, $B = 0.3$, $C = 0.2$, $D = 0.3$, $E = 0.1$), *risk-averse* ($A = 0.25$, $B = 0.05$, $C = 0.15$, $D = 0.1$, $E = 0.45$) and *equal* (all 0.2). The scores themselves are unchanged.

16.4 Rationale by sub-topic

Prefix caching / radix reuse (rank 1, confidence high). Near-universal in production engines; savings are large and *lossless* for the common workloads (chat, agents, RAG with repeated context); research momentum is high because agents multiply reuse; hardware tailwind is strong (tiered memory); marginal benefit remains in non-prefix reuse and in cross-instance sharing. Risks are modest (privacy, side channels, hybrid checkpoint semantics). Its first place is robust to small perturbations of scores and holds across all weightings (ranks 1–3).

MLA and latent architectures (rank 2, high). MLA is used by the DeepSeek family, Kimi and, as full-attention layers, by Kimi Linear; conversion papers broaden access; the V4 design extends the latent idea with compression. It ranks second to fourth across the five weightings. Risk: kernels and parallelism complications.

Paging, chunked prefill, scheduling (rank 3, medium). Adopted everywhere, mature; but its marginal benefit is low (the large win was taken in 2023). Its rank depends strongly on weighting (ranks 1–9): high when adoption and safety dominate, low when headroom dominates. It is a *must-have* rather than a place for new effort.

Table 16.2: Scores (A–E), weighted priority P, the 10th–90th percentile of rank under random ± 0.5 perturbation of every individual score (20,000 draws; a measure of the ranking's fragility to small scoring disagreement, computed from the author's scores, not from external data) and the resulting confidence level. All scores are subjective.

Rank	Sub-topic	A	B	C	D	E	P	Rank 10–90%	Conf.
1	Prefix caching / radix reuse	5	4	5	4	4	4.50	1–1	High
2	MLA and latent architectures	4	5	4	4	4	4.15	2–4	High
3	Paging, chunked prefill, scheduling	5	3	5	2	5	4.10	2–5	Medium
4	KV quantization (cross-ref.)	4	4	5	3	3.5	3.92	3–7	Medium
5	Disaggregated prefill/decode	4	4	4	4	3	3.85	3–8	Medium
6	Hybrid linear / SSM architectures	3	5	4	5	2.5	3.83	4–9	Low
7	Agent context management	3.5	5	3	5	2.5	3.78	5–10	Low
8	GQA / MQA	5	2	5	1	5	3.75	6–10	Low
9	Trainable sparse attention	3	5	4	4	3	3.70	5–11	Low
10	Local–global / sliding window	4	3	4	3	4	3.65	6–11	Medium
11	Tiered offload & KV network compression	3	4	4	4	3	3.55	8–11	Medium
12	Query-aware selection	2	4	3	3	3	2.85	12–13	Medium
13	Prompt / context compression	3	3	2	3	2.5	2.73	12–14	Medium
14	Post-hoc low-rank / SVD	1.5	3	3	3	3	2.55	13–15	Medium
15	Training-free token eviction	2	4	2	3	2	2.50	13–15	Medium
16	KV merging	1.5	3	2	2	2.5	2.08	16–16	Medium

KV quantization (rank 4, medium; cross-reference). 8-bit KV is commonly available and 4-bit is increasingly so; strong hardware tailwind (low-precision formats); headroom in 2–3-bit schemes with theoretical guarantees (Chapter 8). Error interaction with eviction and recurrent state is a known concern. The quantization paper in this series treats it in depth.

Disaggregated prefill/decode (rank 5, medium). In production at large operators; valuable at scale; limited relevance for small deployments. Cache-reducing architectures lower its network bill (Table 9.1).

Hybrid linear / SSM architectures (rank 6, low confidence). The highest momentum and benefit scores in the set (shipped by Qwen, Kimi, NVIDIA; the lowest per-token caches), balanced by risk (recall, prefix caching, speculation, one credible report of multi-hop weakness and a reversion to full attention at MiniMax [36]). Its rank ranges from 1 (forward-looking weighting) to 10 (risk-averse), reflecting real uncertainty about where the field will settle. *Confidence is low because outcomes depend on evidence not yet in.*

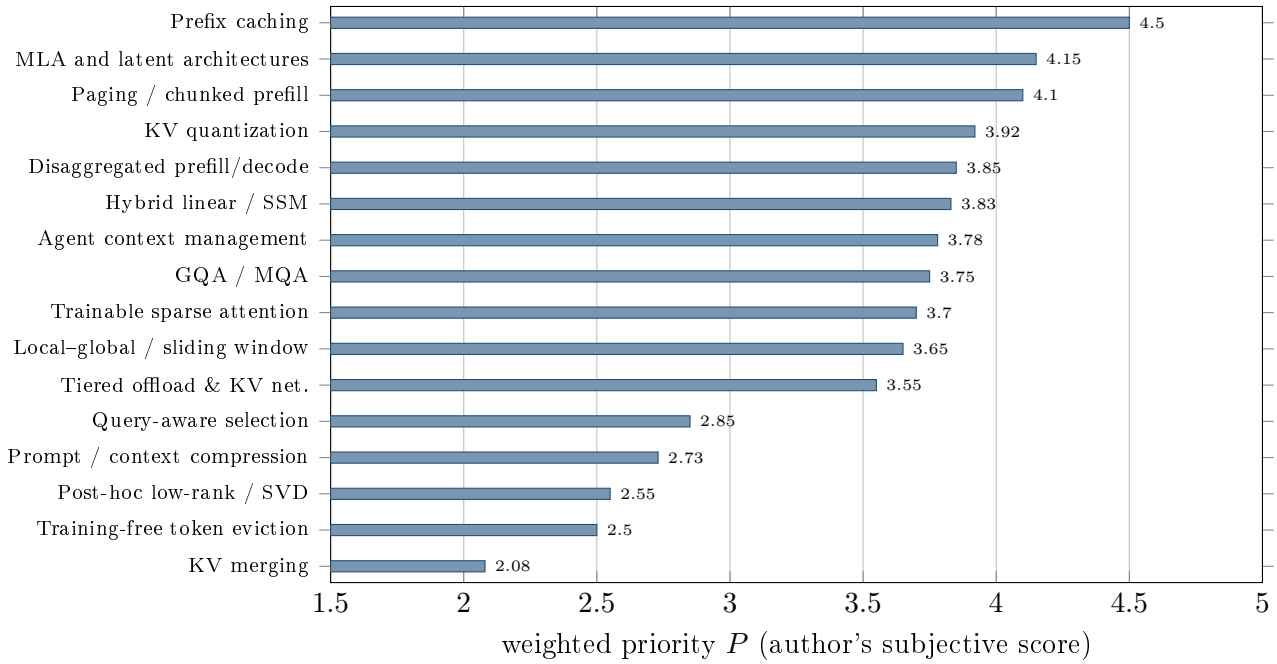


Figure 16.1: **Author's subjective scores**, not measurements: weighted priority of the sixteen sub-topics (Table 16.2). The middle of the ranking is tightly packed (3.55–3.92), so its internal order is not meaningful.

Agent context management (rank 7, low). The workload that matters most for 2026–27 products; research momentum high (folding, typed eviction); production practice is mostly LLM summarisation ("compaction"); learned methods rest on a few preprints [144, 145]. The score is held down by thin evidence and by risk (R4, R5).

GQA / MQA (rank 8, low confidence in rank, high confidence in importance). Ubiquitous and essential; the lowest marginal benefit of all (score 1 on D). Its rank is highly sensitive to weighting (ranks 2–12): second under risk-aversion, twelfth under a forward-looking weighting. We emphasise that its middling rank does not mean it is unimportant; it means little new effort is warranted.

Trainable sparse attention (rank 9, low). Highest strategic interest: DSA and V4 are in a leading open family and the cost reductions are large, and NSA/MoBA are credible. Held back by concentrated evidence (few labs), kernel and hardware coupling, and the fact that for most organisations it is a model-selection question, not a deployable technique. Its confidence is low because the rank interval is wide and the evidence base is narrow; we expect its priority to rise if independent replications appear.

Local–global / sliding window (rank 10, medium). Shipped in Gemma and gpt-oss; simple; benefits are real but moderate relative to alternatives and limited by global-layer cache.

Tiered offload and KV network compression (rank 11, medium). Increasingly used for prefix reuse (Proposition 9.5); per-step offload remains niche. Priority rises with agent

Table 16.3: Rank under alternative weightings (same subjective scores). The last column is the range of ranks across the default and four alternative weightings; a large range means the topic's priority depends on what one values.

Sub-topic	Default	Adoption	Forward	Risk-averse	Equal	Range
Prefix caching / radix reuse	1	1	2	3	1	2
MLA and latent architectures	2	4	3	4	2	2
Paging, chunked prefill, scheduling	3	2	9	1	3	8
KV quantization (cross-ref.)	4	5	7	6	5	3
Disaggregated prefill/decode	5	6	6	7	6	2
Hybrid linear / SSM architectures	6	9	1	10	4	9
Agent context management	7	8	4	11	8	7
GQA / MQA	8	3	12	2	11	10
Trainable sparse attention	9	10	5	8	7	5
Local–global / sliding window	10	7	10	5	10	5
Tiered offload & KV network compression	11	11	8	9	9	3
Query-aware selection	12	13	11	12	12	2
Prompt / context compression	13	12	15	14	14	3
Post-hoc low-rank / SVD	14	15	14	13	13	2
Training-free token eviction	15	14	13	15	15	2
KV merging	16	16	16	16	16	0

workloads and with cache-reducing architectures.

Query-aware selection (rank 12, medium). Excellent technically and an active research area, but production adoption is limited (needs full cache, kernels), and the capacity bill is untouched.

Prompt and context compression (rank 13, medium). Useful in RAG and cost-sensitive API settings; limited by fidelity risk and loss of prefix reuse.

Post-hoc low-rank / SVD compression (rank 14, medium). Principled but largely superseded in priority by MLA-style architectures and conversion; relevant to organisations committed to a fixed GQA model.

Training-free token eviction (rank 15, medium). This is the largest *research* literature in the set (by far the most papers) and the lowest in production adoption for general serving, because of silent-failure risk, prefix-sharing incompatibility, incompatibility with fused kernels and the availability of architectural alternatives. It remains useful where model choice is fixed and memory is the hard constraint (edge devices, single-GPU long documents). The ranking of this topic is the one most likely to surprise readers who track publications rather than deployments.

KV merging (rank 16, medium). Interesting as a learned, trained component (CSA/HCA compression) but post-hoc merging is niche; most robust finding of the sensitivity analysis is that it is lowest under every weighting.

Table 16.4: Final tiers (author's subjective judgment). Confidence combines evidence quality, the width of the perturbation-based rank interval and the spread across weightings.

Tier	Sub-topics (in rank order)	Confidence	Meaning
1: Essential, deploy now	prefix caching; MLA/latent architectures; paging, chunked prefill and scheduling	High (first two); Medium (third)	mature or rapidly maturing with strong evidence; start here
2: Strong, selective	KV quantization; disaggregation; GQA/MQA (baseline); local–global; tiered offload	Medium (Low for GQA's rank)	valuable where the bottleneck matches; GQA is the assumed baseline
3: High momentum, uncertain	hybrid linear/SSM; agent context management; trainable sparse attention	Low	biggest upside; outcomes depend on evidence not yet in
4: Conditional / niche	query-aware selection; prompt compression; post-hoc low-rank; training-free eviction; KV merging	Medium	use when the model is fixed and a measured constraint demands it; evaluate heavily

16.5 Final ranking in tiers, with confidence

What the ranking says

1. The *safest, highest-return* items are lossless serving techniques and the architecture-level latent cache; they do not gamble on quality.
2. The *highest-upside* items (hybrids, trainable sparse attention, agent context policies) are also the ones with the least independent evidence; treat them as active experiments with strong evaluation.
3. The *largest literature* (post-hoc eviction) is *not* the highest priority for general production; its role is niche and conditional.
4. These are judgments. A reader whose workload is mostly short-context, single-GPU or edge, or whose risk tolerance differs, should re-weight (Table 16.3) or re-score.

16.6 Adjacent topics not scored

Speculative decoding is widely deployed in production and, as Proposition 9.6 shows, is complementary to cache reductions in the cache-bound regime; we would rank it in Tier 1–2 if scored, with the same caveat that recurrent-state models complicate it. **Retrieval versus long context** is a design decision rather than a technique: the break-even analysis of Equation (10.1) depends on cache reuse and decode-time cache reads, and so it moves in favour of long context as items in Tier 1 are adopted.

Chapter 17

Open Problems and Research Directions

The preceding chapters describe a field that has made rapid practical progress while leaving basic scientific questions unresolved. We list the problems we consider most important, each phrased, where possible, as a question that experiments or theorems could settle. Expectations stated here are the author's own and are labelled as such.

17.1 Evaluation and measurement

1. **A predictive, cheap proxy for compression damage.** Attention-mass retention, logit KL, and evidence-attention ratios are cheap but are only moderately predictive of task failure (the diagnostic study found AUROCs between roughly 0.68 and 0.87 [168]). *Question:* can a proxy computed without generation predict end-task degradation across methods and models with high AUROC, and does it transfer from retrieval to reasoning tasks?

2. **Failure attribution.** In a recent diagnostic study, most compressed-run failures could not be explained by the measurements available [168]. *Question:* which additional instrumentation (per-head ablation, causal patching of retained and evicted tokens, evidence-tracking) explains the bulk of failures, and does a taxonomy of failure types emerge that differs across method families?

3. **Benchmarks that resist optimisation.** Held-out, procedurally generated, dependency-rich long-context and long-generation tasks, with private variants, to limit benchmark overfitting [36, 162, 164]. *Question:* can a benchmark be constructed whose scores for compression methods rank-correlate with production workload quality across several organisations?

4. **Standards for reporting.** Bytes per token including metadata, hardware, baseline kernel, batch size, latency percentiles, and a same-model FullCache control. A model-card field for cache bytes per token (and recurrent state bytes) would make comparisons routine.

17.2 Theory

1. **Conditional heavy hitters.** Space-Saving guarantees concern stationary aggregate importance (Proposition 11.4); retrieval needs *conditional* importance. *Question:* for a distribution over future queries with bounded information content, what is the minimal number of retained tokens (or bits) to achieve expected output error $\leq \epsilon$, as a function of the query distribution's entropy? The INDEX bound (Proposition 11.6) is the worst case; the average case under realistic query distributions is open.

2. **Contraction of reasoning dynamics.** Does the evolution of a reasoning model's cache under perturbation behave contractively ($\rho < 1$ in Proposition 11.3) or expansively, and does this depend on task type? Empirical estimates of ρ from perturbation experiments on real reasoning traces would settle the plausibility of decoding-time compression for long chains.

3. **Lower bounds for hybrids.** How many global-attention layers, as a function of the number of hops h in a retrieval task and the recurrent state size, are necessary and sufficient for a hybrid model to solve h-hop retrieval over a context of length n? Communication-complexity techniques apply directly and results would inform the attention:recurrence ratio.

4. **Learnability versus capacity in soft compression.** We argued that capacity does not limit gist-style compression at current ratios (Chapter 10); what *does* limit it, and how does the limit scale with model size and training data?

5. **Rate–distortion for attention.** Extend the Gaussian analysis (Theorem 11.2, Proposition 11.8) to the softmax nonlinearity and to the sequence axis (inter-token redundancy), giving bounds on compressions that combine quantization, low-rank and merging.

17.3 Architecture

1. **The right mix of attention and recurrence.** Open models use 1:3 to 1:8 ratios of global to recurrent layers [29, 30, 33, 34, 38], while one major lab reverted to full attention [36]. *Questions:* how do the optimal ratio and placement scale with model size, data and target context; which capabilities (multi-hop reasoning, aggregation, exact copy) degrade first as the ratio falls; can the loss be repaired by training recipes rather than layers?

2. **Combining latent, sparse and hybrid ideas.** V4 combines compression and sparse selection [7]; Kimi Linear combines recurrence and latent attention [33]. What is the Pareto frontier of cache per token versus quality when all three are combined and trained at scale, and which component pays for itself at each context length?

3. **Trainable capacity reduction with sparse selection.** DSA, NSA and MoBA reduce compute but not stored capacity [6, 73, 74]. Can a learned policy *drop* tokens permanently (not only skip them) without the silent-failure risks of post-hoc eviction, e.g., by training the model with a stochastic drop policy that it learns to tolerate?

4. **Sinks, gating and normalisation.** If sinks are an artefact of softmax (Section 6.2), do normalisation-free or gated variants [40] yield caches that tolerate eviction better, and does that persist at scale?

5. **Test-time memory at scale.** Neural memories updated at inference [142, 143] remain largely small-scale; whether they offer a better recall–throughput frontier than gated delta rules at the 10B+ scale is open.

17.4 Algorithms for eviction, selection and merging

1. **Learned eviction by reinforcement learning.** Train a small policy network to decide retention under a budget, optimising end-task reward on reasoning and agent trajectories,

rather than hand-designed scores. Is the learned policy robust across tasks and models, and does it learn to pin privileged text?

2. **Value-aware and dependency-aware scores.** Proposition 6.1 suggests scores that include value deviation $\|\mathbf{v}_i - \mathbf{o}\|$; reasoning traces suggest dependency graphs (which later steps used which earlier results). Do such scores outperform attention-only scores at equal budgets?

3. **Safe eviction.** Retention policies that are provably robust to adversarial inputs (cap the share of budget attributable to untrusted spans) and that provably retain privileged text. What guarantees are achievable in practice, and what do they cost in accuracy?

4. **Merging with provable fidelity.** Size-corrected merging is exact only for identical keys and values (Proposition 8.1); can learned merges bound the maximal logit error for *all* queries from a calibrated query distribution?

17.5 Systems

1. **Non-prefix reuse with guarantees.** CacheBlend-like methods repair cross-chunk attention approximately [99]. Is there a cheap criterion that certifies when reuse is safe, with bounded deviation from full recomputation?

2. **Unified memory management for KV pages and recurrent states.** Hybrids need both growing and fixed-size state, with checkpoints for prefix reuse and rollback for speculation. What allocator and cache-key design handles both with low overhead?

3. **Deterministic attention at low cost.** Deterministic split-KV and reproducible batch-invariant kernels with negligible slowdown, so that cache hits and misses produce identical outputs.

4. **Secure shared caches.** Formal treatment of timing and content leakage through prefix caches in multi-tenant serving, and efficient isolation mechanisms.

5. **Portable kernels for sparse and compressed attention.** Reducing hardware coupling: abstractions that compile token-sparse, block-sparse and latent-compressed attention efficiently across accelerator generations and vendors.

17.6 Joint compression

1. **End-to-end learned KV codecs** combining rotation, rank truncation, bit allocation and token selection under a rate–distortion objective, trained or calibrated per model; do they beat stacked independent methods (Proposition 8.3)?

2. **Quantization-aware eviction scoring**: when scores are computed from quantized keys, how much do rankings change, and can training or calibration reduce the sensitivity?

17.7 Long-context capability itself

1. **Effective versus claimed context.** Models accept more context than they use [156, 157, 163]. How do training-data composition and curriculum determine effective context, and can architectural reductions (hybrids, sparse attention) be *trained* to improve rather than merely preserve effective context?

2. **Mechanistic understanding of long-range retrieval in hybrids and sparse models.** Where do retrieval and induction circuits live when most layers are recurrent [187, 188]? Understanding would guide placement of global layers and head classification.

3. **Agents: memory beyond context.** Durable external state, learned folding and typed eviction suggest that the right long-horizon memory is not a bigger cache but a different memory architecture [144–146]. What are the principled interfaces between context, tools and long-term memory?

17.8 Predictions (author's expectations, not findings)

- By late 2027, cache bytes per token will appear on most open model cards, and at least one widely used open model family will ship with a hybrid or trained-sparse design with a 1M-token window as its default configuration.
- Training-free eviction will remain concentrated in edge and resource-constrained deployments rather than flagship serving, unless a method with demonstrated robustness on long-generation and multi-turn tests emerges.
- Evidence will accumulate on whether hybrid designs retain multi-hop reasoning at frontier scale; we regard this as the single most consequential open empirical question for the field.
- Context-management policies for agents will increasingly be trained jointly with the agent rather than implemented as external heuristics.

Chapter 18

Conclusions

The key–value cache is a memory of the past that a transformer builds by itself and cannot do without. Its size, $2LH_{\mathrm{kv}}d_h b$ bytes per token for standard attention, grows linearly with context and with the number of in-flight sequences, and because decode-time attention has an arithmetic intensity of only g FLOP per byte, its cost in time is proportional to its size. These two facts, derived in Chapter 2, organise everything else.

Four conclusions follow from the survey.

1. The center of gravity has moved from compressing a trained cache to training a model that needs a small one. The sequence GQA, MLA, local–global patterns, hybrids with gated delta-rule layers, trained sparse attention and V4-style compressed attention is a sequence of architectural answers, each reducing what must be stored or read, each verified by pretraining rather than by post-hoc evaluation. Open models now carry per-token caches of roughly 8–25 KiB (derived from their public configurations), one to two orders of magnitude below a dense MHA design, and ship with million-token windows. Post-hoc methods remain useful where the model cannot be changed, but their evidence base is weaker where it matters most: long generation, multi-turn reuse, adversarial and agentic settings.

2. The lossless serving layer is the stable foundation. Paged memory, prefix caching, chunked prefill, disaggregation and tiered storage preserve the model's function, compose with every architectural reduction, and become *more* attractive as per-token caches shrink (cheaper to transfer, cheaper to load than to recompute). They are the safest investments.

3. Theory says what is impossible, and therefore where to look. Worst-case sub-quadratic attention is blocked by fine-grained complexity; query-agnostic compression of n arbitrary facts into $o(n)$ bits is blocked by communication complexity; fixed-state models cannot copy long strings. None of this prevents practical compression, because real text is highly compressible and real attention is structured; but it does mean that every compressor embodies a prior over future questions, and that failures will occur exactly when the question falls outside the prior. The quality of the prior, not the cleverness of the kernel, is what separates robust methods from brittle ones.

4. Evaluation, not asymptotics, is the binding constraint. Needle-in-a-haystack saturates; aggregation, multi-hop, low-overlap retrieval, multi-turn and long-generation tasks reveal damage that standard suites miss; and in recent diagnostic work most failures of compressed runs could not be explained by available instrumentation. The most credible account of a team reverting from a hybrid to full attention turns on evaluation: the hybrid looked equivalent on standard suites and deficient on complex multi-hop reasoning at scale.

What to do. For practitioners: adopt the lossless serving stack; choose models whose cache per token fits your workload; quantize the cache before evicting it; use query-agnostic or conservative policies when contexts are shared or generations are long; pin privileged text; monitor with canaries; keep a rollback path. For researchers: build evaluations that resist optimisation, study hybrids at frontier scale for multi-hop reasoning, develop learned and safe retention policies, and extend the theory from worst-case limits to query-distribution-dependent bounds.

The priority estimate of Chapter 16 is one reader's judgment, and it will date quickly: the field moved from MQA to million-token hybrids in seven years, and several of the entries in the timeline are less than a year old. The framework that will outlast particular methods is the cost model: cache bytes, read bytes, arithmetic intensity, load-versus-recompute, and the information floor set by the text itself.

Appendix A

Notation

Table A.1: Symbols used throughout.

Symbol	Meaning
n	sequence (context) length in tokens; n_p prefix length
B	batch size (number of concurrent sequences)
L	number of transformer layers; L_g, L_l global and local layers
d	model (hidden) width
H_q, H_{kv}	number of query heads, number of key–value heads
$g = H_q/H_{\text{kv}}$	group size: query heads per KV head
d_h	head dimension
b	bytes per stored element (2 for bf16/fp16); b_q bits of a quantized element; b_{eff} effective bits including metadata
c_ℓ	cached elements per token in layer ℓ (e.g., $2H_{\text{kv}}d_h$)
S_{KV}, $S_{\text{KV}}^{(1)}$	cache bytes (batch; one sequence)
W	bytes of weights read per decode step
β	memory bandwidth (bytes/s); P peak compute (FLOP/s); $I^\star = P/\beta$ ridge point
I	arithmetic intensity (FLOP/byte)
$\mathbf{q}, \mathbf{k}, \mathbf{v}, \mathbf{o}$	query, key, value, output vectors
$\mathbf{K}, \mathbf{V}, \mathbf{Q}$	stacked keys, values, queries
$\mathbf{h}$	hidden state of one token; $\mathbf{X}$ stacked hidden states
p_s	softmax attention weight on token s; $\mathbf{p}$ the weight vector
ϵ	dropped attention mass (eviction), ϵ_k outside top-k
$\mathcal{S}, \mathcal{E}$	retained set, evicted set of tokens
k	budget: number of retained or selected tokens (context-dependent)
w	window size (sliding window)
s_0	number of attention-sink tokens retained
$\tau = 1/\sqrt{d_h}$	logit scale
s_i	attention logit $\mathbf{q}^\top \mathbf{k}_i/\sqrt{d_h}$
η	maximum absolute logit perturbation
V	bound on value norms
TV	total variation distance
d_c, d_h^R	MLA latent width and decoupled rotary width
$\mathbf{W}^{DKV}, \mathbf{W}^{UK}, \mathbf{W}^{UV}$	MLA down- and up-projections
$\mathbf{c}_t$	cached latent vector of token t
$\mathbf{S}_t$	recurrent (matrix) state of a linear-attention layer

Symbol	Meaning
α_t, β_t	decay gate, write strength (delta rule)
d_k, d_v	key and value widths of a linear-attention state
H^I, $\mathbf{k}_s^I$, $I_{t,s}$	indexer heads, indexer key, indexer score (DSA)
ρ	fraction of cache read per step (selection) or retention ratio (context-dependent)
η (cost model)	fraction of baseline cache removed; context makes clear
α	power-law exponent of attention weights; acceptance probability in speculative decoding
γ	number of speculative draft tokens
σ_j, λ_j	singular values; eigenvalues (variances)
Σ, Σ_q	second-moment matrices of hidden states, queries
$R(D)$	rate–distortion function
μ	achieved hardware utilisation (Proposition 9.5)
P_{act}	active parameters of the model

Appendix B

Additional Derivations and Worked Examples

B.1 A numerical check of single-token removal

Take three tokens with attention weights $\mathbf{p} = (0.5, 0.3, 0.2)$ and values $\mathbf{v}_1 = (1, 0)$, $\mathbf{v}_2 = (0, 1)$, $\mathbf{v}_3 = (-1, -1)$. Then

$$\mathbf{o} = 0.5(1, 0) + 0.3(0, 1) + 0.2(-1, -1) = (0.3,\ 0.1).$$

Remove token 2 ($p_2 = 0.3$) and renormalise: $\mathbf{o}' = \big(0.5(1, 0) + 0.2(-1, -1)\big)/0.7 = (0.3, -0.2)/0.7 = (0.4286, -0.2857)$. Proposition 6.1 predicts $\mathbf{o}' - \mathbf{o} = \frac{p_2}{1-p_2}(\mathbf{o} - \mathbf{v}_2) = 0.4286 \cdot (0.3, -0.9) = (0.1286, -0.3857)$, and indeed $\mathbf{o} + (0.1286, -0.3857) = (0.4286, -0.2857)$. The value of the evicted token, $(0, 1)$, is far from the mean output; had its value equalled $\mathbf{o}$, removal would have changed nothing despite the same weight 0.3.

B.2 The sink logit equals a zero-value key

Let a head have logits $s_1, \ldots, s_n$ over real tokens and a learned scalar sink logit z. The sink-augmented softmax is $p_i = e^{s_i}/(e^z + \sum_j e^{s_j})$ for real tokens (Equation (5.2)). Define an extra key $\mathbf{k}_0$ whose logit with the current query is z and an extra value $\mathbf{v}_0 = \mathbf{0}$. The ordinary attention output over $n + 1$ entries is

$$\mathbf{o} = \frac{e^z \cdot \mathbf{0} + \sum_i e^{s_i}\mathbf{v}_i}{e^z + \sum_j e^{s_j}} = \sum_i p_i \mathbf{v}_i,$$

identical to the sink-augmented formula. Hence a learned sink logit is equivalent to a token with zero value whose logit is learned directly (independent of the query if the key is constructed to give a constant logit). The head may now output a near-zero vector by setting $z \gg s_i$, which in plain softmax attention requires attending to some real token with a harmless value.

B.3 Decode-time latency: the batch crossover

From Equation (2.8), $t_{\text{step}}(B) = (W + BS^{(1)})/\beta$. The per-token latency is $t_{\text{step}}/B = S^{(1)}/\beta + W/(B\beta)$, decreasing in B and converging to $S^{(1)}/\beta$. The two terms are equal at $B^\star = W/S^{(1)}$. For Llama-3-8B-class numbers ($W = 16.06$ GB):

Context n	$S^{(1)}$ (GB)	$B^\star$	Asymptotic per-token latency (ms, 3.35 TB/s)
4,096	0.537	29.9	0.16
32,768	4.29	3.7	1.28
131,072	17.2	0.93	5.13

(derived; compute and communication time ignored). At 128K the asymptote exceeds the batch-1 weight-read time $W/\beta = 4.8$ ms: even a single sequence is cache-dominated.

B.4 Chunkwise-parallel form of a gated linear recurrence

Consider scalar-gated linear attention, $\mathbf{S}_t = \gamma_t \mathbf{S}_{t-1} + \mathbf{v}_t \mathbf{k}_t^\top$ and $\mathbf{o}_t = \mathbf{S}_t \mathbf{q}_t$. Unrolling, $\mathbf{o}_t = \sum_{s \le t} \big(\prod_{u=s+1}^{t} \gamma_u\big) \mathbf{v}_s (\mathbf{k}_s^\top \mathbf{q}_t)$. Split the sequence into chunks of length C. Let $\Gamma_t = \prod_{u \le t} \gamma_u$ (cumulative decay from the sequence start; for numerical stability computed in log space within chunks) and index positions within a chunk c by $i = 1, \dots, C$ with chunk-start state $\mathbf{S}_{[c]}$. Then for t in chunk c:

$$\mathbf{o}_t = \underbrace{\frac{\Gamma_t}{\Gamma_{[c]}} \mathbf{S}_{[c]} \mathbf{q}_t}_{\text{inter-chunk: read carried state}} + \underbrace{\sum_{s \in c,\, s \le t} \frac{\Gamma_t}{\Gamma_s} \mathbf{v}_s (\mathbf{k}_s^\top \mathbf{q}_t)}_{\text{intra-chunk: small masked matmul}} ,$$

and the carried state is updated per chunk by $\mathbf{S}_{[c+1]} = \frac{\Gamma_{[c+1]}}{\Gamma_{[c]}} \mathbf{S}_{[c]} + \sum_{s \in c} \frac{\Gamma_{[c+1]}}{\Gamma_s} \mathbf{v}_s \mathbf{k}_s^\top$. The intra-chunk term is a $C \times C$ masked product (dense matrix multiplication, tensor-core friendly), and the inter-chunk term is a recurrence with n/C steps. Total cost is $\mathcal{O}(nCd + nd^2)$ rather than $\mathcal{O}(nd^2)$ for pure recurrence, but with far better hardware utilisation; this is the structure exploited in [19, 26, 27].

B.5 Why MLA absorption preserves the output exactly

With up-projections $\mathbf{W}_i^{UK} \in \mathbb{R}^{d_h \times d_c}$ and $\mathbf{W}_i^{UV} \in \mathbb{R}^{d_h \times d_c}$, query $\mathbf{q}_{t,i}^C \in \mathbb{R}^{d_h}$, and latent $\mathbf{c}_s \in \mathbb{R}^{d_c}$: the content logit is $\mathbf{q}_{t,i}^{C\top} \mathbf{W}_i^{UK} \mathbf{c}_s = (\mathbf{W}_i^{UK\top} \mathbf{q}_{t,i}^C)^\top \mathbf{c}_s$ because $\mathbf{a}^\top \mathbf{M} \mathbf{c} = (\mathbf{M}^\top \mathbf{a})^\top \mathbf{c}$ for any conformable $\mathbf{a}, \mathbf{M}, \mathbf{c}$. The attention output is $\sum_s p_s \mathbf{W}_i^{UV} \mathbf{c}_s = \mathbf{W}_i^{UV} \sum_s p_s \mathbf{c}_s$ by linearity, and the output projection $\mathbf{W}_i^O$ multiplies this: $\mathbf{W}_i^O \mathbf{W}_i^{UV}$ is a single precomputable matrix. FLOP count per decoded token per layer, with n cached tokens: scores $2H_q n(d_c + d_h^R)$, weighted latent sum $2H_q n d_c$, total $2H_q n(2d_c + d_h^R) = 2 \cdot 128 \cdot 1088\, n \approx 2.79 \times 10^5\, n$; bytes $nb(d_c + d_h^R) = 1152\, n$.

B.6 Why the expected-mass scoring rule is optimal for additive loss

Suppose the loss of evicting set $\mathcal{E}$ for a random future query $\mathbf{q}$ is the dropped mass $\epsilon(\mathcal{E}; \mathbf{q}) = \sum_{s \in \mathcal{E}} p_s(\mathbf{q})$ and the budget fixes $\mathbf{a}s\mathcal{E} = n - k$. Then $\mathbb{E}_{\mathbf{q}} \epsilon(\mathcal{E}; \mathbf{q}) = \sum_{s \in \mathcal{E}} \bar{p}_s$ with $\bar{p}_s = \mathbb{E}_{\mathbf{q}} p_s(\mathbf{q})$, minimised by evicting the $n - k$ tokens of smallest $\bar{p}_s$ (and keeping the k largest). Every score in Chapter 6 is an estimator of $\bar{p}_s$ under a different assumption about $\mathbf{q}$'s distribution. Note this rule is optimal for expected *dropped mass*, which only upper-bounds output error (Proposition 11.1); under the exact single-removal formula (Proposition 6.1) the optimal score also carries the factor $\|\mathbf{v}_s - \mathbf{o}\|$.

B.7 Break-even of load versus recompute for several models

With illustrative $P = 10^{15}$ FLOP/s, $\mu = 0.5$ and a 50 GB/s link, per-token recompute cost is $2P_{\text{act}}/(\mu P)$ and per-token load cost is $c_{\text{tok}}/\beta_{\text{tier}}$:

Model	P_{act}	KiB/tok	Recompute (μs/tok)	Load (μs/tok)
Llama-3-8B-class	8 B	128	32	2.6
Llama-3-70B-class	70 B	320	280	6.6
DeepSeek-V3-class (MLA)	37 B active	68.6	148	1.4

(derived and illustrative; DeepSeek-V3's active parameter count of about 37B is from [5]; attention FLOPs ignored). In all three cases loading from a fast tier is one to two orders of magnitude cheaper than recomputation; from a 5 GB/s NVMe drive the advantage falls by ten.

B.8 Expected tokens per speculative step

With i.i.d. acceptance probability $\alpha < 1$ per drafted token and γ drafted tokens, the number N of tokens produced per target pass (accepted drafts plus the one token sampled from the target at the first rejection or after all accepted) satisfies $\Pr[N \geq j] = \alpha^{j-1}$ for $j = 1, \ldots, \gamma+1$. Hence $\mathbb{E}N = \sum_{j=1}^{\gamma+1} \alpha^{j-1} = \frac{1-\alpha^{\gamma+1}}{1-\alpha}$, as used in Proposition 9.6 and [124]. For $\alpha = 0.8$: $\gamma = 2$ gives 2.44, $\gamma = 4$ gives 3.36, $\gamma = 8$ gives 4.50; diminishing returns set in because $\mathbb{E}N \to 1/(1-\alpha) = 5$ as $\gamma \to \infty$ (derived).

Appendix C

Glossary

Absorption (MLA)
: Folding the key and value up-projections into the query and output projections so that attention runs directly on the cached latent.

Arithmetic intensity
: FLOPs performed per byte moved from memory; decides whether a kernel is compute- or memory-bound.

Attention sink
: A token (often the first) that receives a large share of attention regardless of content, acting as a place to dump surplus softmax mass.

Block table
: Per-sequence map from logical cache blocks to physical memory blocks in paged attention.

Cache-bound regime
: The regime in which decode time is dominated by reading the KV cache rather than the weights.

Chunked prefill
: Splitting a long prompt into chunks processed across several iterations to avoid stalling decodes.

CLA
: Cross-layer attention: sharing keys and values between adjacent layers.

Context folding
: An agent policy of branching into a subtask and replacing its steps by a short result upon return.

CSA / HCA
: Compressed Sparse Attention and Heavily Compressed Attention, the two layer types of DeepSeek-V4.

Decoupled RoPE
: The MLA design that adds a small separate rotary key so that content keys need not be rotated.

Delta rule
: A recurrent update that corrects the current prediction before writing a new association (online regression).

Disaggregation
: Running prefill and decode on separate machine pools and transferring the cache between them.

DSA
: DeepSeek Sparse Attention: token-level top-k selection driven by a lightning indexer.

Eviction
: Permanently removing cache entries by a retention policy.

FlashAttention
: IO-aware exact attention using tiling and online softmax.

GQA / MQA
: Grouped-query / multi-query attention: several query heads share one key–value head.

Gist tokens Learned tokens whose activations summarise a preceding prompt, enabling prompt compression.

Heavy hitter A token with high accumulated attention; also a frequent item in streaming algorithms.

Hybrid model A model that mixes full-attention layers with recurrent, state-space or linear-attention layers.

Lightning indexer The cheap, low-precision scorer in DSA that ranks previous tokens for each query.

Linear attention Attention with a feature-map kernel such that keys and values are summarised in a constant-size state.

MLA Multi-head latent attention: joint low-rank compression of keys and values into a cached latent.

Miss mass Attention mass lying outside the retained or selected set.

MoBA Mixture of block attention: queries route to the top-k blocks by mean-pooled keys.

NSA Native sparse attention: compression, selection and window branches with gating, trained natively.

Needle in a haystack A single-fact retrieval test at varying depth and context length.

Online softmax Streaming computation of softmax with a running maximum and normaliser; basis of fused attention.

PagedAttention Block-based KV memory management with a block table.

Prefill / decode Processing the prompt in parallel / generating tokens one at a time.

Prefix caching Reusing the cache of an identical token prefix across requests.

Query-aware / query-agnostic Whether a compression method uses the question when deciding what to keep.

Radix tree Tree of token sequences used to index cached prefixes (SGLang).

RoPE Rotary position embedding.

Roofline Performance model bounding throughput by compute peak and memory bandwidth times intensity.

Selection Choosing a subset of cache entries to read at a step without deleting the rest.

Sliding-window attention Attention restricted to the most recent w tokens.

Speculative decoding
: Draft-and-verify decoding that produces several tokens per target pass.

Split-KV
: Parallelising decode attention across chunks of the cache with an exact merge.

State-space model (SSM)
: A recurrent layer with a (selective) linear state update, e.g., Mamba.

StreamingLLM
: Eviction keeping sink tokens and a recent window to stabilise unbounded streams.

TTFT / TPOT
: Time to first token / time per output token.

YOCO
: You Only Cache Once: a decoder–decoder architecture with a single global cache.

Bibliography

[1] A. Vaswani et al. Attention is all you need. NeurIPS 2017. arXiv:1706.03762.

[2] N. Shazeer. Fast transformer decoding: one write-head is all you need. 2019. arXiv:1911.02150.

[3] J. Ainslie et al. GQA: training generalized multi-query transformer models from multi-head checkpoints. EMNLP 2023. arXiv:2305.13245.

[4] DeepSeek-AI. DeepSeek-V2: a strong, economical, and efficient mixture-of-experts language model. 2024. arXiv:2405.04434.

[5] DeepSeek-AI. DeepSeek-V3 technical report. 2024. arXiv:2412.19437.

[6] DeepSeek-AI. DeepSeek-V3.2: pushing the frontier of open large language models. 2025. arXiv:2512.02556.

[7] DeepSeek-AI. DeepSeek-V4: towards highly efficient million-token context intelligence. 2026. arXiv:2606.19348.

[8] DeepSeek-AI. DeepSeek-R1: incentivizing reasoning capability in LLMs via reinforcement learning. 2025. arXiv:2501.12948.

[9] W. Brandon et al. Reducing transformer key-value cache size with cross-layer attention. NeurIPS 2024. arXiv:2405.12981.

[10] Y. Sun et al. You only cache once: decoder-decoder architectures for language models. NeurIPS 2024. arXiv:2405.05254.

[11] I. Beltagy, M. Peters, A. Cohan. Longformer: the long-document transformer. 2020. arXiv:2004.05150.

[12] R. Child, S. Gray, A. Radford, I. Sutskever. Generating long sequences with sparse transformers. 2019. arXiv:1904.10509.

[13] M. Zaheer et al. Big Bird: transformers for longer sequences. NeurIPS 2020. arXiv:2007.14062.

[14] A. Jiang et al. Mistral 7B. 2023. arXiv:2310.06825.

[15] Gemma Team. Gemma 2: improving open language models at a practical size. 2024. arXiv:2408.00118.

[16] Gemma Team. Gemma 3 technical report. 2025. arXiv:2503.19786.

[17] OpenAI. gpt-oss-120b and gpt-oss-20b model card. 2025. arXiv:2508.10925.

[18] A. Gu, T. Dao. Mamba: linear-time sequence modeling with selective state spaces. 2023. arXiv:2312.00752.

[19] T. Dao, A. Gu. Transformers are SSMs: generalized models and efficient algorithms through structured state space duality. ICML 2024. arXiv:2405.21060.

[20] A. Gu, K. Goel, C. Ré. Efficiently modeling long sequences with structured state spaces. ICLR 2022. arXiv:2111.00396.

[21] A. Katharopoulos et al. Transformers are RNNs: fast autoregressive transformers with linear attention. ICML 2020. arXiv:2006.16236.

[22] I. Schlag, K. Irie, J. Schmidhuber. Linear transformers are secretly fast weight programmers. ICML 2021. arXiv:2102.11174.

[23] B. Peng et al. RWKV: reinventing RNNs for the transformer era. Findings of EMNLP 2023. arXiv:2305.13048.

[24] B. Peng et al. Eagle and Finch: RWKV with matrix-valued states and dynamic recurrence. 2024. arXiv:2404.05892.

[25] Y. Sun et al. Retentive network: a successor to transformer for large language models. 2023. arXiv:2307.08621.

[26] S. Yang et al. Gated linear attention transformers with hardware-efficient training. ICML 2024. arXiv:2312.06635.

[27] S. Yang et al. Parallelizing linear transformers with the delta rule over sequence length. NeurIPS 2024. arXiv:2406.06484.

[28] S. Yang, J. Kautz, A. Hatamizadeh. Gated delta networks: improving Mamba2 with delta rule. ICLR 2025. arXiv:2412.06464.

[29] O. Lieber et al. Jamba: a hybrid transformer-mamba language model. 2024. arXiv:2403.19887.

[30] R. Waleffe et al. An empirical study of Mamba-based language models. 2024. arXiv:2406.07887.

[31] NVIDIA. Nemotron-H: a family of accurate and efficient hybrid Mamba-Transformer models. 2025. arXiv:2504.03624.

[32] NVIDIA. Nemotron 3 Nano: open, efficient mixture-of-experts hybrid Mamba-Transformer model for agentic reasoning. 2025. arXiv:2512.20848.

[33] Kimi Team. Kimi Linear: an expressive, efficient attention architecture. 2025. arXiv:2510.26692.

[34] MiniMax. MiniMax-01: scaling foundation models with lightning attention. 2025. arXiv:2501.08313.

[35] Z. Qin et al. Lightning attention-2: a free lunch for handling unlimited sequence lengths in large language models. 2024. arXiv:2401.04658.

[36] MiniMax. Why did M2 end up as a full attention model? Hugging Face blog, 2025. `https://huggingface.co/blog/MiniMax-AI/why-did-m2-end-up-as-a-full-attention-model`.

[37] A. Yang et al. (Qwen Team). Qwen3 technical report. 2025. arXiv:2505.09388.

[38] Qwen Team. Qwen3-Next-80B-A3B-Instruct model configuration. Hugging Face model repository, accessed Oct. 2026.

[39] Moonshot AI. Kimi-Linear-48B-A3B-Instruct model configuration. Hugging Face model repository, accessed Oct. 2026.

[40] Z. Qiu et al. Gated attention for large language models: non-linearity, sparsity, and attention-sink-free. 2025. arXiv:2505.06708.

[41] A. Yang et al. Qwen2.5-1M technical report. 2025. arXiv:2501.15383.

[42] A. Grattafiori et al. (Meta). The Llama 3 herd of models. 2024. arXiv:2407.21783.

[43] Gemini Team. Gemini 1.5: unlocking multimodal understanding across millions of tokens of context. 2024. arXiv:2403.05530.

[44] Kimi Team. Kimi K2: open agentic intelligence. 2025. arXiv:2507.20534.

[45] GLM-4.5 Team. GLM-4.5: agentic, reasoning, and coding (ARC) foundation models. 2025. arXiv:2508.06471.

[46] G. Xiao et al. Efficient streaming language models with attention sinks. ICLR 2024. arXiv:2309.17453.

[47] Z. Zhang et al. H_2O: heavy-hitter oracle for efficient generative inference of large language models. NeurIPS 2023. arXiv:2306.14048.

[48] Z. Liu et al. Scissorhands: exploiting the persistence of importance hypothesis for LLM KV cache compression at test time. NeurIPS 2023. arXiv:2305.17118.

[49] Y. Li et al. SnapKV: LLM knows what you are looking for before generation. NeurIPS 2024. arXiv:2404.14469.

[50] Z. Cai et al. PyramidKV: dynamic KV cache compression based on pyramidal information funneling. 2024. arXiv:2406.02069.

[51] D. Yang et al. PyramidInfer: pyramid KV cache compression for high-throughput LLM inference. Findings of ACL 2024. arXiv:2405.12532.

[52] S. Ge et al. Model tells you what to discard: adaptive KV cache compression for LLMs. ICLR 2024. arXiv:2310.01801.

[53] Y. Feng et al. Ada-KV: optimizing KV cache eviction by adaptive budget allocation for efficient LLM inference. 2024. arXiv:2407.11550.

[54] Y. Fu et al. Not all heads matter: a head-level KV cache compression method with integrated retrieval and reasoning. 2024. arXiv:2410.19258.

[55] H. Tang et al. RazorAttention: efficient KV cache compression through retrieval heads. 2024.

arXiv:2407.15891.

[56] M. Oren et al. Transformers are multi-state RNNs. EMNLP 2024. arXiv:2401.06104.

[57] M. Adnan et al. Keyformer: KV cache reduction through key tokens selection for efficient generative inference. MLSys 2024. arXiv:2403.09054.

[58] A. Devoto et al. A simple and effective L_2 norm-based strategy for KV cache compression. EMNLP 2024. arXiv:2406.11430.

[59] S. Anagnostidis et al. Dynamic context pruning for efficient and interpretable autoregressive transformers. NeurIPS 2023. arXiv:2305.15805.

[60] P. Nawrot et al. Dynamic memory compression: retrofitting LLMs for accelerated inference. ICML 2024. arXiv:2403.09636.

[61] J. Tang et al. Quest: query-aware sparsity for efficient long-context LLM inference. ICML 2024. arXiv:2406.10774.

[62] L. Ribar et al. SparQ attention: bandwidth-efficient LLM inference. ICML 2024. arXiv:2312.04985.

[63] C. Xiao et al. InfLLM: training-free long-context extrapolation for LLMs with an efficient context memory. NeurIPS 2024. arXiv:2402.04617.

[64] W. Lee et al. InfiniGen: efficient generative inference of large language models with dynamic KV cache management. OSDI 2024. arXiv:2406.19707.

[65] D. Liu et al. RetrievalAttention: accelerating long-context LLM inference via vector retrieval. 2024. arXiv:2409.10516.

[66] Z. Chen et al. MagicPIG: LSH sampling for efficient LLM generation. ICLR 2025. arXiv:2410.16179.

[67] H. Sun et al. ShadowKV: KV cache in shadows for high-throughput long-context LLM inference. ICML 2025. arXiv:2410.21465.

[68] G. Xiao et al. DuoAttention: efficient long-context LLM inference with retrieval and streaming heads. ICLR 2025. arXiv:2410.10819.

[69] J.-H. Kim et al. KVzip: query-agnostic KV cache compression with context reconstruction. NeurIPS 2025. arXiv:2505.23416.

[70] Z. Cai et al. R-KV: redundancy-aware KV cache compression for reasoning models. 2025. arXiv:2505.24133.

[71] W. Liu et al. ReasonAlloc: hierarchical decoding-time KV cache budget allocation for reasoning models. 2026. arXiv:2606.11164.

[72] ThinKV: thought-adaptive KV cache compression for efficient reasoning models. 2025. arXiv:2510.01290.

[73] J. Yuan et al. Native sparse attention: hardware-aligned and natively trainable sparse attention. ACL 2025. arXiv:2502.11089.

[74] E. Lu et al. MoBA: mixture of block attention for long-context LLMs. 2025. arXiv:2502.13189.

[75] Y. Gao et al. SeerAttention: learning intrinsic sparse attention in your LLMs. 2024. arXiv:2410.13276.

[76] H. Jiang et al. MInference 1.0: accelerating pre-filling for long-context LLMs via dynamic sparse attention. NeurIPS 2024. arXiv:2407.02490.

[77] X. Lai et al. FlexPrefill: a context-aware sparse attention mechanism for efficient long-sequence inference. ICLR 2025. arXiv:2502.20766.

[78] R. Xu et al. XAttention: block sparse attention with antidiagonal scoring. ICML 2025. arXiv:2503.16428.

[79] S. Yang et al. LServe: efficient long-sequence LLM serving with unified sparse attention. MLSys 2025. arXiv:2502.14866.

[80] C.-C. Chang et al. Palu: compressing KV-cache with low-rank projection. ICLR 2025. arXiv:2407.21118.

[81] T. Ji et al. Towards economical inference: enabling DeepSeek's multi-head latent attention in any transformer-based LLMs. 2025. arXiv:2502.14837.

[82] F. Meng et al. TransMLA: multi-head latent attention is all you need. 2025. arXiv:2502.07864.
[83] Y. Zhang et al. Tensor product attention is all you need. 2025. arXiv:2501.06425.
[84] T. Zadouri et al. Hardware-efficient attention for fast decoding. 2025. arXiv:2505.21487.
[85] Z. Wang et al. Model tells you where to merge: adaptive KV cache merging for LLMs on long-context tasks. 2024. arXiv:2407.08454.
[86] A. Liu et al. MiniCache: KV cache compression in depth dimension for large language models. NeurIPS 2024. arXiv:2405.14366.
[87] D. Bolya et al. Token merging: your ViT but faster. ICLR 2023. arXiv:2210.09461.
[88] M. Zhang et al. LoLCATs: on low-rank linearizing of large language models. 2024. arXiv:2410.10254.
[89] J. Wang et al. The Mamba in the Llama: distilling and accelerating hybrid models. NeurIPS 2024. arXiv:2408.15237.
[90] J. Mercat et al. Linearizing large language models. 2024. arXiv:2405.06640.
[91] C. Hooper et al. KVQuant: towards 10 million context length LLM inference with KV cache quantization. NeurIPS 2024. arXiv:2401.18079.
[92] Z. Liu et al. KIVI: a tuning-free asymmetric 2bit quantization for KV cache. ICML 2024. arXiv:2402.02750.
[93] A. Zandieh, M. Daliri, I. Han. QJL: 1-bit quantized JL transform for KV cache quantization with zero overhead. 2024. arXiv:2406.03482.
[94] A. Zandieh et al. TurboQuant: online vector quantization with near-optimal distortion rate. 2025. arXiv:2504.19874.
[95] W. Kwon et al. Efficient memory management for large language model serving with PagedAttention. SOSP 2023. arXiv:2309.06180.
[96] R. Prabhu et al. vAttention: dynamic memory management for serving LLMs without PagedAttention. ASPLOS 2025. arXiv:2405.04437.
[97] L. Zheng et al. SGLang: efficient execution of structured language model programs. NeurIPS 2024. arXiv:2312.07104.
[98] I. Gim et al. Prompt Cache: modular attention reuse for low-latency inference. MLSys 2024. arXiv:2311.04934.
[99] J. Yao et al. CacheBlend: fast large language model serving for RAG with cached knowledge fusion. EuroSys 2025. arXiv:2405.16444.
[100] J. Juravsky et al. Hydragen: high-throughput LLM inference with shared prefixes. 2024. arXiv:2402.05099.
[101] L. Ye et al. ChunkAttention: efficient self-attention with prefix-aware KV cache and two-phase partition. ACL 2024. arXiv:2402.15220.
[102] A. Agrawal et al. Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve. OSDI 2024. arXiv:2403.02310.
[103] Y. Zhong et al. DistServe: disaggregating prefill and decoding for goodput-optimized large language model serving. OSDI 2024. arXiv:2401.09670.
[104] P. Patel et al. Splitwise: efficient generative LLM inference using phase splitting. ISCA 2024. arXiv:2311.18677.
[105] R. Qin et al. Mooncake: a KVCache-centric disaggregated architecture for LLM serving. FAST 2025. arXiv:2407.00079.
[106] Y. Liu et al. CacheGen: KV cache compression and streaming for fast large language model serving. SIGCOMM 2024. arXiv:2310.07240.
[107] Y. Liu et al. LMCache: an efficient KV cache layer for enterprise-scale LLM inference. 2025. arXiv:2510.09665.
[108] C. Jin et al. RAGCache: efficient knowledge caching for retrieval-augmented generation. 2024. arXiv:2404.12457.
[109] Y. Sheng et al. FlexGen: high-throughput generative inference of large language models with a

single GPU. ICML 2023. arXiv:2303.06865.

[110] T. Dao et al. FlashAttention: fast and memory-efficient exact attention with IO-awareness. NeurIPS 2022. arXiv:2205.14135.

[111] T. Dao. FlashAttention-2: faster attention with better parallelism and work partitioning. ICLR 2024. arXiv:2307.08691.

[112] J. Shah et al. FlashAttention-3: fast and accurate attention with asynchrony and low-precision. NeurIPS 2024. arXiv:2407.08608.

[113] T. Dao, D. Haziza, F. Massa, G. Sizov. Flash-Decoding for long-context inference. PyTorch blog, Oct. 2023.

[114] K. Hong et al. FlashDecoding++: faster large language model inference on GPUs. MLSys 2024. arXiv:2311.01282.

[115] Z. Ye et al. FlashInfer: efficient and customizable attention engine for LLM inference serving. MLSys 2025. arXiv:2501.01005.

[116] M. Milakov, N. Gimelshein. Online normalizer calculation for softmax. 2018. arXiv:1805.02867.

[117] M. Rabe, C. Staats. Self-attention does not need $O(n^2)$ memory. 2021. arXiv:2112.05682.

[118] H. Liu, M. Zaharia, P. Abbeel. Ring attention with blockwise transformers for near-infinite context. 2023. arXiv:2310.01889.

[119] S. Jacobs et al. DeepSpeed Ulysses: system optimizations for enabling training of extreme long sequence transformer models. 2023. arXiv:2309.14509.

[120] R. Pope et al. Efficiently scaling transformer inference. MLSys 2023. arXiv:2211.05102.

[121] S. Williams, A. Waterman, D. Patterson. Roofline: an insightful visual performance model for multicore architectures. Commun. ACM 52(4), 2009.

[122] Z. Yuan et al. LLM inference unveiled: survey and roofline model insights. 2024. arXiv:2402.16363.

[123] NVIDIA. H100 Tensor Core GPU datasheet. Vendor documentation (peak figures quoted approximately).

[124] Y. Leviathan, M. Kalman, Y. Matias. Fast inference from transformers via speculative decoding. ICML 2023. arXiv:2211.17192.

[125] C. Chen et al. Accelerating large language model decoding with speculative sampling. 2023. arXiv:2302.01318.

[126] T. Cai et al. Medusa: simple LLM inference acceleration framework with multiple decoding heads. ICML 2024. arXiv:2401.10774.

[127] Y. Li et al. EAGLE: speculative sampling requires rethinking feature uncertainty. ICML 2024. arXiv:2401.15077.

[128] X. Miao et al. SpecInfer: accelerating generative LLM serving with tree-based speculative inference and verification. ASPLOS 2024. arXiv:2305.09781.

[129] H. Jiang et al. LLMLingua: compressing prompts for accelerated inference of large language models. EMNLP 2023. arXiv:2310.05736.

[130] H. Jiang et al. LongLLMLingua: accelerating and enhancing LLMs in long context scenarios via prompt compression. ACL 2024. arXiv:2310.06839.

[131] Z. Pan et al. LLMLingua-2: data distillation for efficient and faithful task-agnostic prompt compression. Findings of ACL 2024. arXiv:2403.12968.

[132] J. Mu, X. Li, N. Goodman. Learning to compress prompts with gist tokens. NeurIPS 2023. arXiv:2304.08467.

[133] A. Chevalier et al. Adapting language models to compress contexts. EMNLP 2023. arXiv:2305.14788.

[134] T. Ge et al. In-context autoencoder for context compression in a large language model. ICLR 2024. arXiv:2307.06945.

[135] B. Lester, R. Al-Rfou, N. Constant. The power of scale for parameter-efficient prompt tuning. EMNLP 2021. arXiv:2104.08691.

[136] X. Li, P. Liang. Prefix-tuning: optimizing continuous prompts for generation. ACL 2021.

arXiv:2101.00190.

[137] A. Bulatov, Y. Kuratov, M. Burtsev. Recurrent memory transformer. NeurIPS 2022. arXiv:2207.06881.

[138] Z. Dai et al. Transformer-XL: attentive language models beyond a fixed-length context. ACL 2019. arXiv:1901.02860.

[139] J. Rae et al. Compressive transformers for long-range sequence modelling. ICLR 2020. arXiv:1911.05507.

[140] T. Munkhdalai, M. Faruqui, S. Gopal. Leave no context behind: efficient infinite context transformers with Infini-attention. 2024. arXiv:2404.07143.

[141] Y. Wu et al. Memorizing transformers. ICLR 2022. arXiv:2203.08913.

[142] A. Behrouz, P. Zhong, V. Mirrokni. Titans: learning to memorize at test time. 2024. arXiv:2501.00663.

[143] Y. Sun et al. Learning to (learn at test time): RNNs with expressive hidden states. 2024. arXiv:2407.04620.

[144] W. Sun et al. Scaling long-horizon LLM agent via context-folding. 2025. arXiv:2510.11967.

[145] A. Semenov, S. Dorofeev. Beyond compaction: structured context eviction for long-horizon agents. 2026. arXiv:2606.11213.

[146] C. Packer et al. MemGPT: towards LLMs as operating systems. 2023. arXiv:2310.08560.

[147] J. Wu et al. Recursively summarizing books with human feedback. 2021. arXiv:2109.10862.

[148] J. Su et al. RoFormer: enhanced transformer with rotary position embedding. 2021. arXiv:2104.09864.

[149] O. Press, N. Smith, M. Lewis. Train short, test long: attention with linear biases enables input length extrapolation. ICLR 2022. arXiv:2108.12409.

[150] S. Chen et al. Extending context window of large language models via positional interpolation. 2023. arXiv:2306.15595.

[151] B. Peng et al. YaRN: efficient context window extension of large language models. ICLR 2024. arXiv:2309.00071.

[152] Y. Ding et al. LongRoPE: extending LLM context window beyond 2 million tokens. ICML 2024. arXiv:2402.13753.

[153] H. Jin et al. LLM maybe LongLM: self-extend LLM context window without tuning. ICML 2024. arXiv:2401.01325.

[154] W. Xiong et al. Effective long-context scaling of foundation models. 2023. arXiv:2309.16039.

[155] T. Gao et al. How to train long-context language models (effectively). 2024. arXiv:2410.02660.

[156] N. Liu et al. Lost in the middle: how language models use long contexts. TACL 2024. arXiv:2307.03172.

[157] C.-P. Hsieh et al. RULER: what's the real context size of your long-context language models? COLM 2024. arXiv:2404.06654.

[158] G. Kamradt. Needle in a haystack: pressure testing LLMs. GitHub repository, 2023.

[159] Y. Bai et al. LongBench: a bilingual, multitask benchmark for long context understanding. ACL 2024. arXiv:2308.14508.

[160] Y. Bai et al. LongBench v2: towards deeper understanding and reasoning on realistic long-context multitasks. 2024. arXiv:2412.15204.

[161] X. Zhang et al. ∞Bench: extending long context evaluation beyond 100K tokens. ACL 2024. arXiv:2402.13718.

[162] H. Yen et al. HELMET: how to evaluate long-context language models effectively and thoroughly. ICLR 2025. arXiv:2410.02694.

[163] Y. Kuratov et al. BABILong: testing the limits of LLMs with long context reasoning-in-a-haystack. NeurIPS 2024. arXiv:2406.10149.

[164] A. Modarressi et al. NoLiMa: long-context evaluation beyond literal matching. ICML 2025. arXiv:2502.05167.

[165] K. Vodrahalli et al. Michelangelo: long context evaluations beyond haystacks via latent structure queries. 2024. arXiv:2409.12640.

[166] Y. Li et al. SCBench: a KV cache-centric analysis of long-context methods. ICLR 2025. arXiv:2412.10319.

[167] J. Yuan et al. KV cache compression, but what must we give in return? A comprehensive benchmark of long context capable approaches. Findings of EMNLP 2024. arXiv:2407.01527.

[168] C. Qiu et al. KVDiagnosis: a diagnostic benchmark for KV-cache compression in long-context language models. 2026. arXiv:2608.09412.

[169] L. Shi et al. Keep the cost down: a review on methods to optimize LLM's KV-cache consumption. 2024. arXiv:2407.18003.

[170] H. Li et al. A survey on large language model acceleration based on KV cache management. 2024. arXiv:2412.19442.

[171] Z. Zhou et al. A survey on efficient inference for large language models. 2024. arXiv:2404.14294.

[172] Key, value, compress: a systematic exploration of KV cache compression techniques. 2025. arXiv:2503.11816.

[173] Y. Tay et al. Efficient transformers: a survey. ACM Comput. Surv. 2022. arXiv:2009.06732.

[174] N. Kitaev, L. Kaiser, A. Levskaya. Reformer: the efficient transformer. ICLR 2020. arXiv:2001.04451.

[175] I. Han et al. HyperAttention: long-context attention in near-linear time. ICLR 2024. arXiv:2310.05869.

[176] J. Alman, Z. Song. Fast attention requires bounded entries. NeurIPS 2023. arXiv:2302.13214.

[177] F. D. Keles, P. M. Wijewardena, C. Hegde. On the computational complexity of self-attention. ALT 2023. arXiv:2209.04881.

[178] C. Sanford, D. Hsu, M. Telgarsky. Representational strengths and limitations of transformers. NeurIPS 2023. arXiv:2306.02896.

[179] K. Choromanski et al. Rethinking attention with Performers. ICLR 2021. arXiv:2009.14794.

[180] S. Wang et al. Linformer: self-attention with linear complexity. 2020. arXiv:2006.04768.

[181] S. Jelassi et al. Repeat after me: transformers are better than state space models at copying. ICML 2024. arXiv:2402.01032.

[182] S. Arora et al. Zoology: measuring and improving recall in efficient language models. ICLR 2024. arXiv:2312.04927.

[183] S. Arora et al. Simple linear attention language models balance the recall-throughput tradeoff. ICML 2024. arXiv:2402.18668.

[184] W. Merrill, J. Petty, A. Sabharwal. The illusion of state in state-space models. ICML 2024. arXiv:2404.08819.

[185] K. Wen et al. RNNs are not transformers (yet): the key bottleneck on in-context retrieval. 2024. arXiv:2402.18510.

[186] M. Poli et al. Hyena hierarchy: towards larger convolutional language models. ICML 2023. arXiv:2302.10866.

[187] C. Olsson et al. In-context learning and induction heads. Transformer Circuits, 2022. arXiv:2209.11895.

[188] W. Wu et al. Retrieval head mechanistically explains long-context factuality. ICLR 2025. arXiv:2404.15574.

[189] M. Sun et al. Massive activations in large language models. COLM 2024. arXiv:2402.17762.

[190] X. Gu et al. When attention sink emerges in language models: an empirical view. ICLR 2025. arXiv:2410.10781.

[191] P. Lewis et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. NeurIPS 2020. arXiv:2005.11401.

[192] Z. Li et al. Retrieval augmented generation or long-context LLMs? A comprehensive study and hybrid approach. EMNLP Industry 2024. arXiv:2407.16833.

[193] P. Xu et al. Retrieval meets long context large language models. ICLR 2024. arXiv:2310.03025.

[194] T. Yu, A. Xu, R. Akkiraju. In defense of RAG in the era of long-context language models. 2024. arXiv:2409.01666.

[195] J. Wei et al. Chain-of-thought prompting elicits reasoning in large language models. NeurIPS 2022. arXiv:2201.11903.

[196] F. Perez, I. Ribeiro. Ignore previous prompt: attack techniques for language models. 2022. arXiv:2211.09527.

[197] K. Greshake et al. Not what you've signed up for: compromising real-world LLM-integrated applications with indirect prompt injection. 2023. arXiv:2302.12173.

[198] E. Wallace et al. The instruction hierarchy: training LLMs to prioritize privileged instructions. 2024. arXiv:2404.13208.

[199] J. Alman, Z. Song. The fine-grained complexity of gradient computation for training large language models. NeurIPS 2024. arXiv:2402.04497.

[200] C. Eckart, G. Young. The approximation of one matrix by another of lower rank. Psychometrika 1(3), 1936.

[201] L. Mirsky. Symmetric gauge functions and unitarily invariant norms. Q. J. Math. 11(1), 1960.

[202] W. B. Johnson, J. Lindenstrauss. Extensions of Lipschitz mappings into a Hilbert space. Contemp. Math. 26, 1984.

[203] P. Indyk, R. Motwani. Approximate nearest neighbors: towards removing the curse of dimensionality. STOC 1998.

[204] J. Misra, D. Gries. Finding repeated elements. Sci. Comput. Programming 2(2), 1982.

[205] A. Metwally, D. Agrawal, A. El Abbadi. Efficient computation of frequent and top-k elements in data streams. ICDT 2005.

[206] G. Cormode, S. Muthukrishnan. An improved data stream summary: the count-min sketch and its applications. J. Algorithms 55(1), 2005.

[207] T. Cover, J. Thomas. Elements of Information Theory, 2nd ed. Wiley, 2006.

[208] C. E. Shannon. A mathematical theory of communication. Bell Syst. Tech. J. 27, 1948.

[209] N. Tishby, F. Pereira, W. Bialek. The information bottleneck method. 2000. arXiv:physics/0004057.

[210] S. Lloyd. Least squares quantization in PCM. IEEE Trans. Inf. Theory 28(2), 1982.

[211] A. Bick et al. Transformers to SSMs: distilling quadratic knowledge to subquadratic models. NeurIPS 2024. arXiv:2408.10189.

[212] NVIDIA. Nemotron 3 Nano: a new standard for efficient, open, and intelligent agentic models. Hugging Face blog, 2025.

[213] Hugging Face. DeepSeek-V4: a million-token context that agents can actually use. Blog, 2026.

[214] M. Labonne. Qwen3.5: nobody agrees on attention anymore. Hugging Face blog, 2026.

[215] S. Raschka. Beyond standard LLMs. Ahead of AI magazine, 2025.

[216] C. Anil et al. Many-shot jailbreaking. NeurIPS 2024 (Anthropic).

[217] vLLM Project. vLLM: easy, fast, and cheap LLM serving with PagedAttention. Project blog and documentation, 2023.